

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG HỆ THỐNG PHÁT HIỆN VI PHẠM VƯỢT ĐÈN ĐỎ

Red-Light Violation Detection (RLVD)

Kiến trúc ba tầng: Node biên — Máy chủ trung tâm — Bảng điều khiển web

Ngành: Kỹ thuật phần mềm

Tháng 9 năm 2026

LỜI CẢM ƠN

Nhóm thực hiện xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã định hướng đề tài, góp ý về kiến trúc hệ thống và kiên nhẫn theo sát từng giai đoạn của đồ án. Những nhận xét phản biện của thầy/cô ở các buổi nghiệm thu trung gian đã giúp nhóm điều chỉnh nhiều quyết định thiết kế quan trọng, trong đó có việc chuyển từ mô hình hàng đợi trung gian sang mẫu outbox bền vững tại node biên.

Nhóm cũng xin cảm ơn cộng đồng mã nguồn mở đã cung cấp các công cụ nền tảng mà đồ án kế thừa: Ultralytics (YOLO), supervision (ByteTrack), fast-plate-ocr, Spring Boot, Next.js, PostgreSQL, MinIO và FFmpeg. Việc được xây dựng trên những thư viện trưởng thành cho phép nhóm tập trung vào phần đóng góp riêng: logic xét vi phạm, cơ chế giao hàng không mất dữ liệu và quy trình duyệt hồ sơ có sự tham gia của con người.

Cuối cùng, nhóm cảm ơn các bạn đã hỗ trợ thu thập video tại một số nút giao thông để phục vụ huấn luyện và kiểm thử, cùng gia đình đã tạo điều kiện về thời gian và thiết bị trong suốt quá trình thực hiện.

TÓM TẮT

Đồ án nghiên cứu và xây dựng hệ thống phần mềm phát hiện hành vi vượt đèn đỏ của phương tiện giao thông từ luồng video giám sát tại nút giao, theo kiến trúc phân tán ba tầng: node biên thực hiện toàn bộ phân tích thị giác máy tính ngay tại vị trí camera; máy chủ trung tâm lưu trữ hồ sơ vi phạm, quản lý người dùng và cung cấp giao diện lập trình ứng dụng; bảng điều khiển web phục vụ giám sát, hiệu chuẩn từ xa và duyệt hồ sơ.

Tại node biên, đồ án xây dựng một pipeline thị giác hoàn chỉnh: mô hình YOLO26m được huấn luyện lại trên dữ liệu phương tiện Việt Nam đạt mAP50 = 0,944 với tốc độ 73,4 khung hình/giây ở độ chính xác nửa (FP16) trên GPU RTX 2060; ByteTrack được tinh chỉnh tham số để giữ liên tục vết của xe máy ở tốc độ khung hình thấp; bộ phân loại đèn tín hiệu kết hợp mô hình học sâu với bảng chứng màu HSV và vị trí vật lý của bóng đèn; bộ đọc biển số tự động kèm công kiểm tra cấu trúc theo Thông tư 24/2023/TT-BGTVT. Việc xét vi phạm dựa trên vạch dừng ảo có hướng, trạng thái đèn đã ổn định nhờ cơ chế debounce có trễ (hysteresis) và điểm neo bottom-center của phương tiện.

Đóng góp đáng chú ý nhất về mặt kiến trúc là cơ chế giao hàng bền vững: mỗi hồ sơ vi phạm cùng ảnh bằng chứng được ghi vào kho trung gian SQLite ngay tại thời điểm phát hiện, trước mọi thao tác mạng, và được một luồng nền đẩy lên trung tâm theo lô có tính idempotent. Nhờ đó hệ thống chỉ có thể trễ chứ không thể mất dữ liệu khi mất kết nối hoặc khởi động lại. Máy chủ trung tâm Spring Boot bảo vệ bằng JWT phân quyền ba vai trò, kèm token riêng cho node biên và cơ chế xoay refresh token có phát hiện việc dùng lại.

Kết quả kiểm thử: 124 kiểm thử đơn vị đạt toàn bộ trong 1,09 giây; biên dịch Java và TypeScript sạch; kiểm thử end-to-end trên video có nhãn cho kết quả đúng với ground-truth (một vi phạm thật được phát hiện trên video aziz1.MP4, không có vi phạm giả nào trên video đối chứng 384 khung hình). Toàn bộ hệ thống vận hành được bằng một lệnh Docker Compose với năm dịch vụ. Đồ án cũng ghi nhận trung thực các hạn chế: chưa kiểm thử tải HTTP, chưa có bộ kiểm thử tích hợp tự động cho tầng Java, và chưa có tập video biển số Việt Nam đủ lớn để đo độ chính xác OCR độc lập.

Từ khoá: thị giác máy tính, YOLO, ByteTrack, nhận dạng biển số, tính toán biên, kiến trúc ba tầng, Spring Boot, Next.js, Docker, PostgreSQL, MinIO, JWT, outbox.

ABSTRACT

This thesis designs and implements a software system that detects red-light running violations from surveillance video at road intersections, organised as a distributed three-tier architecture: an edge node performing all computer-vision analysis at the camera site, a central server storing violation records and exposing APIs, and a web dashboard for monitoring, remote calibration and human-in-the-loop review.

The edge pipeline combines a YOLO26m detector fine-tuned on Vietnamese traffic data (mAP50 = 0.944 at 73.4 FPS in FP16 on an RTX 2060), a parameter-tuned ByteTrack tracker that keeps motorbike tracks continuous at low frame rates, a traffic-light classifier fusing a deep model with HSV colour evidence and physical lamp position, and an automatic plate reader gated by a structural validator conforming to Circular 24/2023/TT-BGTVT. Violation decisions rely on a directional virtual stop line, a hysteresis-debounced light state and the bottom-center anchor of each vehicle.

The main architectural contribution is durable delivery: every violation record and its evidence image are written to a local SQLite outbox at detection time, before any network operation, and are then pushed to the centre in idempotent batches by a background thread. The system can therefore lag but cannot lose data across outages or restarts. The Spring Boot central server is protected by role-based JWT authentication, a separate ingest token for edge nodes, and refresh-token rotation with reuse detection.

Testing results: all 124 unit tests pass in 1.09 s; Java and TypeScript builds are clean; end-to-end runs on labelled video match ground truth (one true violation detected in aziz1.MP4, zero false positives across a 384-frame control video). The complete stack runs from a single Docker Compose command with five services. The thesis also reports its limitations honestly: no HTTP load testing, no automated integration test suite for the Java tier, and no sufficiently large Vietnamese plate-video corpus for an independent OCR accuracy measurement.

Keywords: computer vision, YOLO, ByteTrack, license plate recognition, edge computing, three-tier architecture, Spring Boot, Next.js, Docker, PostgreSQL, MinIO, JWT, outbox pattern.

MỤC LỤC

LỜI CẢM ƠN	2
TÓM TẮT	3
ABSTRACT	4
MỤC LỤC	5
DANH SÁCH HÌNH	8
DANH SÁCH BẢNG	10
DANH SÁCH TỪ VIẾT TẮT	12
CHƯƠNG 1. GIỚI THIỆU	14
1.1. Lý do chọn đề tài	14
1.2. Mục tiêu	15
1.3. Đối tượng nghiên cứu	15
1.4. Phạm vi	16
1.5. Phương pháp thực hiện	16
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH	19
2.1. Khảo sát hiện trạng	19
2.1.1. Quy trình xử lý vi phạm hiện nay	19
2.1.2. Hạn chế của quy trình thủ công	20
2.2. Các hệ thống liên quan	21
2.2.1. Hệ thống camera ứng dụng trí tuệ nhân tạo của lực lượng cảnh sát giao thông	21
2.2.2. Các giải pháp thương mại và xu hướng kỹ thuật	22
2.2.3. Các công trình và dự án mã nguồn mở liên quan	22
2.2.4. Vị trí của hệ thống RLVD	23
2.3. Yêu cầu chức năng	24
2.4. Yêu cầu phi chức năng	26
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG	29
3.1. Actors	29
3.2. Use Case Diagram	30
3.3. Use Case Specification	33
3.3.1. UC-01: Phát hiện xe vượt đèn đỏ	33
3.3.2. UC-02: Gửi hồ sơ vi phạm về trung tâm	34
3.3.3. UC-03: Hiệu chuẩn từ xa	36
3.3.4. UC-04: Duyệt hồ sơ vi phạm	37

3.3.5. UC-05: Hỏi dữ liệu bằng tiếng Việt.....	38
3.4. Activity Diagram.....	39
3.4.1. Hoạt động xét vi phạm tại node biên.....	39
3.4.2. Hoạt động duyệt hồ sơ.....	41
3.4.3. Hoạt động hiệu chuẩn từ xa.....	41
3.4.4. Máy trạng thái tín hiệu đèn.....	43
3.4.5. Máy trạng thái vòng đời hồ sơ vi phạm.....	44
3.5. Sequence Diagram.....	44
3.5.1. Luồng phân phối hồ sơ bền vững.....	44
3.5.2. Luồng xác thực và làm mới token.....	45
3.5.3. Luồng hiệu chuẩn từ xa.....	46
3.5.4. Luồng trợ lý hỏi dữ liệu.....	47
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG.....	48
4.1. System Architecture.....	48
4.2. Class Diagram.....	50
4.3. ERD.....	52
4.4. Database Design.....	54
4.5. API Design.....	57
4.6. UI/UX Design.....	62
CHƯƠNG 5. CÀI ĐẶT VÀ PROTOTYPE.....	66
5.1. Frontend.....	66
5.2. Backend.....	68
5.2.1. Node biên.....	68
5.2.2. Máy chủ trung tâm.....	71
5.3. Database.....	72
5.4. Demo.....	73
CHƯƠNG 6. KIỂM THỬ.....	82
6.1. Test Case.....	82
6.2. Functional Testing.....	85
6.2.1. Kiểm thử đơn vị.....	85
6.2.2. Kiểm thử chức năng trên hệ thống đang chạy.....	86
6.2.3. Kiểm thử phân quyền.....	87
6.3. Kiểm thử hiệu năng.....	87
6.4. Kiểm thử khâu nhận dạng biển số.....	89
6.5. Kiểm thử hồi quy trên video có nhãn.....	90

6.6. Kiểm thử cơ chế giao hàng không mất dữ liệu.....	<u>91</u>
6.7. Kiểm thử bảo mật.....	<u>92</u>
6.8. Kết quả.....	<u>93</u>
CHƯƠNG 7. KẾT LUẬN.....	<u>95</u>
7.1. Kết quả đạt được.....	<u>95</u>
7.2. Hạn chế.....	<u>97</u>
7.3. Hướng phát triển.....	<u>98</u>
TÀI LIỆU THAM KHẢO.....	<u>101</u>

DANH SÁCH HÌNH

- Hình 2.1 — Quy trình xử lý vi phạm vượt đèn đỏ hiện nay (as-is)
- Hình 2.2 — Quy trình đề xuất khi có hệ thống RLVD (to-be)
- Hình 3.1 — Quan hệ actor, vai trò và quyền hạn trong hệ thống
- Hình 3.2 — Sơ đồ use case tổng thể
- Hình 3.3 — Sơ đồ use case của Administrator
- Hình 3.4 — Sơ đồ use case của Officer
- Hình 3.5 — Sơ đồ use case của Operator
- Hình 3.6 — Sơ đồ hoạt động xét vi phạm tại node biên
- Hình 3.7 — Sơ đồ hoạt động duyệt hồ sơ vi phạm
- Hình 3.8 — Sơ đồ hoạt động hiệu chuẩn từ xa
- Hình 3.9 — Máy trạng thái tín hiệu đèn có debounce
- Hình 3.10 — Máy trạng thái vòng đời hồ sơ vi phạm
- Hình 3.11 — Sơ đồ sequence phân phối hồ sơ bền vững
- Hình 3.12 — Sơ đồ sequence xác thực và làm mới token
- Hình 3.13 — Sơ đồ sequence hiệu chuẩn vạch dừng từ xa
- Hình 3.14 — Sơ đồ sequence trợ lý hỏi dữ liệu bằng tiếng Việt
- Hình 4.1 — Kiến trúc hệ thống ba tầng
- Hình 4.2 — Sơ đồ thành phần và giao diện giữa các tầng
- Hình 4.3 — Sơ đồ triển khai bằng Docker Compose với năm dịch vụ
- Hình 4.4 — Sơ đồ lớp của hệ thống
- Hình 4.5 — Mô hình thực thể quan hệ của cơ sở dữ liệu rlvd_central
- Hình 4.6 — Luồng dữ liệu hồ sơ vi phạm qua ba tầng
- Hình 4.7 — Các lớp bảo vệ của hệ thống theo nhóm yêu cầu
- Hình 4.8 — Kiến trúc thông tin của bảng điều khiển web
- Hình 4.9 — Wireflow nghiệp vụ duyệt hồ sơ của cán bộ xử lý vi phạm
- Hình 5.1 — Trang đăng nhập
- Hình 5.2 — Trang bảng điều khiển tổng quan sau khi đăng nhập
- Hình 5.3 — Trang tra cứu hồ sơ vi phạm có phân trang và lọc
- Hình 5.4 — Trang chi tiết hồ sơ với ảnh bằng chứng và nút quyết định xử lý
- Hình 5.5 — Trang duyệt hồ sơ hàng loạt
- Hình 5.6 — Trang xem camera trực tiếp với các bố cục lưới 1, 4, 9 và 16

Hình 5.7 — Trang danh sách node biên

Hình 5.8 — Trang chi tiết node với luồng video trực tiếp

Hình 5.9 — Công cụ hiệu chuẩn vạch dừng trên video trực tiếp

Hình 5.10 — Trợ lý hỏi dữ liệu bằng tiếng Việt

Hình 5.11 — Trang cài đặt giao diện

DANH SÁCH BẢNG

Bảng 0.1 — Danh mục từ viết tắt sử dụng trong đồ án

Bảng 1.1 — Các giai đoạn thực hiện đồ án

Bảng 2.1 — Hạn chế của quy trình xử lý thủ công

Bảng 2.2 — So sánh hệ thống RLVD với các giải pháp liên quan

Bảng 2.3 — Yêu cầu chức năng

Bảng 2.4 — Yêu cầu phi chức năng

Bảng 3.1 — Danh sách actor và quyền hạn

Bảng 3.2 — Đặc tả UC-01 Phát hiện xe vượt đèn đỏ

Bảng 3.3 — Đặc tả UC-02 Gửi hồ sơ vi phạm về trung tâm

Bảng 3.4 — Đặc tả UC-03 Hiệu chuẩn từ xa

Bảng 3.5 — Đặc tả UC-04 Duyệt hồ sơ vi phạm

Bảng 3.6 — Đặc tả UC-05 Hỏi dữ liệu bằng tiếng Việt

Bảng 4.1 — Thiết kế bảng violations

Bảng 4.2 — Thiết kế bảng edge_nodes

Bảng 4.3 — Thiết kế bảng users và refresh_tokens

Bảng 4.4 — Lược đồ kho outbox tại node biên (SQLite)

Bảng 4.5 — Quy ước thiết kế API

Bảng 4.6 — Danh sách nhóm API của máy chủ trung tâm

Bảng 4.7 — Danh sách API điều khiển tại node biên

Bảng 4.8 — Mã trạng thái HTTP và ý nghĩa trong hệ thống

Bảng 4.9 — Nguyên tắc thiết kế giao diện và cách thể hiện

Bảng 5.1 — Quy mô mã nguồn theo tầng

Bảng 5.2 — Các trang của bảng điều khiển web

Bảng 5.3 — Các mô-đun chính của tầng web

Bảng 5.4 — Các module chính của node biên

Bảng 5.5 — Các lớp chính của máy chủ trung tâm

Bảng 5.6 — Ảnh xạ cổng dịch vụ khi triển khai bằng Docker Compose

Bảng 5.7 — Biến môi trường cấu hình hệ thống

Bảng 6.1 — Danh sách ca kiểm thử

Bảng 6.2 — Kết quả kiểm thử đơn vị theo tệp

Bảng 6.3 — Kết quả đánh giá mô hình phát hiện phương tiện

Bảng 6.4 — Kết quả đánh giá mô hình phân loại đèn và mô hình phát hiện biển số

Bảng 6.5 — Kết quả đo thời gian xử lý khâu biển số

Bảng 6.6 — Kết quả đo khâu nhận dạng biển số

Bảng 6.7 — Kết quả kiểm thử hồi quy trên video có nhãn

Bảng 6.8 — Tổng hợp kết quả kiểm thử

Bảng 7.1 — Đối chiếu mục tiêu và kết quả đạt được

Bảng 7.2 — Hạn chế của đồ án theo mức độ ảnh hưởng

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	Nghĩa đầy đủ
RLVD	Red-Light Violation Detection — phát hiện vi phạm vượt đèn đỏ
ANPR	Automatic Number Plate Recognition — nhận dạng biển số tự động
YOLO	You Only Look Once — họ mô hình phát hiện đối tượng thời gian thực
mAP	mean Average Precision — độ chính xác trung bình, chỉ số đánh giá mô hình phát hiện
mAP50	mAP tính tại ngưỡng IoU = 0,50
IoU	Intersection over Union — tỉ lệ giao trên hợp của hai khung giới hạn
MOT	Multi-Object Tracking — theo dõi đa đối tượng
OCR	Optical Character Recognition — nhận dạng ký tự quang học
FP16	Half Precision Floating Point — số thực dấu phẩy động 16 bit
FPS	Frames Per Second — số khung hình xử lý mỗi giây
ROI	Region of Interest — vùng quan tâm
HSV	Hue–Saturation–Value — không gian màu sắc độ, độ bão hoà, độ sáng
EMA	Exponential Moving Average — trung bình trượt theo hàm mũ
API	Application Programming Interface — giao diện lập trình ứng dụng
REST	Representational State Transfer — phong cách thiết kế API
DTO	Data Transfer Object — đối tượng truyền dữ liệu giữa các tầng
ERD	Entity Relationship Diagram — sơ đồ thực thể quan hệ
CSDL	Cơ sở dữ liệu
JPA	Jakarta Persistence API — chuẩn ánh xạ đối tượng – CSDL của Java
JWT	JSON Web Token — token xác thực có chữ ký số
RBAC	Role-Based Access Control — phân quyền theo

	vai trò
BCrypt	Hàm băm mật khẩu thích ứng dựa trên Blowfish
HLS	HTTP Live Streaming — giao thức phát video phân đoạn qua HTTP
RTSP	Real-Time Streaming Protocol — giao thức truyền video thời gian thực
FR	Functional Requirement — yêu cầu chức năng
NFR	Non-Functional Requirement — yêu cầu phi chức năng
UC	Use Case — ca sử dụng
CI	Continuous Integration — tích hợp liên tục
Outbox	Kho trung gian bền vững đặt phía node sản sinh dữ liệu (mẫu outbox)
TT 24/2023	Thông tư 24/2023/TT-BGTVT về cấp, thu hồi đăng ký và biển số xe cơ giới

Bảng 0.1. Danh mục từ viết tắt sử dụng trong đồ án

CHƯƠNG 1. GIỚI THIỆU

1.1. Lý do chọn đề tài

Không chấp hành tín hiệu đèn giao thông là một trong những hành vi vi phạm phổ biến và nguy hiểm nhất tại Việt Nam, bởi xung đột xảy ra đúng ở vị trí các luồng giao cắt có tốc độ khác nhau và hầu như không có cơ hội tránh né. Hậu quả thường là va chạm ngang hông với mức độ thương tích cao. Trong khi đó, việc phát hiện hành vi này hiện vẫn dựa chủ yếu vào hai hình thức: cảnh sát giao thông trực tiếp tại chốt, hoặc xem lại video ghi hình một cách thủ công.

Cả hai hình thức đều có giới hạn cố hữu. Giám sát trực tiếp chỉ phủ được số lượng nút giao bằng số tổ công tác có thể bố trí, phụ thuộc thời tiết, ca kíp và sự hiện diện của lực lượng chức năng — nghĩa là hành vi vi phạm dễ tái diễn ngay khi vắng bóng người kiểm soát. Xem lại video thủ công tuy bao phủ rộng hơn nhưng tốn rất nhiều nhân lực: một cán bộ phải quan sát hàng giờ footage để tìm vài chục giây có vi phạm, và kết quả phụ thuộc vào sự tập trung của người xem. Ngoài ra, cả hai hình thức đều tạo ra bằng chứng ở dạng mô tả, khó chuẩn hoá và khó đối chiếu tự động với cơ sở dữ liệu đăng ký xe.

Về mặt kỹ thuật, hai điều kiện để tự động hoá bài toán này đã chín muồi. Thứ nhất, các mô hình phát hiện đối tượng họ YOLO đã đạt đồng thời độ chính xác và tốc độ đủ để xử lý video thời gian thực trên phần cứng phổ thông, không còn yêu cầu hạ tầng chuyên dụng. Thứ hai, mô hình tính toán biên cho phép đặt năng lực xử lý ngay tại vị trí camera: chỉ hồ sơ vi phạm vài kilobyte kèm một ảnh bằng chứng cần truyền về trung tâm, thay vì toàn bộ luồng video liên tục vốn đòi hỏi băng thông lớn và không mở rộng được theo số nút giao.

Tuy nhiên, một hệ thống hoàn chỉnh không dừng lại ở việc nhận diện được phương tiện và màu đèn. Những khó khăn thực sự nằm ở phần tích hợp: đọc biển số đúng chuẩn Việt Nam và loại được kết quả OCR sai; xác định đúng thời điểm vi phạm để không bỏ sót nhưng cũng không tạo hồ sơ giả; truyền hồ sơ về trung tâm một cách tin cậy trong điều kiện mạng không ổn định; xây dựng quy trình duyệt có sự tham gia của con người để bảo đảm giá trị pháp lý; và cung cấp công cụ hiệu chuẩn từ xa để một kỹ thuật viên có thể cấu hình vạch dừng cho nhiều nút giao mà không cần đến tận nơi. Chính yêu cầu tích hợp đa tầng này là lý do nhóm lựa chọn đề tài: bài toán đủ hẹp để hoàn thiện trọn vẹn trong khuôn khổ đồ án, nhưng đủ phức tạp để vận dụng kiến thức của nhiều học phần — thị giác máy tính, kiến trúc phần mềm, cơ sở dữ liệu, an toàn thông tin và phát triển giao diện người dùng.

1.2. Mục tiêu

Mục tiêu tổng quát của đồ án là xây dựng một hệ thống phần mềm phân tán có khả năng tự động phát hiện phương tiện vượt đèn đỏ từ luồng video giám sát, lập hồ sơ vi phạm kèm ảnh bằng chứng và biển số xe, sau đó phục vụ tra cứu, thống kê và duyệt xử lý thông qua giao diện web.

Từ mục tiêu tổng quát, đồ án xác định các mục tiêu cụ thể sau:

- Xây dựng pipeline thị giác máy tính chạy thời gian thực tại node biên, gồm bốn khâu: phát hiện phương tiện, theo dõi đa đối tượng, nhận biết trạng thái đèn tín hiệu và xét vi phạm theo vạch dừng ảo có hướng.
- Huấn luyện lại và đánh giá định lượng các mô hình học sâu trên dữ liệu giao thông Việt Nam, có đối chiếu với mô hình gốc huấn luyện trên tập COCO để xác định mức độ cần thiết của việc huấn luyện lại.
- Thiết kế bộ đọc biển số tự động kèm cổng kiểm tra cấu trúc biển Việt Nam theo Thông tư 24/2023/TT-BGTVT, có khả năng sửa các lỗi OCR thường gặp.
- Thiết kế cơ chế giao hàng bền vững bảo đảm hai tính chất: không mất hồ sơ khi mất kết nối hoặc khởi động lại, và không tạo hồ sơ trùng khi gửi lại.
- Xây dựng máy chủ trung tâm cung cấp REST API có xác thực và phân quyền theo ba vai trò, lưu hồ sơ trong PostgreSQL và ảnh bằng chứng trong MinIO.
- Xây dựng bảng điều khiển web phục vụ thống kê, tra cứu, duyệt hồ sơ, xem camera trực tiếp và hiệu chuẩn vạch dừng từ xa với hiệu lực tức thời.
- Tích hợp trợ lý cho phép đặt câu hỏi bằng tiếng Việt và nhận kết quả dưới dạng bảng hoặc biểu đồ, với cơ chế chặn câu lệnh SQL nguy hiểm.
- Kiểm thử hệ thống ở nhiều mức và đóng gói toàn bộ để vận hành bằng một lệnh.

Mục tiêu được coi là đạt khi hệ thống chạy thông suốt từ đầu vào video đến hồ sơ hiển thị trên giao diện web, các chỉ tiêu hiệu năng đo được nằm trong ngưỡng chấp nhận, và toàn bộ kiểm thử đã viết đều cho kết quả đạt.

1.3. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đồ án gồm hai nhóm: nhóm đối tượng nghiệp vụ mà hệ thống nhận biết, và nhóm đối tượng kỹ thuật mà đồ án tập trung giải quyết.

Về nghiệp vụ, hệ thống làm việc với bốn loại đối tượng sau:

- Phương tiện đường bộ thuộc bốn lớp: ô tô con, xe mô tô và xe gắn máy, xe van/xe khách, xe tải. Trong đó xe mô tô là lớp khó nhất vì kích thước nhỏ trong khung hình, dễ bị che khuất và chiếm tỉ trọng lớn nhất trong dữ liệu thực tế.
- Tín hiệu đèn giao thông ba trạng thái đỏ, vàng, xanh, quan sát qua một vùng quan tâm cố định trong khung hình do người vận hành hiệu chuẩn.

- Biển số xe cơ giới Việt Nam một dòng, cấu trúc gồm hai chữ số mã tỉnh từ 11 đến 99, tiếp theo là ký tự seri và bốn đến năm chữ số, theo quy định tại Thông tư 24/2023/TT-BGTVT.
- Hành vi vi phạm, được định nghĩa hình học là việc phương tiện cắt qua vạch dừng theo hướng giám sát trong khi trạng thái đèn đỏ đã ổn định.

Về kỹ thuật, đồ án tập trung nghiên cứu các vấn đề sau: kiến trúc phân tán theo mô hình tính toán biên; mẫu thiết kế outbox cho giao hàng bền vững trong điều kiện mạng không tin cậy; cơ chế debounce có trễ để ổn định tín hiệu nhiễu; bài toán domain-shift khi mô hình đạt độ chính xác cao trên tập kiểm định nhưng giảm đáng kể trên dữ liệu thực tế; và mô hình xác thực không trạng thái kết hợp phân quyền theo vai trò cho cả người dùng lẫn tác nhân máy.

1.4. Phạm vi

Phạm vi thực hiện:

- Nguồn video đầu vào là tệp MP4 được phát lập để mô phỏng camera, hoặc luồng RTSP; camera cố định, giám sát một hướng của một nút giao.
- Node biên chạy trên một máy trạm có GPU NVIDIA, đóng gói bằng Docker.
- Một máy chủ trung tâm duy nhất; kiến trúc được thiết kế để tiếp nhận nhiều node biên nhưng quá trình kiểm thử chỉ thực hiện với một node.
- Ba vai trò người dùng: Administrator, Operator và Officer.
- Loại vi phạm được xét là vượt đèn đỏ; hồ sơ tạo ra ở trạng thái chờ duyệt và do con người ra quyết định cuối cùng.
- Giao diện web hỗ trợ tra cứu, thống kê, duyệt, xem camera trực tiếp, hiệu chuẩn và hỏi dữ liệu bằng tiếng Việt.

Ngoài phạm vi thực hiện:

- Không tự động ra quyết định xử phạt hành chính hay sinh biên bản có giá trị pháp lý; hệ thống chỉ lập hồ sơ và cung cấp bằng chứng.
- Không xử lý luồng giao thông nhiều hướng từ cùng một camera; mỗi node biên giám sát một hướng.
- Không tích hợp với cơ sở dữ liệu đăng ký xe quốc gia để xác định chủ phương tiện; đây là bước đối soát thuộc hệ thống nghiệp vụ khác.
- Chưa kiểm chứng ở quy mô lớn trong điều kiện thời tiết xấu, ban đêm không có chiếu sáng bổ trợ, hoặc ngược sáng mạnh.
- Chưa đo tải HTTP với nhiều người dùng và nhiều node đồng thời.

1.5. Phương pháp thực hiện

Đồ án áp dụng phương pháp phát triển lặp và tăng dần: mỗi chức năng được cài đặt ở mức tối thiểu, đo lường bằng số liệu thực, rồi tinh chỉnh dựa trên kết quả đo. Nguyên

tắc xuyên suốt là mọi khẳng định về chất lượng đều phải có bằng chứng đo lường hoặc kiểm thử đi kèm; những hạng mục chưa đo được sẽ được ghi nhận là chưa đo thay vì ước lượng.

Quy trình thực hiện được chia thành các giai đoạn sau:

Giai đoạn	Nội dung công việc	Sản phẩm
1. Khảo sát	Tìm hiểu quy trình xử lý vi phạm hiện nay; khảo sát các hệ thống camera giám sát đã triển khai tại Việt Nam; nghiên cứu tài liệu học thuật về phát hiện đối tượng, theo dõi đa đối tượng và nhận dạng biển số.	Phân tích hiện trạng, bảng yêu cầu chức năng và phi chức năng
2. Phân tích	Xác định actor và quyền hạn; xây dựng sơ đồ use case; đặc tả các use case quan trọng; mô hình hoá luồng xử lý bằng sơ đồ hoạt động và máy trạng thái.	Bộ sơ đồ use case, activity, state machine
3. Thiết kế	Thiết kế kiến trúc ba tầng; xây dựng sơ đồ lớp; thiết kế mô hình thực thể quan hệ và lược đồ CSDL; thiết kế bề mặt API; thiết kế kiến trúc thông tin và wireflow cho giao diện web.	Tài liệu thiết kế, ERD, đặc tả API, thiết kế UI/UX
4. Huấn luyện mô hình	Chuẩn bị và gán nhãn dữ liệu; huấn luyện lại YOLO26m cho bốn lớp phương tiện, YOLO26n-cls cho phân loại đèn, mô hình phát hiện biển số; đánh giá bằng precision, recall, mAP50, mAP50-95 và đối chiếu với mô hình gốc.	Ba tệp trọng số và bảng số liệu benchmark
5. Cài đặt	Lập trình node biên bằng Python; máy chủ trung tâm bằng Java với Spring Boot; bảng điều khiển web bằng TypeScript với Next.js; đóng gói toàn bộ bằng Docker Compose.	Mã nguồn ba tầng, tập tin cấu hình triển khai
6. Kiểm thử	Viết kiểm thử đơn vị cho logic nghiệp vụ; chạy end-to-end trên video có nhãn và video đối chứng; kiểm thử tích hợp và phân quyền trên stack đang	Bộ kiểm thử, báo cáo kết quả

	chạy; đo hiệu năng.	
7. Hoàn thiện	Chụp ảnh minh hoạ giao diện; dựng lại các sơ đồ cho khớp mã nguồn hiện hành; viết báo cáo.	Báo cáo đồ án, bộ hình minh hoạ

Bảng 1.1. Các giai đoạn thực hiện đồ án

Về công cụ, đồ án sử dụng Git để quản lý phiên bản theo quy ước Conventional Commits; GitHub Actions để chạy kiểm thử và biên dịch tự động ở ba tầng mỗi khi có thay đổi; pytest cho kiểm thử đơn vị phần Python; PlantUML và Mermaid để dựng sơ đồ; Docker và Docker Compose để đóng gói và vận hành.

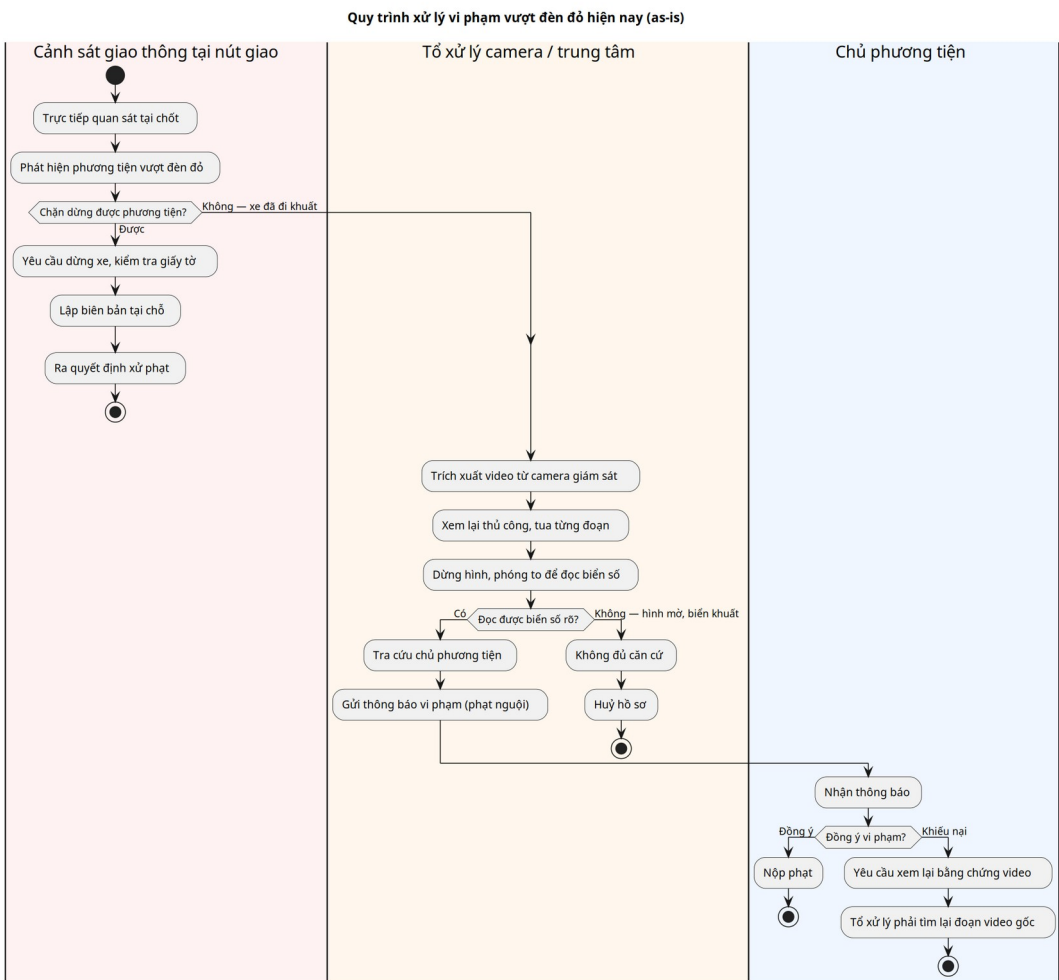
Cấu trúc của báo cáo gồm bảy chương. Chương 1 trình bày lý do chọn đề tài, mục tiêu, đối tượng, phạm vi và phương pháp thực hiện. Chương 2 khảo sát hiện trạng, các hệ thống liên quan và xác định yêu cầu. Chương 3 phân tích hệ thống thông qua actor, use case, sơ đồ hoạt động và sơ đồ sequence. Chương 4 trình bày thiết kế kiến trúc, sơ đồ lớp, ERD, thiết kế CSDL, thiết kế API và thiết kế giao diện. Chương 5 mô tả quá trình cài đặt và trình diễn hệ thống. Chương 6 trình bày các ca kiểm thử và kết quả đo được. Chương 7 kết luận về kết quả đạt được, hạn chế và hướng phát triển.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH

2.1. Khảo sát hiện trạng

2.1.1. Quy trình xử lý vi phạm hiện nay

Quy trình xử lý hành vi vượt đèn đỏ tại Việt Nam hiện diễn ra theo hai nhánh song song. Nhánh thứ nhất là xử lý trực tiếp: cảnh sát giao thông đứng tại nút giao, quan sát bằng mắt, ra hiệu lệnh dừng phương tiện, kiểm tra giấy tờ và lập biên bản tại chỗ. Nhánh thứ hai là xử lý qua ghi hình, thường gọi là phạt nguội: hình ảnh được thu thập từ camera giám sát hoặc thiết bị ghi hình của lực lượng chức năng, sau đó được xem lại để trích xuất các trường hợp vi phạm, đối chiếu biển số với cơ sở dữ liệu đăng ký xe, xác định chủ phương tiện và gửi thông báo.



Hình 2.1. Quy trình xử lý vi phạm vượt đèn đỏ hiện nay (as-is)

Điểm nghẽn của cả hai nhánh nằm ở khâu nhận biết và trích xuất. Ở nhánh trực tiếp, năng lực phát hiện bị giới hạn bởi số lượng cán bộ có thể bố trí và tầm quan sát tại một thời điểm; một nút giao rộng có nhiều hướng xung đột thì một tổ công tác khó bao

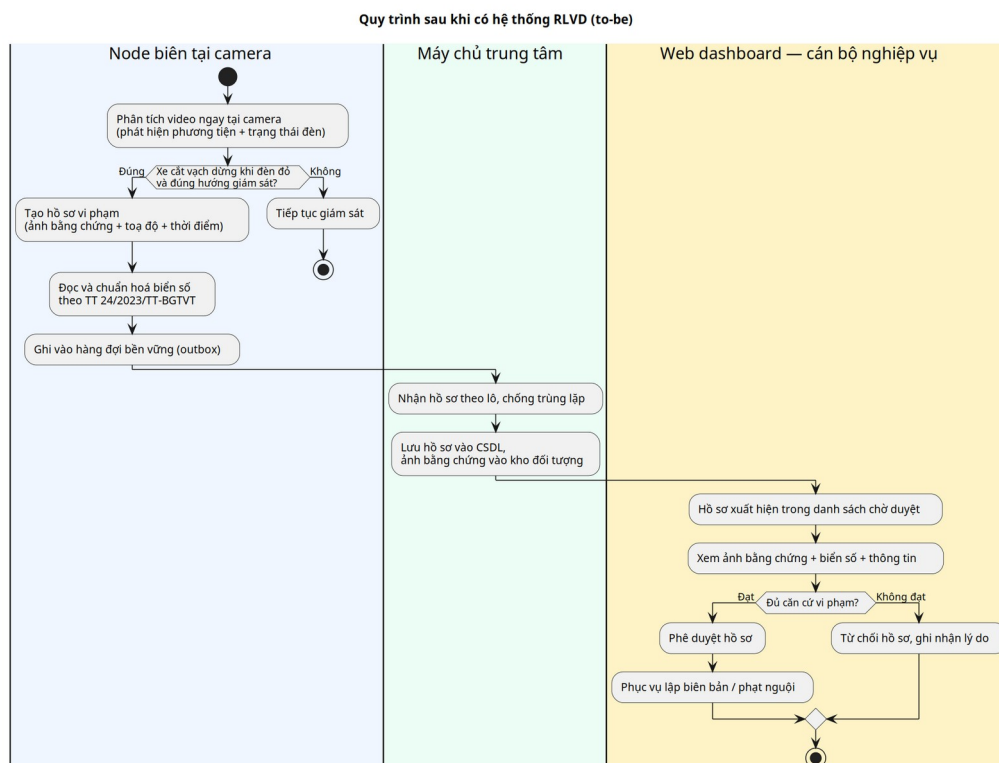
quát hết. Ở nhánh phạt nguội, khối lượng video cần xem lại rất lớn trong khi tỉ lệ khung hình thực sự chứa vi phạm lại nhỏ, khiến công sức bỏ ra không tỉ lệ với số hồ sơ lập được.

2.1.2. Hạn chế của quy trình thủ công

Nhóm hạn chế	Biểu hiện cụ thể	Hệ quả
Độ bao phủ	Chỉ giám sát được nơi có cán bộ trực hoặc nơi có camera được xem lại; phụ thuộc ca kíp, thời tiết và quân số.	Tỉ lệ vi phạm được phát hiện thấp hơn nhiều so với thực tế; hành vi dễ tái diễn khi vắng lực lượng chức năng.
Năng suất lao động	Một cán bộ phải xem hàng giờ video để tìm vài chục giây có vi phạm.	Chi phí nhân lực lớn, không mở rộng được khi số nút giao tăng.
Tính khách quan	Kết quả phụ thuộc sự tập trung và kinh nghiệm của người xem; dễ bỏ sót ở thời điểm mất chú ý.	Thiếu nhất quán giữa các ca trực; khó bảo đảm công bằng.
Chuẩn hoá bằng chứng	Bằng chứng ở dạng mô tả hoặc ảnh chụp tùy ý, không có cấu trúc trường dữ liệu thống nhất.	Khó đối soát tự động, khó tổng hợp thống kê, khó truy vết.
Thời gian phản hồi	Từ khi vi phạm xảy ra đến khi chủ phương tiện nhận thông báo có thể kéo dài nhiều ngày.	Giảm tác dụng răn đe của biện pháp xử phạt.
Khả năng tổng hợp	Số liệu nằm rải rác trong các biên bản giấy hoặc tệp rời rạc.	Không phân tích được xu hướng theo giờ, theo nút giao để phục vụ tổ chức giao thông.

Bảng 2.1. Hạn chế của quy trình xử lý thủ công

Từ khảo sát trên, đồ án xác định bài toán cần giải quyết là tự động hoá khâu nhận biết và lập hồ sơ, đồng thời chuẩn hoá bằng chứng ở dạng có cấu trúc, để con người chỉ cần thực hiện phần việc thực sự cần phán đoán nghiệp vụ là duyệt hồ sơ. Hình 2.2 mô tả quy trình đề xuất: khâu nhận biết và lập hồ sơ được chuyển cho hệ thống tự động thực hiện liên tục, còn con người tập trung vào khâu duyệt và ra quyết định xử lý trên bộ hồ sơ đã chuẩn hoá.



Hình 2.2. Quy trình đề xuất khi có hệ thống RLVD (to-be)

2.2. Các hệ thống liên quan

2.2.1. Hệ thống camera ứng dụng trí tuệ nhân tạo của lực lượng cảnh sát giao thông

Trong những năm gần đây, Cục Cảnh sát giao thông đã triển khai hệ thống camera tích hợp trí tuệ nhân tạo tại một số tuyến đường và nút giao. Theo thông tin công bố trên báo chí, hệ thống này có khả năng tự động nhận diện hơn hai mươi hành vi vi phạm khác nhau, từ không chấp hành tín hiệu đèn, chạy quá tốc độ, đi ngược chiều, lấn làn, dừng đỗ trái quy định, đến các hành vi khó quan sát hơn như không đội mũ bảo hiểm, không thắt dây an toàn hay sử dụng điện thoại khi lái xe. Về hạ tầng, hệ thống sử dụng camera độ phân giải cao, có ống kính zoom quang học và cảm biến hồng ngoại để ghi hình trong điều kiện thiếu sáng; toàn bộ hình ảnh được truyền về Trung tâm thông tin chỉ huy để phân tích. Sau khi ghi nhận vi phạm, hệ thống tự động trích xuất ảnh hoặc đoạn video liên quan kèm tuyến đường, thời gian và hành vi, đối chiếu với cơ sở dữ liệu đăng ký xe để xác định chủ phương tiện, rồi gửi thông báo qua ứng dụng dành cho người dân trong khoảng hai giờ.

Một số liệu đáng chú ý từ đợt thí điểm trên phố Phạm Văn Bạch, Hà Nội: chỉ trong khoảng thời gian từ 0 giờ đến 12 giờ của một ngày, hệ thống đã phát hiện 1.798 trường hợp vi phạm. Con số này cho thấy quy mô của nhu cầu tự động hoá — không lực

lượng nào có thể lập biên bản thủ công với khối lượng như vậy. Tại Thành phố Hồ Chí Minh, hệ thống camera giám sát cũng đã được lắp đặt trên nhiều tuyến đường trọng điểm, kết nối về Trung tâm điều hành giao thông thông minh, sử dụng công nghệ nhận dạng biển số tự động để phát hiện vượt đèn đỏ, đi sai làn và dừng đỗ sai quy định.

Đánh giá dưới góc độ đồ án: đây là các hệ thống quy mô lớn, có hạ tầng camera chuyên dụng và được tích hợp sâu với cơ sở dữ liệu quốc gia về đăng ký xe. Chúng xác nhận tính đúng đắn của hướng tiếp cận tự động hoá, đồng thời cho thấy khoảng trống mà một hệ thống cỡ nhỏ có thể lấp đầy: chi phí triển khai. Các hệ thống thương mại đòi hỏi camera chuyên dụng có hồng ngoại và zoom quang học, trong khi đồ án này hướng tới việc chạy được trên camera giám sát thông thường và phần cứng phổ thông.

2.2.2. Các giải pháp thương mại và xu hướng kỹ thuật

Khảo sát thị trường thiết bị giám sát giao thông tại Việt Nam cho thấy các thông số triển khai phổ biến là camera độ phân giải hai đến bốn megapixel, cảm biến kích thước 1/2,8 đến 1/1,8 inch, tốc độ 25 đến 30 khung hình mỗi giây, có đèn hồng ngoại tầm 30 đến 100 mét, hỗ trợ dải tương phản động rộng để xử lý ngược sáng, đạt chuẩn bảo vệ IP66/IP67. Đáng chú ý là xu hướng xử lý ngay tại thiết bị: nhiều dòng camera đã tích hợp bộ xử lý chuyên dụng để chạy mô hình ANPR và phân loại phương tiện ngay trên camera, chỉ gửi sự kiện về trung tâm. Xu hướng quốc tế là kiến trúc lai giữa biên và đám mây: xử lý thời gian thực tại biên, còn huấn luyện và cập nhật mô hình thực hiện ở phía máy chủ.

Hướng tiếp cận này trùng khớp với lựa chọn kiến trúc của đồ án. Điểm khác biệt là các giải pháp thương mại xử lý tại biên thường nằm trong firmware của camera nên khó tùy biến logic nghiệp vụ, trong khi đồ án đặt node biên trên một máy tính riêng, cho phép toàn quyền thay đổi quy tắc xét vi phạm và bổ sung loại vi phạm mới mà không phụ thuộc nhà sản xuất thiết bị.

2.2.3. Các công trình và dự án mã nguồn mở liên quan

Về nền tảng học thuật, đồ án kế thừa ba nhóm công trình. Thứ nhất là họ mô hình phát hiện đối tượng thời gian thực YOLO do Ultralytics phát triển, với ưu điểm cân bằng được độ chính xác và tốc độ suy luận trên GPU phổ thông. Thứ hai là ByteTrack, phương pháp theo dõi đa đối tượng liên kết mọi khung giới hạn phát hiện được thay vì chỉ những khung có độ tin cậy cao; đặc điểm này rất phù hợp với bối cảnh xe máy kích thước nhỏ và thường xuyên bị che khuất. Thứ ba là các thư viện nhận dạng biển số mã nguồn mở, trong đó fast-plate-ocr cung cấp mô hình nhẹ chạy được trên cả CPU.

Về dự án mã nguồn mở, đã có một số repository công khai giải quyết bài toán phát hiện vượt đèn đỏ, song phần lớn dừng ở mức minh hoạ thuật toán trên video ngắn: phát hiện phương tiện, xác định màu đèn và in kết quả ra màn hình. Những khía cạnh làm nên một hệ thống dùng được trong thực tế — giao hàng tin cậy khi mạng không ổn định, chống trùng lặp hồ sơ, hiệu chuẩn từ xa, phân quyền người dùng, quy trình duyệt có con người, lưu trữ ảnh bằng chứng — hầu như không được đề cập. Đây chính là phần đồ án tập trung đóng góp.

2.2.4. Vị trí của hệ thống RLVD

Tiêu chí	Xử lý thủ công	Hệ thống thương mại quy mô lớn	RLVD (đồ án)
Khâu nhận biết vi phạm	Con người quan sát trực tiếp hoặc xem lại video	Tự động bằng camera AI chuyên dụng	Tự động bằng node biên chạy trên máy tính phổ thông có GPU
Yêu cầu thiết bị ghi hình	Camera bất kỳ	Camera chuyên dụng 4K, zoom quang, hồng ngoại	Camera giám sát thông thường hoặc tệp video
Vị trí xử lý	Tại trung tâm, bởi con người	Tại camera (firmware) và trung tâm	Tại node biên riêng, logic nghiệp vụ mở
Khả năng tùy biến quy tắc	Hoàn toàn linh hoạt nhưng chậm	Hạn chế, phụ thuộc nhà cung cấp	Toàn quyền, thay đổi được trong mã nguồn
Giao hàng khi mất mạng	Không áp dụng	Tùy giải pháp	Kho outbox SQLite tại node, gửi lại theo lô idempotent
Quyết định xử lý	Con người	Tự động kết hợp đối soát CSDL đăng ký xe	Hệ thống lập hồ sơ, con người duyệt (human-in-the-loop)
Quy mô triển khai	Theo ca kíp	Hàng nghìn camera	Một nút giao mỗi node, mở rộng bằng cách thêm node
Chi phí	Nhân lực cao, thiết bị thấp	Đầu tư thiết bị và hạ tầng rất lớn	Thiết bị thấp, chạy được trên GPU phổ thông

Bảng 2.2. So sánh hệ thống RLVD với các giải pháp liên quan

Kết luận của phần khảo sát: nhu cầu tự động hoá là có thật và đã được khẳng định bằng các triển khai thực tế; tuy nhiên vẫn tồn tại khoảng trống cho một hệ thống mã nguồn mở, chạy trên phần cứng phổ thông, có logic nghiệp vụ mở và cơ chế giao hàng tin cậy. Đồ án định vị sản phẩm của mình vào khoảng trống đó.

2.3. Yêu cầu chức năng

Từ khảo sát hiện trạng và phân tích khoảng trống, đồ án xác định các yêu cầu chức năng sau. Mỗi yêu cầu được đánh mã để truy vết đến use case ở Chương 3, thiết kế ở Chương 4 và ca kiểm thử ở Chương 6.

Mã	Yêu cầu chức năng	Mô tả
FR-01	Phát hiện phương tiện	Nhận diện phương tiện thuộc bốn lớp trong từng khung hình tại node biên, kèm khung giới hạn và độ tin cậy.
FR-02	Theo dõi đa đối tượng	Gán và duy trì mã theo dõi ổn định cho từng phương tiện qua các khung hình, kể cả khi bị che khuất ngắn; cung cấp thông tin chuyển động gồm vận tốc và hướng.
FR-03	Nhận biết trạng thái đèn	Xác định trạng thái đỏ, vàng hoặc xanh trong vùng quan tâm đã hiệu chuẩn; ổn định hoá kết quả để tránh dao động do nhiễu từng khung hình.
FR-04	Hiệu chuẩn từ xa	Cho phép người vận hành kẻ vạch dừng, xác định vùng quan tâm của đèn và chọn hướng giám sát thông qua giao diện web, áp dụng ngay vào pipeline đang chạy mà không cần khởi động lại node.
FR-05	Xét vi phạm	Kết luận một phương tiện vi phạm khi điểm neo của nó cắt qua vạch dừng theo đúng hướng giám sát trong lúc trạng thái đèn đỏ đã ổn định; mỗi phương tiện chỉ sinh một hồ sơ cho một lần vượt.
FR-06	Đọc và chuẩn hoá biển số	Phát hiện biển số, gắn biển với phương tiện tương ứng, đọc nội dung bằng OCR và kiểm tra cấu trúc theo quy định biển số Việt Nam, có sửa các lỗi nhận dạng thường gặp.
FR-07	Lập hồ sơ kèm bằng chứng	Tạo hồ sơ vi phạm gồm mã định danh duy nhất, ảnh bằng chứng toàn cảnh đã chú thích,

		biến số, trạng thái đèn, độ tin cậy, toạ độ cắt vạch và thời điểm.
FR-08	Giao hàng bền vững	Lưu hồ sơ vào kho trung gian tại node trước mọi thao tác mạng; đẩy lên trung tâm theo lô; gửi lại khi thất bại; không mất hồ sơ khi mất kết nối hoặc khởi động lại; không tạo hồ sơ trùng.
FR-09	Quản lý node biên	Node tự đăng ký với trung tâm kèm thông tin cấu hình; gửi heartbeat định kỳ; trung tâm suy ra trạng thái hoạt động dựa trên thời điểm heartbeat gần nhất.
FR-10	Tra cứu và thống kê	Tra cứu hồ sơ có phân trang phía máy chủ, lọc theo trạng thái, node và biến số; thống kê theo trạng thái, theo giờ, theo node và theo trạng thái đèn.
FR-11	Duyệt hồ sơ	Cho phép người có thẩm quyền phê duyệt hoặc từ chối hồ sơ; thay đổi trạng thái được ghi nhận kèm thời điểm; hồ sơ đã duyệt có thể đặt lại trạng thái chờ.
FR-12	Xem camera trực tiếp	Phát luồng video từ node biên qua giao thức HLS trên giao diện web, hỗ trợ xem nhiều camera đồng thời theo lưới; cung cấp ảnh chụp nhanh từng khung hình.
FR-13	Hỏi dữ liệu bằng tiếng Việt	Tiếp nhận câu hỏi tự nhiên bằng tiếng Việt, sinh câu lệnh SQL, thực thi và trả kết quả dạng bảng hoặc biểu đồ; chỉ cho phép truy vấn đọc.
FR-14	Xác thực và phân quyền	Đăng nhập bằng tên người dùng và mật khẩu; cấp JWT; phân quyền theo ba vai trò; hỗ trợ làm mới phiên và đăng xuất.
FR-15	Hiển thị trạng thái đèn trực tiếp	Công bố trạng thái đèn đã ổn định hoá để giao diện web hiển

		thị theo thời gian thực mà không phải chạy lại bộ phân loại.
--	--	--

Bảng 2.3. Yêu cầu chức năng

2.4. Yêu cầu phi chức năng

Các yêu cầu phi chức năng được phát biểu kèm chỉ tiêu đo được hoặc cách đo, để có thể kiểm chứng ở Chương 6.

Mã	Yêu cầu	Chỉ tiêu và cách kiểm chứng
NFR-01	Tính thời gian thực	Tốc độ xử lý của node biên không thấp hơn tốc độ khung hình của nguồn; với nguồn tốc độ thấp hơn thì phải bám được thời gian thực nhờ cơ chế bỏ khung hình cũ. Đo bằng FPS suy luận của mô hình và số khung hình đã xử lý trên tổng thời gian chạy.
NFR-02	Độ trễ ổn định tín hiệu đèn	Thời gian từ khi đèn đổi màu đến khi trạng thái ổn định được công bố không quá 0,8 giây. Đo bằng số khung hình debounce nhận với chu kỳ khung hình của nguồn.
NFR-03	Độ tin cậy giao hàng	Không mất hồ sơ trong các kịch bản mất kết nối và khởi động lại node. Kiểm chứng bằng cách ngắt máy chủ trung tâm, cho node tiếp tục phát hiện, sau đó khôi phục kết nối và đối chiếu số hồ sơ.
NFR-04	Tính idempotent	Gửi lại cùng một lô hồ sơ không tạo bản ghi trùng. Kiểm chứng bằng cách gửi hai lần và đối chiếu số lượng chấp nhận và số lượng trùng ở phản hồi.
NFR-05	Bảo mật	Mật khẩu được băm bằng BCrypt; JWT ký bằng HS256 với khoá bí mật tối thiểu 32 ký tự và từ chối khởi động nếu khoá yếu; token node biên được so sánh theo thời gian không đổi; API ghi của node biên yêu cầu token và giới hạn

		tần suất; nguồn gốc truy cập bị kiểm soát bằng danh sách cho phép; không có thông tin bí mật nào nằm cứng trong mã nguồn.
NFR-06	Khả năng mở rộng	Thêm node biên mới không đòi hỏi thay đổi mã nguồn ở trung tâm; hồ sơ được định danh theo node và theo lần chạy để không xung đột; thống kê được tính bằng truy vấn tổng hợp phía CSDL thay vì tải toàn bộ dữ liệu vào bộ nhớ.
NFR-07	Khả năng vận hành	Toàn bộ hệ thống khởi động bằng một lệnh; mỗi dịch vụ có kiểm tra sức khỏe; dịch vụ phụ thuộc chỉ khởi động sau khi dịch vụ nền đã sẵn sàng; chính sách tự khởi động lại khi lỗi.
NFR-08	Khả năng quan sát	Node biên công bố số khung hình đã xử lý, số vi phạm đã phát hiện, FPS và tự báo trạng thái không sẵn sàng khi pipeline ngừng tiến triển; mỗi tầng ghi nhật ký có ngữ cảnh để truy vết theo mã hồ sơ.
NFR-09	Khả năng khôi phục	Dữ liệu CSDL và ảnh bằng chứng nằm trong volume bền vững; có kịch bản sao lưu và khôi phục; hồ sơ chưa gửi được giữ lại qua các lần khởi động lại container.
NFR-10	Nhất quán thời gian	Mọi dấu thời gian trong hệ thống dùng cùng múi giờ Việt Nam để số liệu thống kê theo giờ không bị lệch.
NFR-11	Khả năng bảo trì	Tách bạch trách nhiệm giữa các tầng; tham số vận hành đọc từ biến môi trường thay vì nằm cứng; các quy tắc nghiệp vụ phức tạp có kiểm thử đơn vị kèm theo; danh sách đường dẫn cần token node chỉ khai báo ở một nơi duy nhất.
NFR-12	Tính sử dụng	Giao diện bằng tiếng Việt; thông báo lỗi diễn đạt theo

		ngôn ngữ nghiệp vụ thay vì mã lỗi kỹ thuật; thao tác hiệu chuẩn thực hiện trực tiếp trên khung hình video; có nhiều chủ đề giao diện phù hợp điều kiện ánh sáng khác nhau.
--	--	--

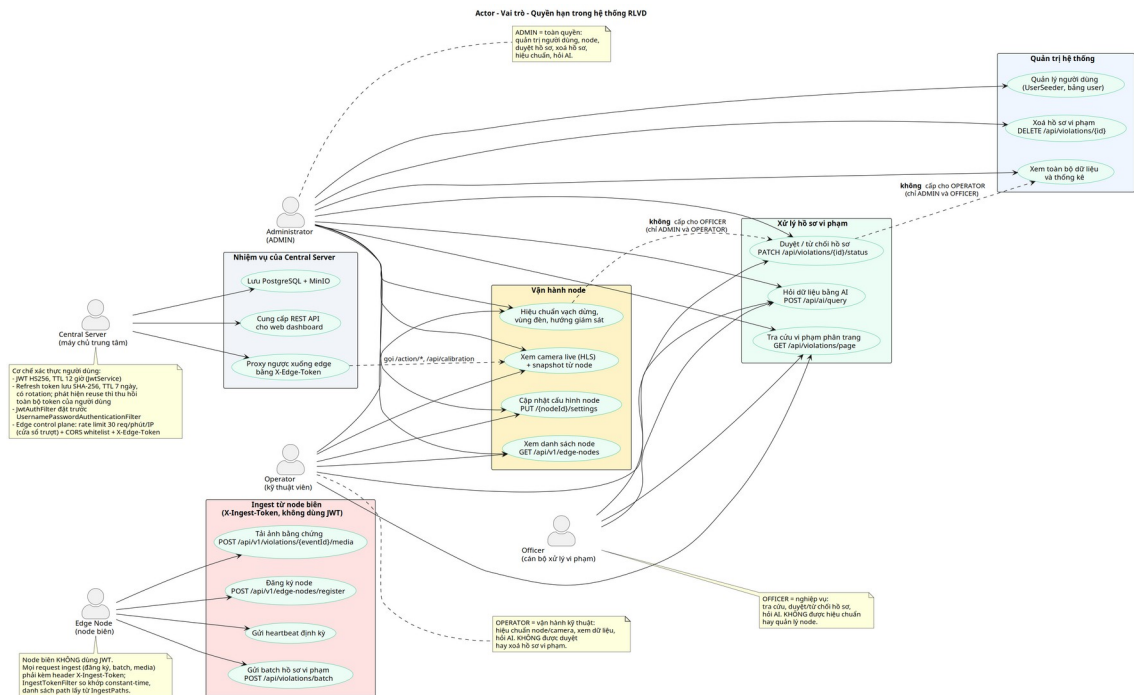
Bảng 2.4. Yêu cầu phi chức năng

Hai yêu cầu NFR-03 và NFR-04 có vai trò đặc biệt quan trọng vì chúng quyết định kiến trúc của tầng giao tiếp giữa node biên và trung tâm, được trình bày chi tiết ở mục 4.1 và kiểm chứng ở Chương 6.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1. Actors

Hệ thống có ba actor là con người, tương ứng với ba vai trò được cài đặt trong cơ chế phân quyền, và hai actor là hệ thống. Việc tách bạch vai trò xuất phát từ nguyên tắc phân nhiệm: người cấu hình thiết bị không nên đồng thời là người ra quyết định xử lý hồ sơ, và cả hai nhóm này đều không nên có quyền quản trị người dùng.



Hình 3.1. Quan hệ actor, vai trò và quyền hạn trong hệ thống

Actor	Loại	Vai trò hệ thống	Quyền hạn chính
Administrator	Con người, actor chính	Người quản trị hệ thống, chịu trách nhiệm về tài khoản và dữ liệu.	Toàn bộ quyền của Operator và Officer; quản lý tài khoản người dùng; xóa hồ sơ vi phạm; xem toàn bộ dữ liệu và thống kê.
Operator	Con người, actor chính	Kỹ thuật viên vận hành thiết bị tại các nút giao.	Xem và cập nhật cấu hình node biên; hiệu chuẩn vạch dừng, vùng đèn và hướng giám sát; xem camera trực tiếp và ảnh chụp nhanh; tra cứu hồ sơ và thống kê. Không có quyền duyệt hồ sơ.
Officer	Con người, actor chính	Cán bộ nghiệp vụ ra	Tra cứu hồ sơ; xem chi

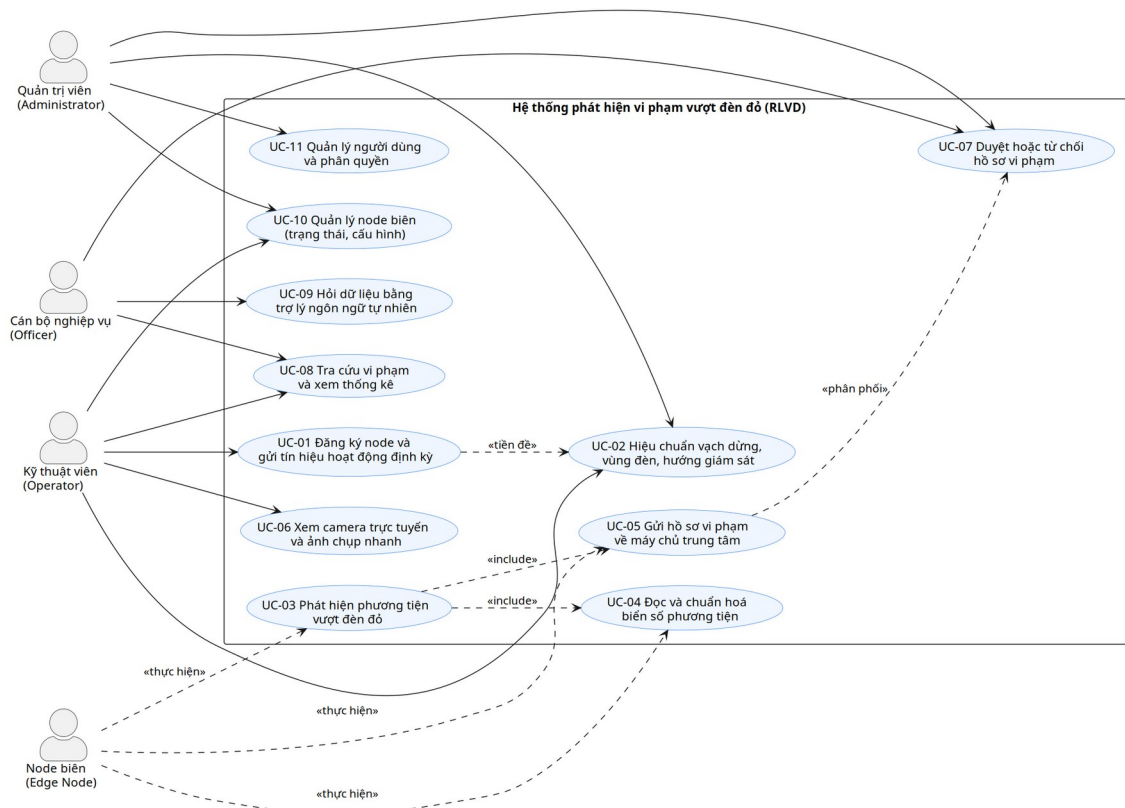
		quyết định xử lý hồ sơ.	tiết và bằng chứng; phê duyệt hoặc từ chối hồ sơ; xem thống kê; đặt câu hỏi bằng tiếng Việt. Không có quyền hiệu chuẩn thiết bị và không có quyền xóa hồ sơ.
Edge Node	Hệ thống, actor máy	Tác nhân tự động đặt tại nút giao, vừa là nguồn dữ liệu vừa là nơi thi hành lệnh hiệu chuẩn.	Đăng ký với trung tâm; gửi heartbeat; phát hiện vi phạm; đọc biển số; đẩy hồ sơ theo lô; tải ảnh bằng chứng; tiếp nhận lệnh hiệu chuẩn; phục vụ luồng HLS và ảnh chụp nhanh.
Central Server	Hệ thống, actor phụ	Máy chủ trung tâm điều phối dữ liệu và làm cầu nối giữa giao diện web với node biên.	Lưu hồ sơ và ảnh; cung cấp REST API; xác thực và phân quyền; tính thống kê; chuyển tiếp lệnh hiệu chuẩn xuống node; đại diện gọi mô hình ngôn ngữ để sinh SQL.

Bảng 3.1. Danh sách actor và quyền hạn

Một điểm cần lưu ý về mô hình actor: Edge Node tham gia hệ thống theo hai chiều. Ở chiều chủ động, node tự khởi xướng việc đăng ký, heartbeat và đẩy hồ sơ, dùng token dành riêng cho node chứ không dùng JWT của người dùng. Ở chiều bị động, node tiếp nhận lệnh hiệu chuẩn do trung tâm chuyển tiếp xuống, và lệnh này được bảo vệ bằng một token khác. Hai chiều dùng hai cơ chế bảo vệ riêng biệt vì mức độ rủi ro khác nhau: chiều đẩy dữ liệu chỉ có thể tạo hồ sơ, còn chiều tiếp nhận lệnh có thể thay đổi hành vi xét vi phạm của cả một nút giao.

3.2. Use Case Diagram

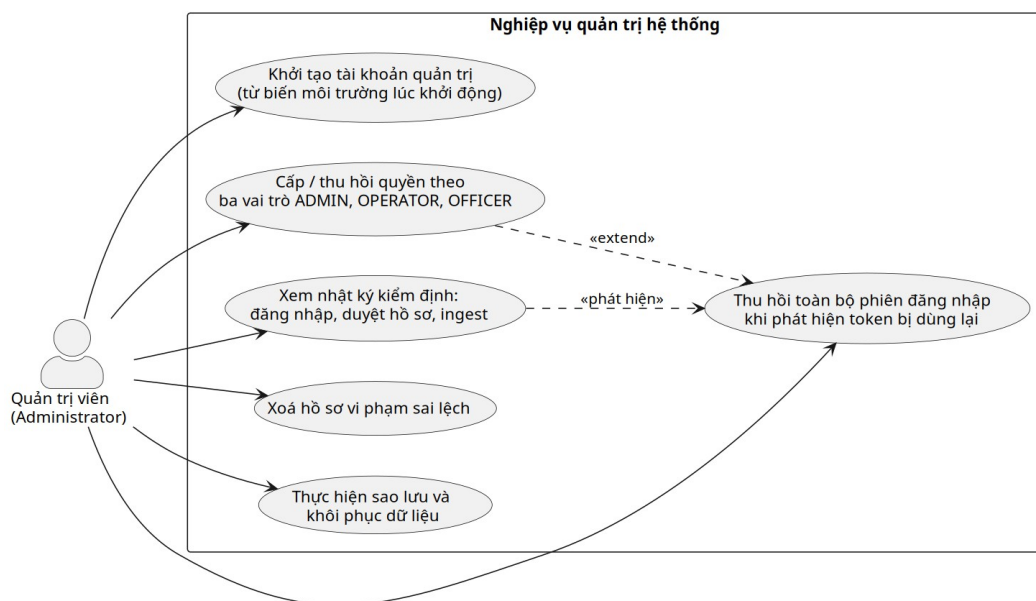
Sơ đồ use case tổng thể được trình bày ở Hình 3.2. Hệ thống gồm mười một use case chính, chia thành bốn nhóm: nhóm vận hành node (đăng ký và heartbeat, quản lý node, hiệu chuẩn, xem camera), nhóm xử lý tại biên (phát hiện vi phạm, đọc biển số, gửi hồ sơ, tải ảnh bằng chứng), nhóm nghiệp vụ hồ sơ (tra cứu, thống kê, duyệt) và nhóm hỗ trợ (hỏi dữ liệu bằng tiếng Việt).



Hình 3.2. Sơ đồ use case tổng thể

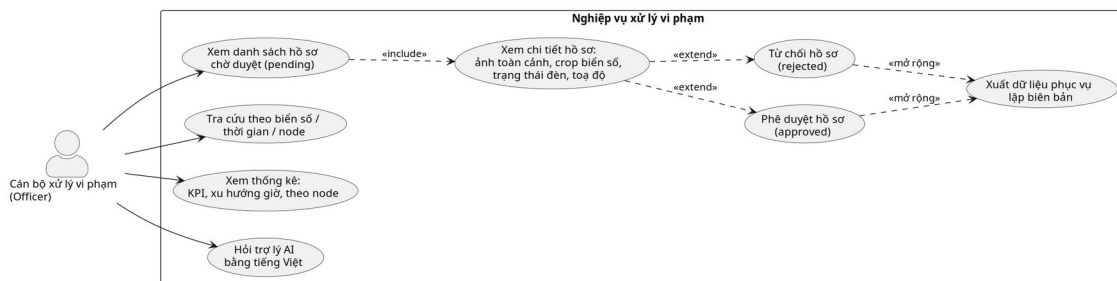
Các quan hệ giữa use case phản ánh đúng thứ tự phụ thuộc nghiệp vụ. Use case phát hiện vi phạm bao gồm use case đọc biến số và use case gửi hồ sơ, vì một hồ sơ hoàn chỉnh cần cả biến số lẫn đường truyền về trung tâm. Use case gửi hồ sơ bao gồm use case tải ảnh bằng chứng. Use case hiệu chuẩn có quan hệ kích hoạt với use case phát hiện vi phạm, bởi khi chưa có vạch dừng thì bộ xét vi phạm ở trạng thái ngừng hoạt động — pipeline vẫn chạy, vẫn phát hiện và theo dõi phương tiện, nhưng không sinh hồ sơ nào. Use case đăng ký node là điều kiện tiên quyết của use case hiệu chuẩn, vì trung tâm cần biết địa chỉ của node mới có thể chuyển tiếp lệnh xuống.

Ba sơ đồ chi tiết theo từng vai trò dưới đây làm rõ ranh giới quyền hạn. Đây cũng là cơ sở để xây dựng các kiểm thử phân quyền ở Chương 6.



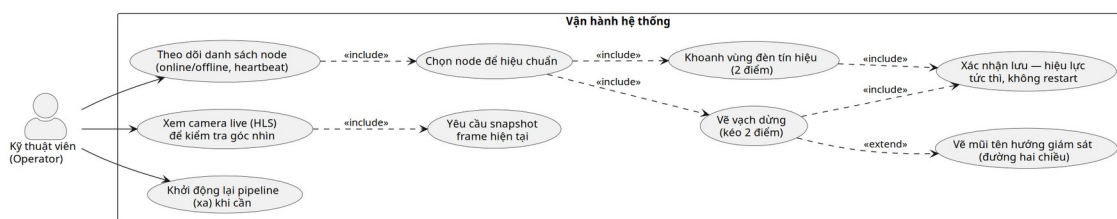
Hình 3.3. Sơ đồ use case của Administrator

Administrator bao trùm toàn bộ use case của hai vai trò còn lại, đồng thời giữ hai quyền riêng là quản lý tài khoản người dùng và xoá hồ sơ vi phạm. Quyền xoá được tách riêng khỏi quyền duyệt để tránh việc một hồ sơ bị loại khỏi hệ thống mà không để lại dấu vết trạng thái.



Hình 3.4. Sơ đồ use case của Officer

Officer là vai trò sử dụng hệ thống thường xuyên nhất. Luồng làm việc điển hình là mở danh sách hồ sơ chờ duyệt, xem chi tiết từng hồ sơ cùng ảnh bằng chứng, rồi phê duyệt hoặc từ chối. Hai use case tra cứu và thống kê phục vụ việc đối chiếu khi cần thêm căn cứ; use case hỏi dữ liệu bằng tiếng Việt cho phép đặt những câu hỏi tổng hợp mà giao diện thống kê cố định chưa đáp ứng.



Hình 3.5. Sơ đồ use case của Operator

Operator tập trung vào khâu thiết bị: kiểm tra trạng thái node, xem camera trực tiếp để đánh giá góc quay, và thực hiện hiệu chuẩn. Điểm đáng chú ý là Operator không có quyền duyệt hồ sơ, còn Officer không có quyền hiệu chuẩn — đây là ràng buộc phân nhiệm được kiểm chứng bằng kiểm thử phân quyền ở Chương 6.

3.3. Use Case Specification

Năm use case quan trọng nhất được đặc tả chi tiết theo mẫu thống nhất, gồm actor tham gia, tiền điều kiện, luồng sự kiện chính, các luồng thay thế và hậu điều kiện.

3.3.1. UC-01: Phát hiện xe vượt đèn đỏ

Trường	Nội dung
Mã use case	UC-01
Tên	Phát hiện xe vượt đèn đỏ
Actor	Edge Node (actor chính); Administrator và Officer tiếp nhận kết quả một cách gián tiếp
Mô tả tóm tắt	Node biên phân tích từng khung hình để xác định phương tiện cắt qua vạch dừng trong khi đèn đỏ đã ổn định, từ đó lập hồ sơ vi phạm kèm ảnh bằng chứng.
Tiền điều kiện	Node đã khởi động và nạp xong ba tệp trọng số; nguồn video đọc được; vạch dừng và vùng quan tâm của đèn đã được hiệu chuẩn; kho outbox mở được.
Luồng sự kiện chính	- Node đọc một khung hình từ nguồn kèm chỉ số khung hình và dấu thời gian. - Bộ phân loại đèn xác định trạng thái trong vùng quan tâm, kết hợp mô hình học sâu với bằng chứng màu HSV và vị trí bóng đèn. - Bộ ổn định tín hiệu cập nhật trạng thái ổn định theo cơ chế debounce có trễ; kết quả được công bố cho API điều khiển. - Mô hình phát hiện phương tiện trả về danh sách khung giới hạn kèm nhãn và độ tin cậy. - Bộ theo dõi gán và duy trì mã vết cho từng phương tiện, đồng thời ước lượng vận tốc, hướng di chuyển và quỹ đạo. - Với mỗi vết đã đủ số lần xác nhận, hệ thống so sánh điểm neo ở khung hình trước và khung hình hiện tại với vạch dừng. - Nếu trạng thái đèn đỏ đã ổn định, vết chuyển từ phía này sang phía kia của vạch theo đúng

	hướng giám sát, và đoạn di chuyển thực sự cắt vạch, hệ thống kết luận vi phạm. 8. Hồ sơ được gán mã định danh duy nhất cấu thành từ tiền tố, mã lần chạy, chỉ số khung hình và mã vết. 9. Nếu biển số đã được đọc cho vết này, nội dung biển đã chuẩn hoá được đính kèm hồ sơ. 10. Ảnh bằng chứng được vẽ chú thích và nén lại. 11. Hồ sơ cùng ảnh được ghi vào kho outbox.
Luồng thay thế	2a. Vùng quan tâm chưa hiệu chuẩn: bộ phân loại dùng vùng suy đoán, kết quả có thể kém tin cậy. 3a. Trạng thái đèn không đạt độ tin cậy tối thiểu: bộ ổn định giữ nguyên trạng thái hiện tại, tăng bộ đếm không xác định; vượt ngưỡng dung sai thì quay về trạng thái không xác định và không xét vi phạm. 6a. Vạch dừng chưa được hiệu chuẩn: bỏ qua toàn bộ bước xét vi phạm, pipeline tiếp tục phát hiện và theo dõi bình thường. 6b. Vết đi ngược hướng giám sát: không kết luận vi phạm. 7a. Vết đã từng được kết luận vi phạm trong lần vượt này: bỏ qua để không sinh hồ sơ trùng. 9a. Biển số đọc được nhưng không khớp cấu trúc biển Việt Nam và không sửa được: trường biển số để trống, hồ sơ vẫn được lập. 11a. Mã định danh đã tồn tại trong outbox: bỏ qua bản ghi mới, không báo lỗi. 1b. Khung hình đọc lỗi hoặc mô hình suy luận lỗi: ghi nhật ký cảnh báo và chuyển sang khung hình kế tiếp, không làm gián đoạn pipeline.
Hậu điều kiện	Hồ sơ vi phạm nằm trong kho outbox ở trạng thái chờ gửi; bộ đếm vi phạm trong metrics được tăng; ảnh bằng chứng đã được lưu kèm bản ghi.
Ràng buộc nghiệp vụ	Mỗi vết chỉ sinh tối đa một hồ sơ cho một lần vượt vạch. Không kết luận vi phạm khi tín hiệu đèn chưa ổn định.

Bảng 3.2. Đặc tả UC-01 Phát hiện xe vượt đèn đỏ

3.3.2. UC-02: Gửi hồ sơ vi phạm về trung tâm

Trường	Nội dung
Mã use case	UC-02
Tên	Gửi hồ sơ vi phạm về trung tâm

Actor	Edge Node (actor chính); Central Server (actor phụ tiếp nhận)
Mô tả tóm tắt	Luồng nền tại node biên rút hồ sơ từ kho outbox, đẩy lên trung tâm theo lô có tính idempotent, sau đó tải ảnh bằng chứng cho các hồ sơ đã được chấp nhận.
Tiền điều kiện	Kho outbox đã được khởi tạo; địa chỉ máy chủ trung tâm và token node đã được cấu hình; luồng gửi đang chạy.
Luồng sự kiện chính	1. Luồng nền thức dậy theo chu kỳ cấu hình, mặc định năm giây. 2. Rút tối đa hai mươi hồ sơ ở trạng thái chờ gửi, chỉ đọc phần dữ liệu JSON, không đọc ảnh để tiết kiệm bộ nhớ. 3. Đóng gói thành một lô và gửi bằng một yêu cầu POST kèm token node. 4. Trung tâm kiểm tra từng mã định danh; bản ghi mới được lưu, bản ghi đã tồn tại được đếm là trùng và bỏ qua. 5. Trung tâm trả về số lượng chấp nhận, số lượng trùng, số lượng lỗi kèm danh sách mã tương ứng. 6. Node đánh dấu các hồ sơ đã được chấp nhận là đã gửi. 7. Node rút tiếp các hồ sơ đã gửi nhưng chưa tải ảnh, tải ảnh lên trung tâm bằng yêu cầu multipart. 8. Trung tâm lưu ảnh vào MinIO theo khoá đối tượng suy ra từ mã hồ sơ và ghi đường dẫn vào bản ghi. 9. Node đánh dấu ảnh đã tải xong.
Luồng thay thế	3a. Không thể kết nối hoặc trung tâm trả lỗi: tăng bộ đếm số lần thử của cả lô, lui lại theo cấp số nhân với trần sáu mươi giây, hồ sơ vẫn ở trạng thái chờ gửi. 3b. Gửi theo lô thất bại lặp lại: thử gửi từng hồ sơ một để cô lập bản ghi hỏng, các bản ghi tốt vẫn được chấp nhận. 7a. Tải ảnh thất bại: hồ sơ vẫn giữ trạng thái đã gửi, chỉ chờ ảnh chưa tải được giữ nguyên để thử lại ở chu kỳ sau. 2a. Không có hồ sơ nào chờ gửi: luồng ngủ tiếp, không phát sinh yêu cầu mạng.
Hậu điều kiện	Hồ sơ đã có mặt trong CSDL trung tâm với trạng thái chờ duyệt; ảnh bằng chứng đã có trong MinIO và đường dẫn đã được ghi; kho

	outbox tại node không còn bản ghi chờ gửi tương ứng.
Ràng buộc nghiệp vụ	Thứ tự hai bước bắt buộc: dữ liệu JSON phải được chấp nhận trước, ảnh tải sau. Điều này bảo đảm không bao giờ có ảnh mờ còi trong MinIO mà thiếu bản ghi trong CSDL.

Bảng 3.3. Đặc tả UC-02 Gửi hồ sơ vi phạm về trung tâm

3.3.3. UC-03: Hiệu chuẩn từ xa

Trường	Nội dung
Mã use case	UC-03
Tên	Hiệu chuẩn vạch dừng, vùng đèn và hướng giám sát từ xa
Actor	Operator hoặc Administrator (actor chính); Central Server và Edge Node (actor phụ)
Mô tả tóm tắt	Người vận hành vẽ vạch dừng trực tiếp trên khung hình video trực tiếp; lệnh được chuyển tiếp qua trung tâm xuống node và có hiệu lực từ khung hình kế tiếp.
Tiền điều kiện	Người dùng đã đăng nhập với vai trò có quyền hiệu chuẩn; node đã đăng ký với trung tâm kèm địa chỉ và token điều khiển; node đang phục vụ luồng video.
Luồng sự kiện chính	1. Người dùng mở trang chi tiết node, chọn chế độ vẽ vạch dừng. 2. Giao diện hiển thị khung hình trực tiếp lấy từ node qua đường proxy. 3. Người dùng kéo hai điểm trên khung hình để xác định vạch dừng. 4. Giao diện quy đổi tọa độ hiển thị về tọa độ gốc của khung hình video. 5. Người dùng chọn hướng giám sát. 6. Giao diện gửi yêu cầu POST tới trung tâm kèm JWT. 7. Trung tâm kiểm tra vai trò, tra cứu node trong sổ đăng ký để lấy địa chỉ và token điều khiển. 8. Trung tâm chuyển tiếp yêu cầu xuống node bằng giao thức HTTP phiên bản 1.1, kèm token điều khiển của node. 9. Node kiểm tra token theo thời gian không đổi, ghi nhận vạch dừng mới vào biến trạng thái dùng chung có khoá bảo vệ. 10. Ở khung hình kế tiếp, bộ xét vi phạm đọc lại vạch dừng đang hoạt động và áp dụng ngay. 11. Giao diện truy vấn lại trạng thái hiệu chuẩn

	để xác nhận và vẽ vạch lên khung hình.
Luồng thay thế	3a. Người dùng chọn chế độ vẽ vùng đèn: thao tác kéo thả tạo hình chữ nhật thay vì đoạn thẳng, gửi tới endpoint tương ứng. 5a. Người dùng chỉ đối hướng giám sát mà không vẽ lại vạch. 8a. Node không phản hồi hoặc từ chối token: trung tâm trả lỗi cổng upstream, giao diện hiển thị thông báo và không thay đổi trạng thái. 7a. Node chưa có trong sổ đăng ký hoặc địa chỉ không gọi được: trung tâm báo lỗi, giao diện gợi ý kiểm tra kết nối node.
Hậu điều kiện	Vạch dừng mới có hiệu lực tức thời; nếu trước đó bộ xét vi phạm ở trạng thái ngừng do chưa hiệu chuẩn thì nay được kích hoạt; trạng thái hiệu chuẩn truy vấn được qua API.
Ràng buộc kỹ thuật	Việc gọi ngược từ trung tâm xuống node phải dùng HTTP phiên bản 1.1. Nguyên nhân là cơ chế nâng cấp giao thức h2c từng làm rơi phần thân yêu cầu POST khi đi qua máy chủ ứng dụng Python, dẫn đến lỗi cú pháp ở node.

Bảng 3.4. Đặc tả UC-03 Hiệu chuẩn từ xa

3.3.4. UC-04: Duyệt hồ sơ vi phạm

Trường	Nội dung
Mã use case	UC-04
Tên	Duyệt hồ sơ vi phạm
Actor	Officer hoặc Administrator (actor chính)
Mô tả tóm tắt	Cán bộ nghiệp vụ xem bằng chứng của từng hồ sơ chờ duyệt và ra quyết định phê duyệt hoặc từ chối.
Tiền điều kiện	Người dùng đã đăng nhập với vai trò Officer hoặc Administrator; tồn tại ít nhất một hồ sơ ở trạng thái chờ duyệt.
Luồng sự kiện chính	- Cán bộ mở trang duyệt hồ sơ. - Giao diện tải danh sách hồ sơ chờ duyệt theo lô năm mươi bản ghi, đồng thời tải trước lô kế tiếp. - Cán bộ chọn một hồ sơ để xem chi tiết. - Giao diện hiển thị ảnh bằng chứng, biển số nếu đọc được, trạng thái đèn kèm độ tin cậy, tọa độ cắt vạch, mã vết, node nguồn và thời điểm ghi nhận. - Cán bộ đối chiếu bằng chứng trực quan với

	các trường dữ liệu. 6. Cán bộ chọn phê duyệt hoặc từ chối. 7. Giao diện gửi yêu cầu cập nhật trạng thái kèm JWT. 8. Trung tâm kiểm tra vai trò, cập nhật trạng thái và thời điểm cập nhật. 9. Giao diện làm mới bộ đếm hồ sơ chờ duyệt và chuyển sang hồ sơ kế tiếp.
Luồng thay thế	4a. Ảnh bằng chứng chưa tải được: cán bộ căn cứ vào các trường dữ liệu còn lại để quyết định. 6a. Hồ sơ đã ở trạng thái được chọn: nút tương ứng bị vô hiệu hoá. 6b. Cán bộ muốn đưa hồ sơ đã xử lý về trạng thái chờ: dùng chức năng đặt lại trạng thái, chỉ khả dụng với vai trò phù hợp. 8a. Người dùng không đủ quyền: trung tâm trả lời cấm truy cập, giao diện hiển thị thông báo và giữ nguyên hồ sơ. 2a. Không còn hồ sơ chờ duyệt: giao diện hiển thị trạng thái rỗng.
Hậu điều kiện	Hồ sơ chuyển sang trạng thái đã duyệt hoặc đã từ chối; thời điểm cập nhật được ghi nhận; số liệu thống kê và bộ đếm trên giao diện phản ánh trạng thái mới.
Ràng buộc nghiệp vụ	Hệ thống không tự động ra quyết định xử phạt. Trạng thái duyệt chỉ xác nhận hồ sơ có đủ căn cứ, việc lập biên bản thuộc quy trình nghiệp vụ bên ngoài.

Bảng 3.5. Đặc tả UC-04 Duyệt hồ sơ vi phạm

3.3.5. UC-05: Hỏi dữ liệu bằng tiếng Việt

Trường	Nội dung
Mã use case	UC-05
Tên	Hỏi dữ liệu bằng tiếng Việt
Actor	Officer, Operator hoặc Administrator (actor chính); Central Server và dịch vụ mô hình ngôn ngữ bên ngoài (actor phụ)
Mô tả tóm tắt	Người dùng đặt câu hỏi tự nhiên bằng tiếng Việt; hệ thống sinh câu lệnh SQL, kiểm tra an toàn, thực thi và trả kết quả kèm gợi ý loại biểu đồ.
Tiền điều kiện	Người dùng đã đăng nhập; khoá API của dịch vụ mô hình ngôn ngữ đã được cấu hình; CSDL có dữ liệu.

Luồng sự kiện chính	1. Người dùng mở bảng trợ lý và nhập câu hỏi, tối đa năm trăm ký tự. 2. Giao diện gửi câu hỏi tới bộ định tuyến phía máy chủ của ứng dụng web. 3. Bộ định tuyến chuyển tiếp tới trung tâm kèm JWT. 4. Trung tâm dựng lời nhắc gồm mô tả lược đồ bốn bảng và yêu cầu chỉ viết câu lệnh SELECT. 5. Trung tâm gọi dịch vụ mô hình ngôn ngữ qua giao thức REST. 6. Dịch vụ trả về câu lệnh SQL và gợi ý loại biểu đồ. 7. Trung tâm kiểm tra an toàn: chỉ chấp nhận SELECT, chặn các từ khoá nguy hiểm, tự bổ sung giới hạn năm mươi dòng. 8. Trung tâm thực thi truy vấn chỉ đọc và trả về tên cột, các dòng dữ liệu, số lượng và loại biểu đồ. 9. Giao diện hiển thị câu lệnh SQL đã sinh cùng bảng kết quả hoặc biểu đồ cột.
Luồng thay thế	7a. Câu lệnh chứa từ khoá bị chặn: trung tâm trả thông báo lỗi, không thực thi, giao diện hiển thị lý do. 5a. Dịch vụ mô hình ngôn ngữ không phản hồi hoặc khoá API sai: trả lỗi, giao diện gợi ý kiểm tra cấu hình. 8a. Câu lệnh hợp lệ nhưng sai ngữ pháp hoặc tham chiếu bảng không tồn tại: trả lỗi truy vấn cho người dùng, không làm lộ chi tiết lược đồ. 1a. Câu hỏi vượt giới hạn độ dài: giao diện chặn ngay phía client.
Hậu điều kiện	Người dùng nhận được câu trả lời dưới dạng dữ liệu; không có thay đổi nào trong CSDL.
Ràng buộc an toàn	Toàn bộ câu lệnh sinh ra chỉ được phép đọc. Danh sách từ khoá bị chặn gồm DROP, DELETE, INSERT, UPDATE, ALTER, TRUNCATE, GRANT và REVOKE. Câu lệnh nhiều lệnh bị từ chối.

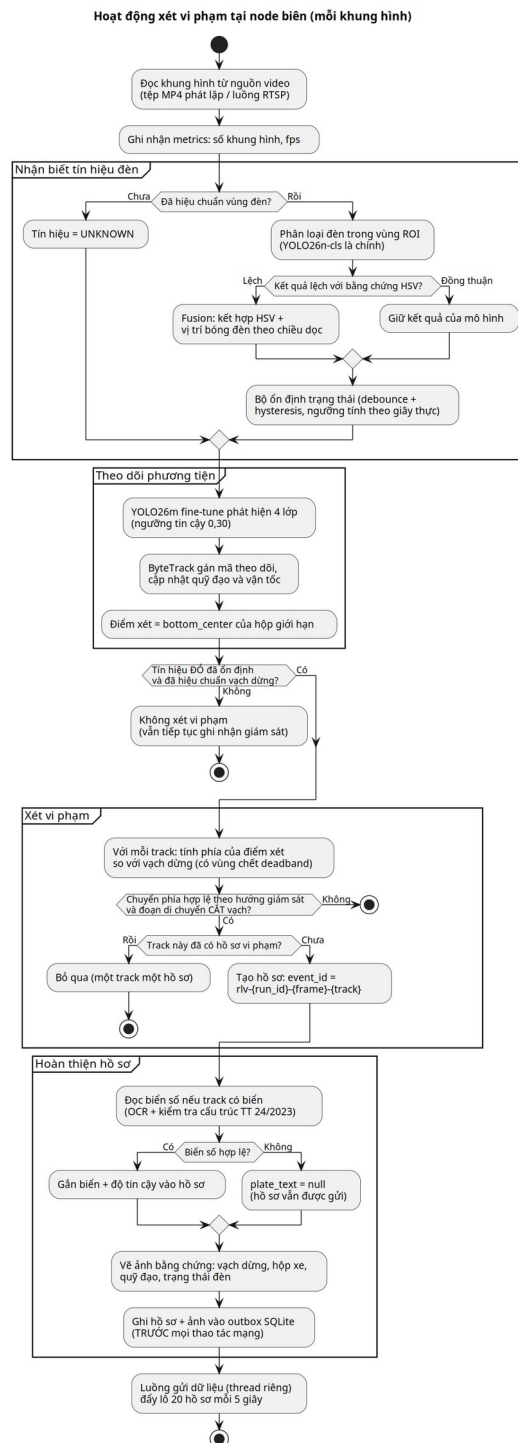
Bảng 3.6. Đặc tả UC-05 Hỏi dữ liệu bằng tiếng Việt

3.4. Activity Diagram

3.4.1. Hoạt động xét vi phạm tại node biên

Sơ đồ hoạt động ở Hình 3.6 mô tả vòng lặp xử lý mỗi khung hình. Điểm thiết kế đáng chú ý là trình tự các khâu: trạng thái đèn được xác định và ổn định hoá trước khi xét

đến phương tiện. Cách sắp xếp này cho phép bỏ qua sớm toàn bộ phần logic vi phạm khi đèn không ở trạng thái đỏ ổn định, giảm chi phí tính toán không cần thiết ở phần lớn khung hình.



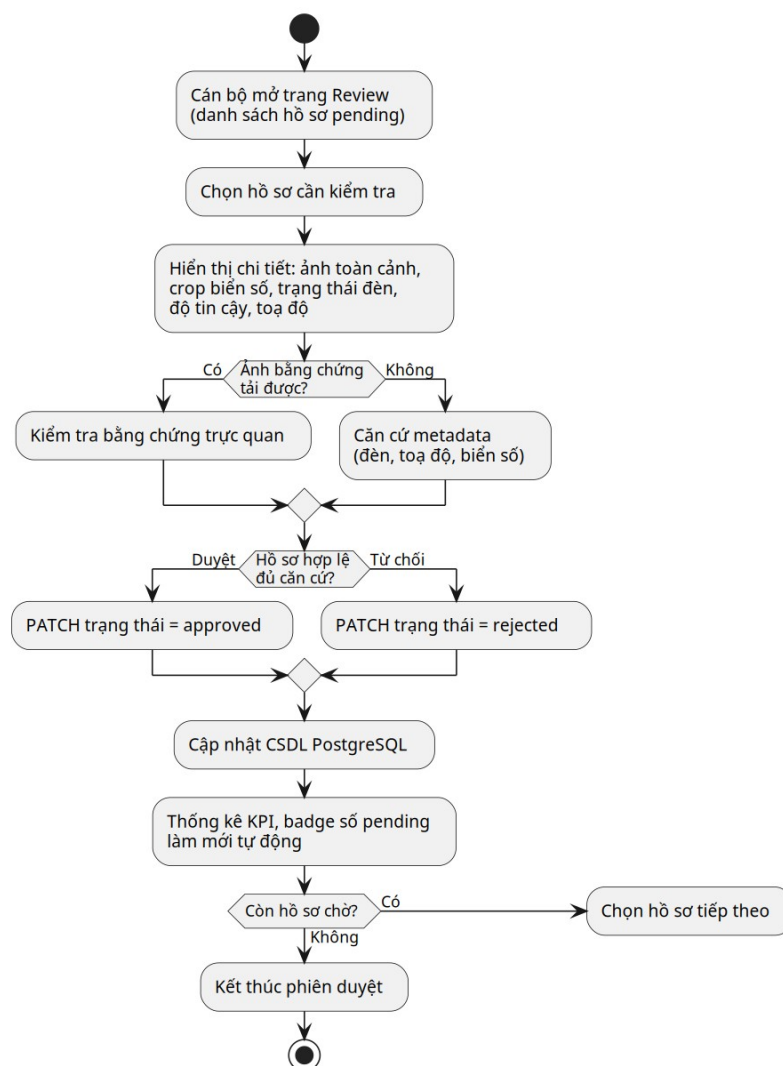
Hình 3.6. Sơ đồ hoạt động xét vi phạm tại node biên

Nhánh xử lý lỗi được đặt ở cấp toàn khối: bất kỳ khâu nào trong khung hình hiện tại gặp lỗi đều dẫn tới việc ghi nhật ký cảnh báo và chuyển sang khung hình kế tiếp. Đây là yêu cầu thực tế bắt buộc, vì một hệ thống giám sát chạy nhiều giờ liên tục không

được phép dừng chỉ vì một khung hình hỏng hoặc một lần suy luận lỗi. Bộ theo dõi và bộ ổn định tín hiệu đều tự dung sai được khoảng trống ngắn trong dữ liệu.

3.4.2. Hoạt động duyệt hồ sơ

Sơ đồ ở Hình 3.7 mô tả luồng duyệt của Officer. Nhánh rẽ quan trọng nhất là trường hợp ảnh bằng chứng không tải được: thay vì chặn toàn bộ quy trình, hệ thống cho phép cán bộ căn cứ vào các trường dữ liệu còn lại. Quyết định này xuất phát từ thực tế vận hành — việc chờ ảnh có thể làm tổn động hồ sơ, trong khi các trường trạng thái đèn, độ tin cậy và tọa độ vẫn cung cấp căn cứ nhất định.

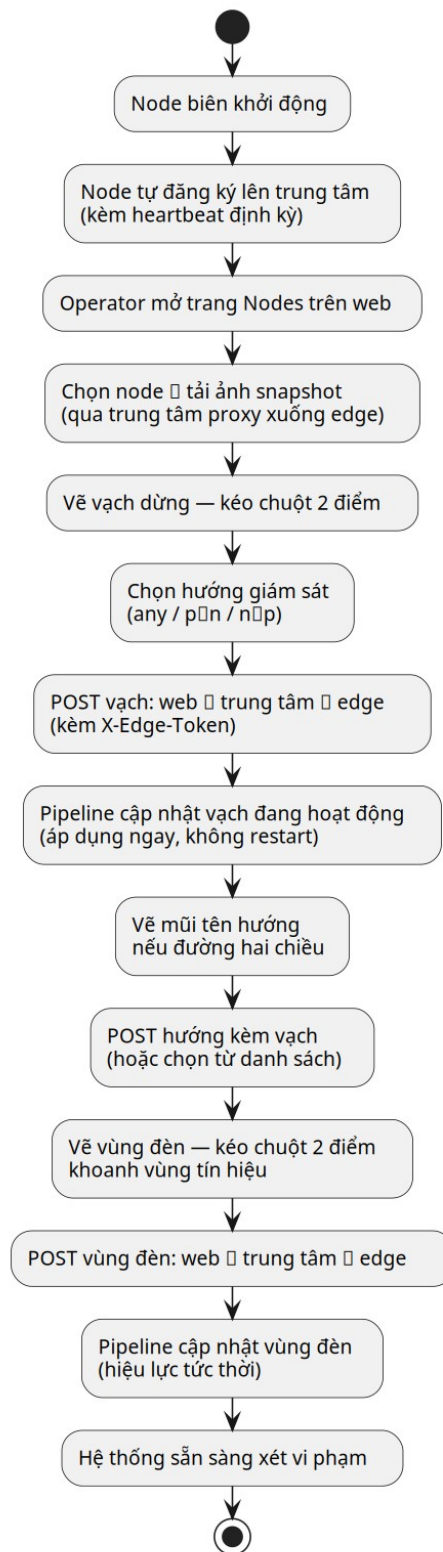


Hình 3.7. Sơ đồ hoạt động duyệt hồ sơ vi phạm

3.4.3. Hoạt động hiệu chuẩn từ xa

Sơ đồ ở Hình 3.8 mô tả luồng hiệu chuẩn, trong đó phần xử lý tại node được tách biệt với phần tương tác trên giao diện. Vòng lặp kiểm tra xác nhận ở cuối luồng là cần thiết

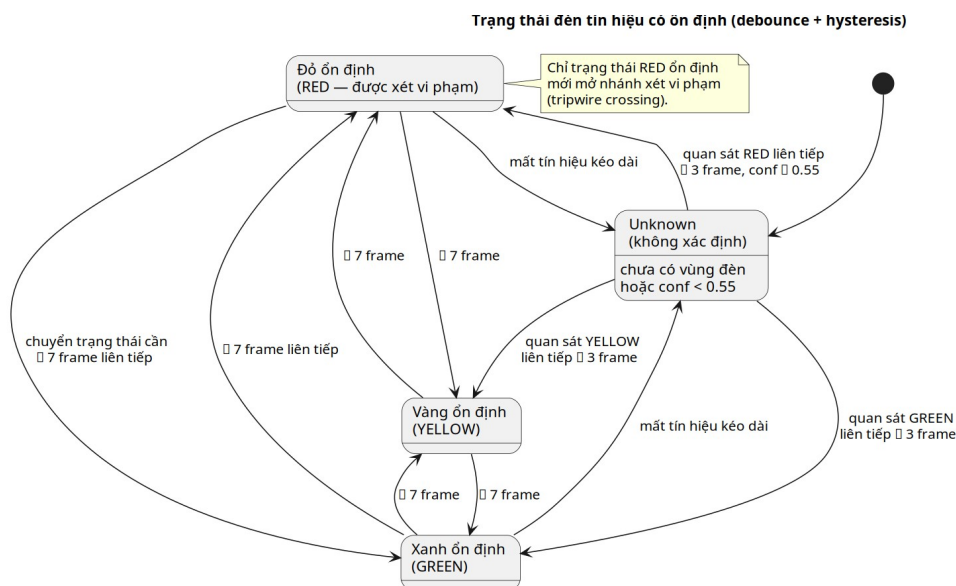
vì lệnh hiệu chuẩn đi qua hai bước chuyển tiếp, và người vận hành cần biết chắc chắn node đã áp dụng giá trị mới.



Hình 3.8. Sơ đồ hoạt động hiệu chuẩn từ xa

3.4.4. Máy trạng thái tín hiệu đèn

Trạng thái đèn tín hiệu không được dùng trực tiếp từ kết quả phân loại từng khung hình mà phải đi qua một máy trạng thái có trễ. Lý do là kết quả phân loại trên dữ liệu thực tế luôn có nhiễu: một vài khung hình bị chói nắng, bị che khuất hoặc chuyển tiếp giữa hai màu có thể cho kết quả sai. Nếu dùng trực tiếp, một khung hình nhiễu duy nhất cũng đủ tạo ra hồ sơ vi phạm giả.



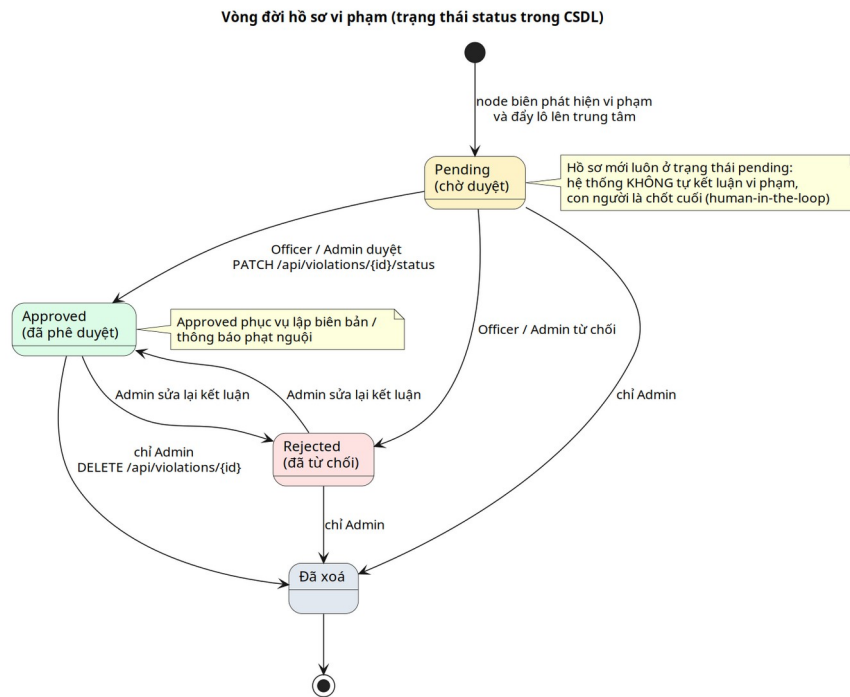
Hình 3.9. Máy trạng thái tín hiệu đèn có debounce

Cơ chế hoạt động như sau. Một trạng thái chỉ được công nhận là ổn định sau khi cùng một kết quả có độ tin cậy đạt ngưỡng xuất hiện liên tiếp đủ số khung hình quy định. Khi đã ở trạng thái ổn định, việc chuyển sang trạng thái khác đòi hỏi số khung hình liên tiếp cao hơn — đây chính là phần trễ tạo nên tính hai chiều của cơ chế: dễ xác lập trạng thái ban đầu nhưng khó bị lật ngược bởi nhiễu. Các kết quả không đạt độ tin cậy không xóa ngay trạng thái ổn định mà chỉ được đếm dồn; vượt ngưỡng dung sai thì máy trạng thái mới quay về trạng thái không xác định.

Một cải tiến quan trọng là các ngưỡng này được quy đổi từ giây sang khung hình theo tốc độ khung hình thực của nguồn, thay vì dùng số khung hình cố định. Với ngưỡng cứng, cùng một giá trị sẽ tương ứng 0,23 giây ở nguồn 30 khung hình mỗi giây nhưng tới 2,3 giây ở nguồn 3 khung hình — nghĩa là hành vi hệ thống phụ thuộc vào thiết bị ghi hình theo cách không kiểm soát được. Sau khi quy đổi theo giây, độ trễ chuyển trạng thái đo được là 0,66 giây ở nguồn 6 khung hình và 0,67 giây ở nguồn 3 khung hình, so với 1,00 giây và 2,3 giây trước khi thay đổi.

3.4.5. Máy trạng thái vòng đời hồ sơ vi phạm

Hồ sơ vi phạm có vòng đời ba trạng thái, được lưu trong trường status của CSDL trung tâm.



Hình 3.10. Máy trạng thái vòng đời hồ sơ vi phạm

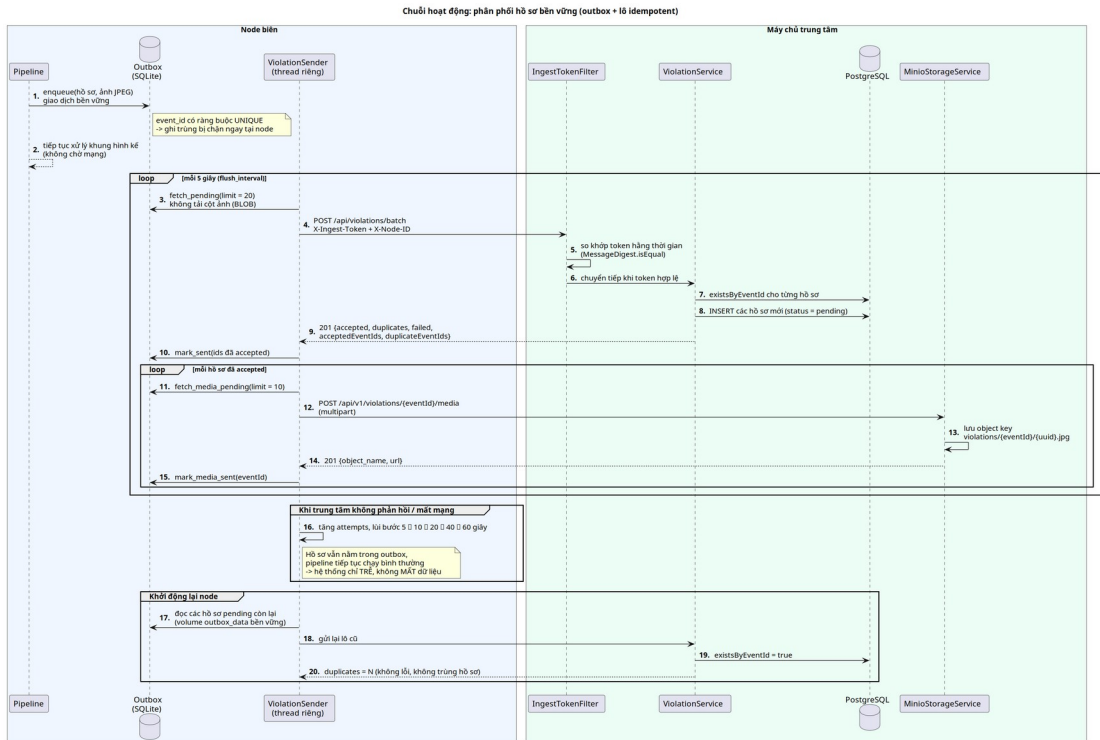
Trạng thái chờ duyệt là trạng thái khởi tạo bắt buộc do node biên gán; node không có quyền tạo hồ sơ ở trạng thái đã duyệt. Hai trạng thái đã duyệt và đã từ chối chỉ đạt được qua thao tác của người dùng có thẩm quyền. Việc cho phép chuyển ngược từ hai trạng thái kết quả về trạng thái chờ là cần thiết trong thực tế vận hành, vì cán bộ có thể quyết định sai khi chưa xem đủ bằng chứng; tuy nhiên thao tác này được giới hạn ở vai trò phù hợp và thời điểm cập nhật luôn được ghi lại để truy vết.

3.5. Sequence Diagram

Bốn sơ đồ sequence dưới đây mô tả các luồng tương tác quan trọng nhất giữa các tầng, được chọn vì mỗi luồng chứa một quyết định thiết kế then chốt.

3.5.1. Luồng phân phối hồ sơ bền vững

Sơ đồ ở Hình 3.11 thể hiện cơ chế giao hàng hai bước. Điểm mấu chốt nằm ở thứ tự: hồ sơ được ghi vào kho SQLite tại node ngay tại thời điểm phát hiện, trước khi bất kỳ thao tác mạng nào diễn ra. Nhờ đó, khoảng thời gian rủi ro mất dữ liệu được thu về bằng không — nếu mạng hỏng hoặc node bị tắt, hồ sơ vẫn nằm trên đĩa và sẽ được gửi ở lần chạy kế tiếp.



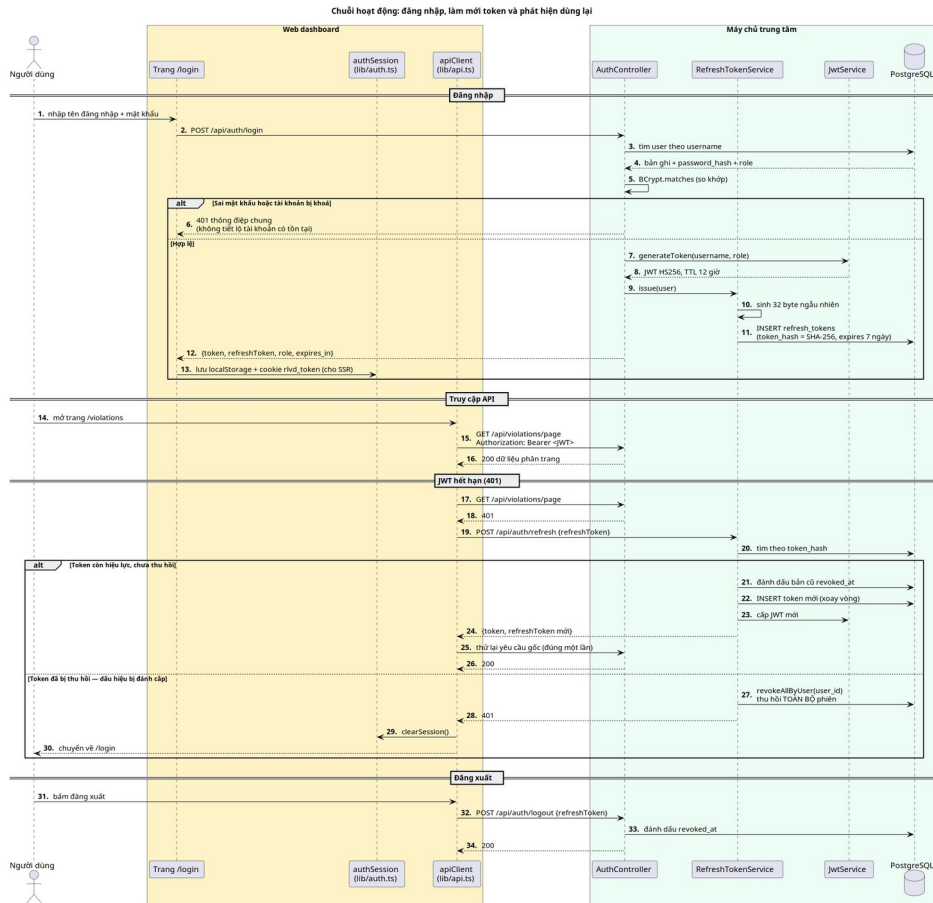
Hình 3.11. Sơ đồ sequence phân phối hồ sơ bền vững

Tính idempotent được bảo đảm ở cả hai đầu. Tại node, trường mã định danh trong kho outbox có ràng buộc duy nhất nên một sự kiện không thể được ghi hai lần. Tại trung tâm, trước khi lưu mỗi bản ghi hệ thống kiểm tra sự tồn tại của mã định danh bằng một truy vấn boolean, và bản ghi trùng được đếm riêng thay vì gây lỗi cho cả lô. Nhờ đó việc gửi lại sau một lần thất bại — tình huống không thể tránh khỏi trong mạng không tin cậy — luôn an toàn.

Ảnh bằng chứng được tải ở bước thứ hai, sau khi dữ liệu JSON đã được chấp nhận. Thứ tự này bảo đảm không bao giờ xuất hiện ảnh mờ côỉ trong kho đối tượng mà thiếu bản ghi tương ứng trong CSDL, đồng thời cho phép theo dõi riêng cờ trạng thái tải ảnh để thử lại độc lập.

3.5.2. Luồng xác thực và làm mới token

Sơ đồ ở Hình 3.12 mô tả cơ chế xác thực hai loại token. Token truy cập dạng JWT ký bằng HS256 với thời hạn mười hai giờ, tương đương một ca trực, mang theo tên người dùng và vai trò để trung tâm phân quyền mà không cần truy vấn CSDL ở mỗi yêu cầu. Token làm mới là chuỗi ngẫu nhiên 32 byte, chỉ được lưu dưới dạng giá trị băm SHA-256 trong CSDL nên ngay cả khi CSDL bị lộ thì token thật vẫn không thể khôi phục.

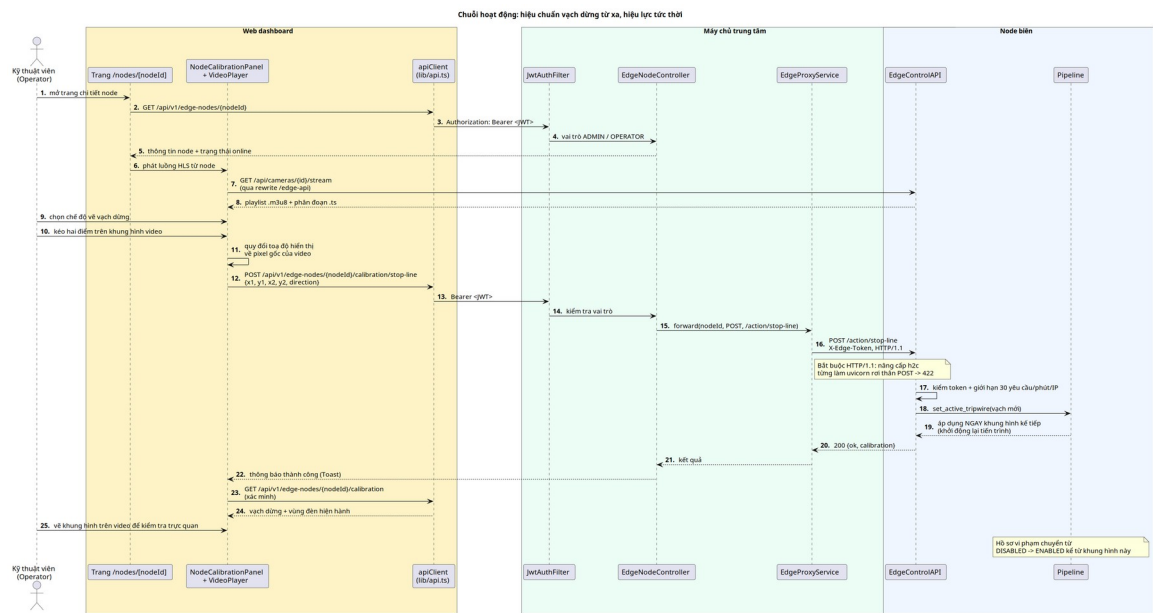


Hình 3.12. Sơ đồ sequence xác thực và làm mới token

Mỗi lần làm mới, token cũ bị thu hồi và một token mới được cấp — cơ chế xoay vòng. Nếu một token đã bị thu hồi lại được trình bày lần nữa, hệ thống coi đó là dấu hiệu bị đánh cắp và thu hồi toàn bộ token làm mới của người dùng đó, buộc phải đăng nhập lại. Đây là biện pháp giảm thiểu thiệt hại tiêu chuẩn cho kịch bản token rò rỉ.

3.5.3. Luồng hiệu chuẩn từ xa

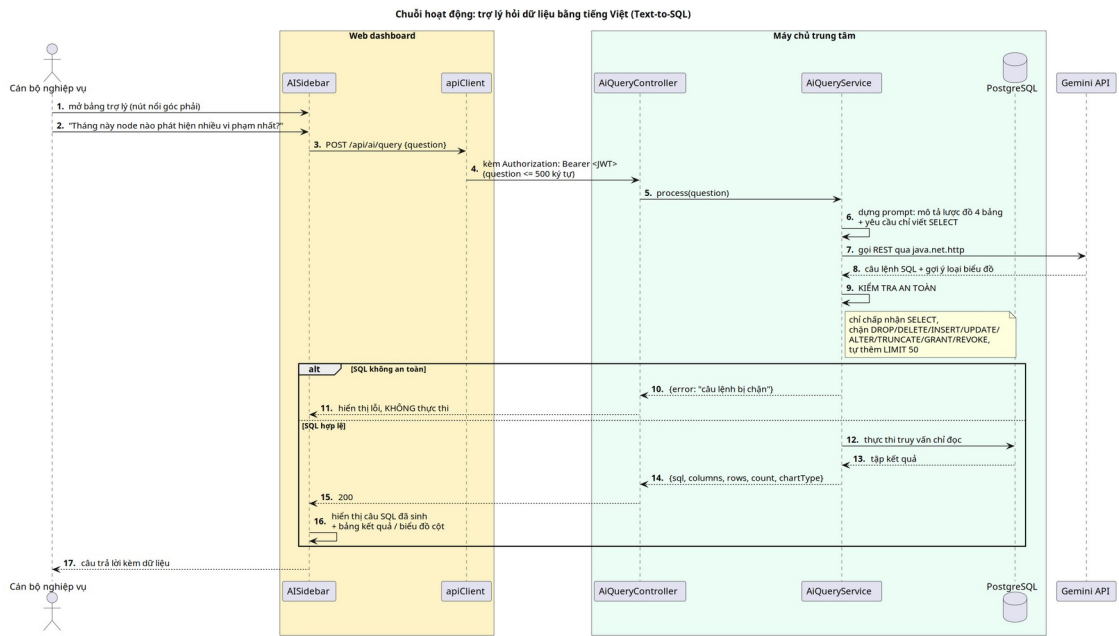
Sơ đồ ở Hình 3.13 thể hiện đường đi của một lệnh hiệu chuẩn, trong đó có hai lần xác thực độc lập: JWT của người dùng ở chặng web tới trung tâm, và token điều khiển của node ở chặng trung tâm tới node. Lệnh không đi thẳng từ trình duyệt tới node, nhờ đó địa chỉ nội bộ và token của node không bao giờ lộ ra phía client.



Hình 3.13. Sơ đồ sequence hiệu chuẩn vạch đường từ xa

3.5.4. Luồng trợ lý hỏi dữ liệu

Sơ đồ ở Hình 3.14 mô tả luồng hỏi dữ liệu bằng tiếng Việt. Rào chắn an toàn nằm ở trung tâm chứ không phải ở phía giao diện, vì lời nhắc gửi cho mô hình ngôn ngữ chỉ là yêu cầu chứ không phải ràng buộc kỹ thuật — mô hình vẫn có thể sinh ra câu lệnh ngoài ý muốn. Do đó mọi câu lệnh đều phải qua bước kiểm tra trước khi thực thi.



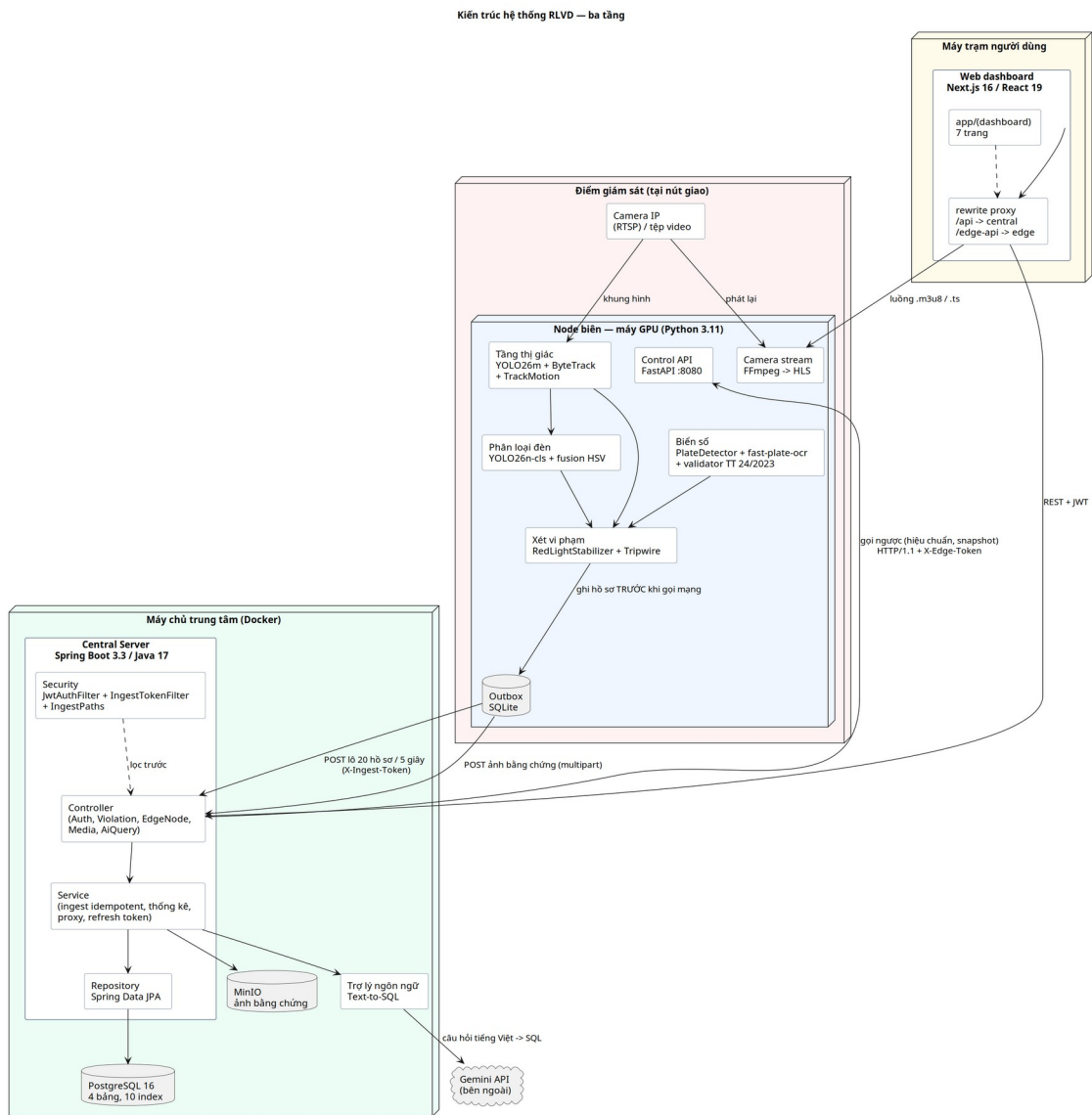
Hình 3.14. Sơ đồ sequence trợ lý hỏi dữ liệu bằng tiếng Việt

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

Chương này trình bày các quyết định thiết kế của hệ thống: kiến trúc tổng thể, sơ đồ lớp, mô hình dữ liệu, thiết kế cơ sở dữ liệu, thiết kế API và thiết kế giao diện. Mỗi mục đều nêu lý do lựa chọn phương án và những phương án đã bị loại bỏ, để người đọc thấy được quá trình cân nhắc chứ không chỉ thấy kết quả cuối cùng.

4.1. System Architecture

Hệ thống được tổ chức theo kiến trúc ba tầng với ranh giới triển khai rõ ràng: node biên đặt gần camera, máy chủ trung tâm đặt tại trung tâm dữ liệu, bảng điều khiển web chạy trên trình duyệt của người vận hành. Hình 4.1 thể hiện kiến trúc tổng thể cùng các luồng giao tiếp giữa ba tầng.

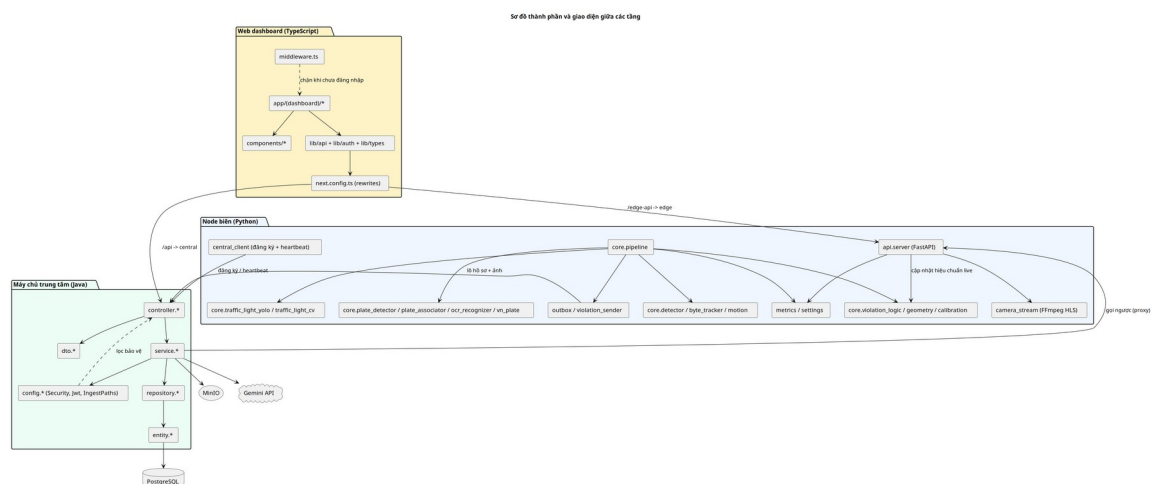


Hình 4.1. Kiến trúc hệ thống ba tầng

Ba nguyên tắc thiết kế chi phối toàn bộ hệ thống:

- Xử lý gần nguồn dữ liệu. Video không bao giờ rời khỏi node biên. Node chỉ gửi về trung tâm những hồ sơ vi phạm đã hoàn chỉnh, gồm một bản ghi JSON khoảng vài trăm byte và một ảnh bằng chứng định dạng JPEG. Một luồng video 1080p chiếm băng thông hàng megabit mỗi giây, trong khi một hồ sơ vi phạm chỉ vài trăm kilobyte và chỉ sinh ra khi có sự kiện, chênh lệch ba đến bốn bậc độ lớn.
- Trung tâm là nguồn sự thật duy nhất. Mọi trạng thái nghiệp vụ lâu dài (hồ sơ vi phạm, kết quả duyệt, danh bạ node, tài khoản người dùng) đều nằm tại trung tâm. Node biên chỉ giữ trạng thái tạm thời: cấu hình hiệu chuẩn đang áp dụng và kho outbox chờ gửi. Nhờ vậy khi một node mất hoàn toàn, dữ liệu nghiệp vụ không bị ảnh hưởng.
- Ranh giới tầng không bị xuyên thủng. Trình duyệt không gọi trực tiếp tới trung tâm hay node biên; mọi yêu cầu đi qua lớp proxy của Next.js. Tầng trình diễn không chứa thư viện cơ sở dữ liệu hay thư viện trí tuệ nhân tạo nào. Quy ước này được giữ bằng cả thiết kế lẫn kiểm tra tự động.

Một điểm khác biệt so với kiến trúc ba tầng cổ điển là luồng gọi ngược chiều: trung tâm chủ động gọi xuống node biên khi người vận hành hiệu chuẩn từ xa. Lý do là node biên nằm sau mạng nội bộ của nút giao, không có địa chỉ công khai cố định, nên trình duyệt không thể gọi thẳng tới node một cách tin cậy. Trung tâm đóng vai trò cầu nối: nó lưu địa chỉ node do chính node đăng ký lên, sau đó chuyển tiếp yêu cầu kèm token dành riêng cho node. Cách làm này cho phép giữ toàn bộ cơ chế xác thực người dùng tại một nơi, đồng thời không phải mở cổng của node ra ngoài.

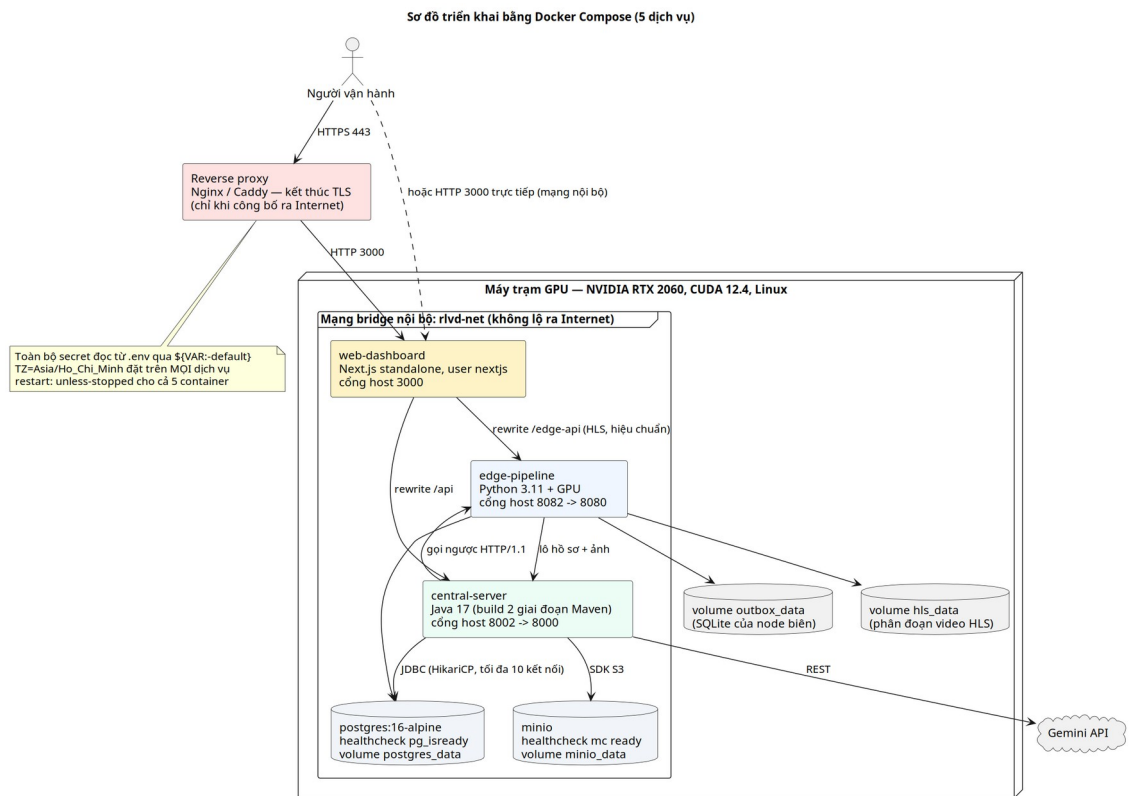


Hình 4.2. Sơ đồ thành phần và giao diện giữa các tầng

Hình 4.2 liệt kê các thành phần bên trong từng tầng và giao diện kết nối giữa chúng. Toàn bộ giao tiếp điều khiển dùng HTTP với định dạng JSON; riêng luồng video trực tiếp dùng HLS qua HTTP, là giao thức phát phân đoạn nên tương thích với trình duyệt mà không cần plugin. Điểm cần lưu ý trong thiết kế thành phần là node biên chứa hai

tiến trình logic chạy song song: vòng lặp xử lý khung hình (chạy liên tục, ưu tiên thời gian thực) và luồng gửi dữ liệu (chạy nền, chịu được trễ). Hai luồng này chỉ gặp nhau tại kho outbox, nhờ vậy việc mạng chậm hoặc trung tâm tạm ngừng không thể làm vòng lặp xử lý bị khựng.

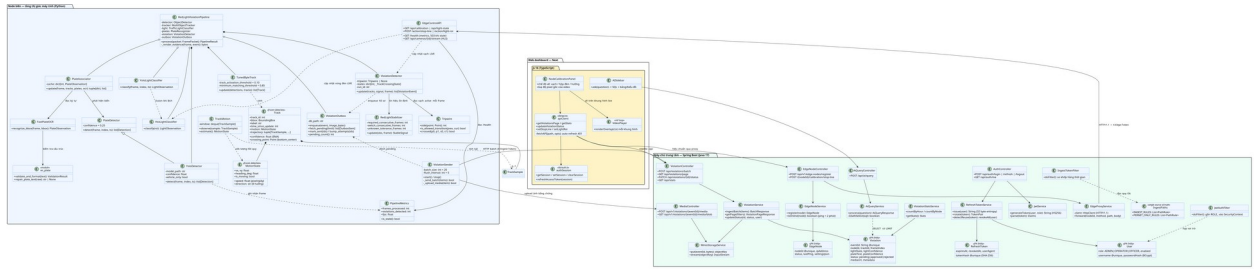
Kiến trúc trên được hiện thực hoá thành năm dịch vụ chạy trong cùng một mạng nội bộ ảo do Docker Compose quản lý, thể hiện ở Hình 4.3. Hai dịch vụ lưu trữ (PostgreSQL và MinIO) chỉ được các dịch vụ ứng dụng truy cập qua tên dịch vụ trong mạng nội bộ, không cần mở cổng ra ngoài. Node biên là dịch vụ duy nhất được cấp quyền truy cập bộ xử lý đồ hoạ, vì toàn bộ khối lượng suy luận học sâu nằm ở tầng này. Việc đóng gói toàn bộ hệ thống thành một tệp cấu hình duy nhất giúp tái lập môi trường chạy giống hệt nhau trên bất kỳ máy nào có Docker và trình điều khiển GPU, đây cũng là cách nhóm nghiệm thu kết quả đo ở Chương 6.



Hình 4.3. Sơ đồ triển khai bằng Docker Compose với năm dịch vụ

4.2. Class Diagram

Sơ đồ lớp ở Hình 4.4 phản ánh đúng cấu trúc mã nguồn hiện hành, chia theo ba gói tương ứng ba tầng triển khai.



Hình 4.4. Sơ đồ lớp của hệ thống

Thiết kế lớp của node biên dựa trên ba mẫu chính:

- Giao thức (Protocol) thay cho kế thừa. Các thành phần phát hiện, theo dõi, phân loại đèn và đọc biển số đều được khai báo dưới dạng giao thức trong `edge_node/core/contracts.py`, gồm `ObjectDetector`, `MultiObjectTracker`, `TrafficLightClassifier` và `PlateRecognizer`. Pipeline chỉ phụ thuộc vào giao thức, không phụ thuộc vào lớp cụ thể, nên có thể thay mô hình phát hiện hoặc thay thuật toán theo dõi mà không sửa logic xét vi phạm. Đây cũng là lý do 124 kiểm thử đơn vị chạy được trên máy không có card đồ họa.
- Đối tượng giá trị bất biến. `Detection`, `Track`, `BoundingBox`, `Point`, `LightObservation` và `ViolationEvent` đều là `dataclass` đóng băng. Dữ liệu đi qua nhiều tầng xử lý trong một khung hình mà không bị thay đổi ngầm, giúp suy diễn nguyên nhân lỗi dễ dàng hơn.
- Tách cấu hình khỏi đối tượng nghiệp vụ. Tham số nằm trong `TripwireConfig`, `RedStabilizerConfig` và `ViolationConfig`; trạng thái hiệu chuẩn đang áp dụng nằm trong biến toàn cục có khoá bảo vệ, đọc lại ở mỗi khung hình. Nhờ đó yêu cầu hiệu chuẩn có hiệu lực tức thời mà không cần khởi động lại pipeline.

Lớp `RedLightViolationPipeline` giữ vai trò điều phối. Mỗi khung hình đi qua bảy bước có thứ tự cố định: phân loại đèn, làm ổn định trạng thái đèn, phát hiện phương tiện, cập nhật theo dõi, gắn biển số, xét vi phạm và ghi kho outbox. Thứ tự này không đổi được, vì bước xét vi phạm cần đồng thời track đã xác nhận và tín hiệu đèn đã ổn định.

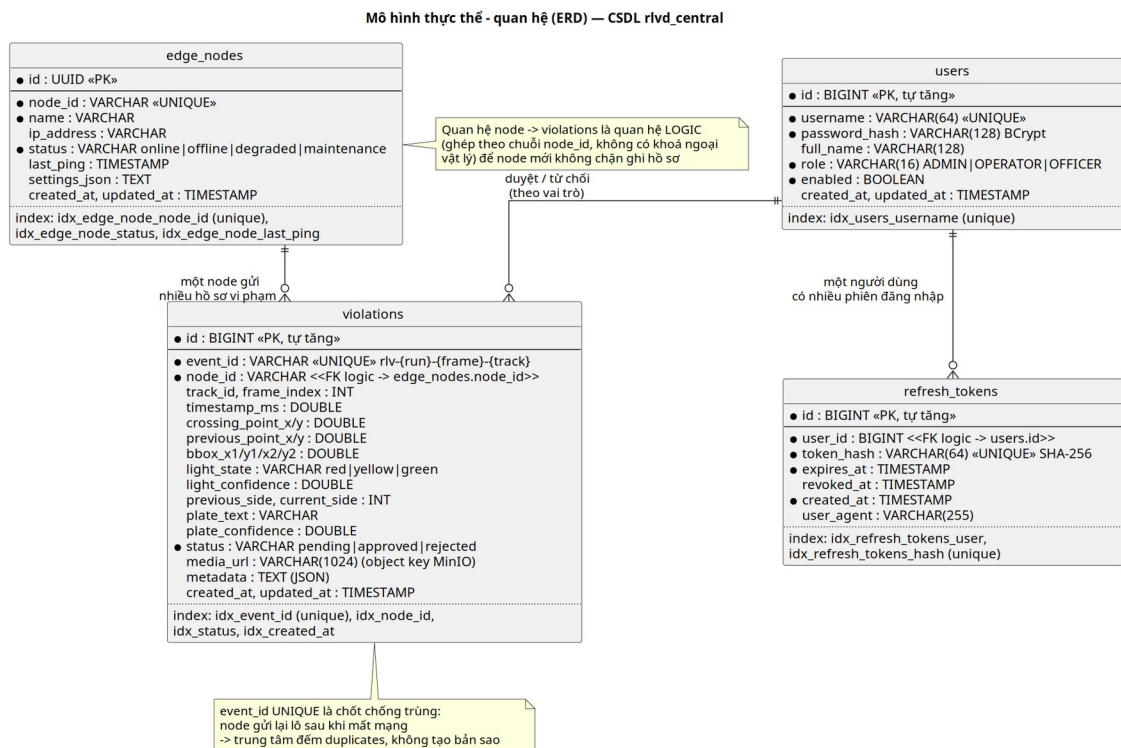
Phía máy chủ trung tâm áp dụng phân lớp chuẩn của Spring: `Controller` nhận yêu cầu và kiểm tra dữ liệu đầu vào, `Service` chứa quy tắc nghiệp vụ, `Repository` truy cập dữ liệu qua Spring Data JPA, `Entity` ánh xạ bảng và `DTO` chuyển dữ liệu ra ngoài. Ba service đáng chú ý về mặt thiết kế: `ViolationService` xử lý nhận dữ liệu theo lô với đặc tính idempotent; `ViolationStatsService` tách riêng phần thống kê để thay đổi hình dạng số liệu tổng hợp không ảnh hưởng tới luồng nhận dữ liệu; `EdgeProxyService` đảm nhận việc gọi ngược xuống node và cố định giao thức HTTP/1.1.

Phía web không dùng sơ đồ lớp theo nghĩa truyền thống vì Next.js tổ chức theo trang và thành phần. Phần logic tái sử dụng nằm trong bốn mô-đun ở thư mục `lib`: `api.ts` bọc toàn bộ lời gọi mạng kèm cơ chế tự làm mới token, `auth.ts` quản lý phiên, `types.ts` định

nghĩa kiểu dữ liệu phản chiếu DTO của trung tâm, và nodes.ts chuẩn hoá trạng thái node. Thiết kế này giữ cho các trang chỉ lo trình bày.

4.3. ERD

Cơ sở dữ liệu của máy chủ trung tâm có bốn thực thể, thể hiện ở Hình 4.5. Quan hệ giữa chúng phản ánh đúng luồng nghiệp vụ: một người dùng có nhiều phiên đăng nhập, một node biên sinh ra nhiều hồ sơ vi phạm, mỗi hồ sơ vi phạm có nhiều nhất một ảnh bằng chứng.



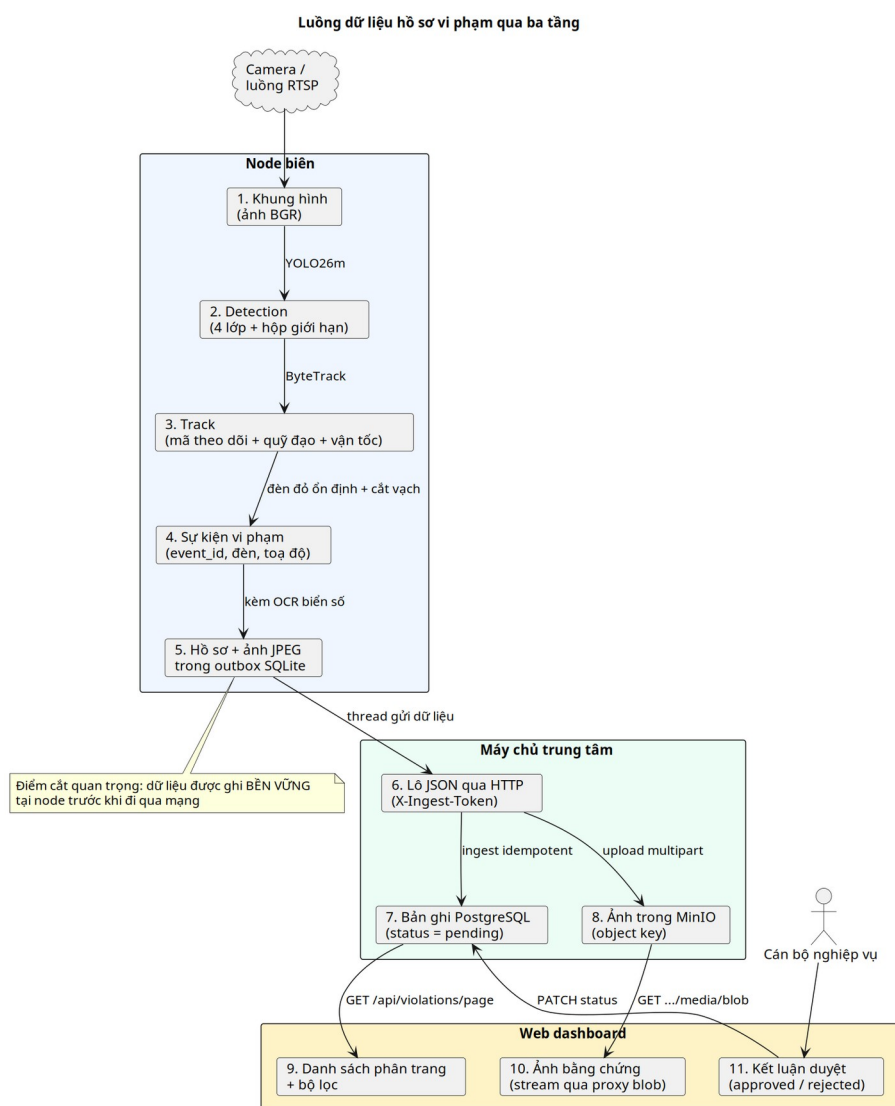
Hình 4.5. Mô hình thực thể quan hệ của cơ sở dữ liệu rlvd_central

Hai đặc điểm thiết kế cần giải thích:

- Quan hệ node biên và hồ sơ vi phạm là quan hệ logic, không có khoá ngoại vật lý. Trường node_id trong bảng violations là chuỗi, không trỏ tới khoá chính của bảng edge_nodes. Lý do là hồ sơ vi phạm phải được chấp nhận ngay cả khi node chưa kịp đăng ký hoặc khi bảng node bị làm sạch để kiểm thử; nếu ràng buộc khoá ngoại, một sự cố đăng ký sẽ khiến toàn bộ lô dữ liệu bị từ chối và node phải gửi lại. Đối lại, việc đối soát được bảo đảm bằng quy trình: node luôn đăng ký trước khi gửi dữ liệu và bộ đếm thống kê theo node_id vẫn hoạt động bình thường.
- Bảng refresh_tokens lưu giá trị băm SHA-256 chứ không lưu token gốc. Khi bảng này bị lộ, kẻ tấn công vẫn không thể dùng token để đăng nhập. Đây là cách xử lý tương tự cách lưu mật khẩu, áp dụng cho cả thông tin đăng nhập dài hạn.

Ngoài bốn bảng của trung tâm, node biên có một kho SQLite riêng (xem mục 4.4) chỉ tồn tại trong phạm vi node và không bao giờ đồng bộ cấu trúc lên trung tâm. Việc tách riêng này là chủ ý: kho tạm không phải là một phần của mô hình dữ liệu nghiệp vụ, nó là hàng đợi giao hàng.

Hình 4.6 tổng hợp đường đi của dữ liệu qua ba tầng dưới dạng luồng biến đổi, bổ sung cho sơ đồ triển khai ở Hình 4.3: dữ liệu thô là khung hình video chỉ tồn tại ở node biên, dữ liệu có cấu trúc tồn tại ở trung tâm, còn dữ liệu hiển thị được tổng hợp tại tầng web. Mỗi mũi tên trên hình tương ứng với một điểm kiểm tra dữ liệu trong mã nguồn, nhờ vậy khi một hồ sơ hiển thị sai trên giao diện, có thể truy ngược theo hình để khoanh vùng tầng gây lỗi.



Hình 4.6. Luồng dữ liệu hồ sơ vi phạm qua ba tầng

4.4. Database Design

Bảng violations là bảng trung tâm của hệ thống. Thiết kế cột xuất phát trực tiếp từ yêu cầu truy vết: khi một hồ sơ bị nghi ngờ là báo động giả, người duyệt phải trả lời được câu hỏi hệ thống căn cứ vào đâu để kết luận. Vì vậy ngoài các trường mô tả sự kiện, bảng còn lưu cả dữ liệu hình học thô và trạng thái đèn tại thời điểm xảy ra vi phạm.

Cột	Kiểu dữ liệu	Ràng buộc và ý nghĩa
id	BIGINT	Khoá chính, tự tăng
event_id	VARCHAR	Khoá duy nhất, có chỉ mục idx_event_id. Định dạng rlv-{mã phiên chạy}-{số khung hình}-{mã theo dõi}. Đây là nền tảng của cơ chế nhận dữ liệu idempotent
node_id	VARCHAR	Có chỉ mục idx_node_id. Node gửi hồ sơ; dùng để lọc và thống kê theo node
track_id, frame_index, timestamp_ms	INTEGER, INTEGER, DOUBLE	Vị trí của sự kiện trong luồng video, cho phép mở lại đúng khung hình để đối chiếu
crossing_point_x, crossing_point_y	DOUBLE	Toạ độ điểm được dùng để xét cắt vạch tại thời điểm kết luận vi phạm
previous_point_x, previous_point_y	DOUBLE	Toạ độ điểm neo liền trước, cùng với điểm cắt tạo thành đoạn thẳng dùng trong phép thử giao cắt
bbox_x1, bbox_y1, bbox_x2, bbox_y2	DOUBLE	Hộp giới hạn của phương tiện lúc vi phạm
previous_side, current_side	INTEGER	Hai phía của vạch dừng trước và sau khi cắt. Cập giá trị này là bằng chứng hình học về hướng di chuyển
light_state, light_confidence	VARCHAR, DOUBLE	Trạng thái đèn đã làm ổn định và độ tin cậy tại thời điểm xét vi phạm
plate_text, plate_confidence	VARCHAR, DOUBLE	Biển số đã qua bộ kiểm tra cấu trúc; để trống nếu không đọc được biển hợp lệ
status	VARCHAR	Có chỉ mục idx_status. Nhận một trong ba giá trị pending, approved, rejected. Mặc định

		pending
media_url	VARCHAR(1024)	Khoá đối tượng trong MinIO theo mẫu violations/{event_id}/{uuid}.jpg; để trống khi chưa tải ảnh lên
metadata	TEXT	Chuỗi JSON mở rộng: hướng giám sát, nguồn video, thông tin mô hình, nhãn phương tiện
created_at, updated_at	DATETIME	Có chỉ mục idx_created_at. Mốc thời gian theo múi giờ Việt Nam; phục vụ sắp xếp và thống kê theo giờ

Bảng 4.1. Thiết kế bảng violations

Bốn chỉ mục trên bảng violations được chọn theo đúng bốn dạng truy vấn mà hệ thống thực sự chạy: tra cứu theo mã sự kiện khi nhận dữ liệu, lọc theo trạng thái khi duyệt hồ sơ, lọc theo node khi xem chi tiết node, và sắp xếp theo thời gian khi liệt kê. Không có chỉ mục thừa, vì mỗi chỉ mục đều làm chậm thao tác ghi.

Cột	Kiểu dữ liệu	Ràng buộc và ý nghĩa
id	UUID	Khoá chính, sinh phía ứng dụng
node_id	VARCHAR	Duy nhất, có chỉ mục idx_edge_node_node_id. Định danh node, dùng làm khoá upsert khi node đăng ký lại
name	VARCHAR	Tên hiển thị trên giao diện; mặc định lấy bằng node_id
ip_address	VARCHAR	Địa chỉ để trung tâm gọi ngược xuống node, đã tách bỏ giao thức và cổng
status	VARCHAR	Có chỉ mục idx_edge_node_status. Trạng thái hoạt động của node
last_ping	DATETIME	Có chỉ mục idx_edge_node_last_ping. Mốc heartbeat gần nhất; node được coi là trực tuyến nếu mốc này cách hiện tại dưới hai phút
settings_json	TEXT	Toàn bộ cấu hình node gửi lên khi đăng ký, bao gồm cổng API điều khiển và token của node để trung tâm dùng khi gọi

		ngược
--	--	-------

Bảng 4.2. Thiết kế bảng *edge_nodes*

Bảng	Cột chính	Ràng buộc và ghi chú thiết kế
users	id (khóa chính), username (duy nhất, tối đa 64 ký tự), password_hash (128 ký tự), full_name, role (16 ký tự), enabled, created_at, updated_at	Chỉ mục duy nhất idx_users_username trên username. Mật khẩu băm bằng BCrypt. Trường role nhận ADMIN, OPERATOR hoặc OFFICER và là căn cứ duy nhất để phân quyền. Tài khoản quản trị đầu tiên được tạo từ biến môi trường, không ghi đè nếu đã tồn tại
refresh_tokens	id (khóa chính), user_id, token_hash (64 ký tự, duy nhất), expires_at, revoked_at, created_at, user_agent (255 ký tự)	Hai chỉ mục: idx_refresh_tokens_user để thu hồi toàn bộ phiên của một người dùng, idx_refresh_tokens_hash để tra cứu khi làm mới token. Trường revoked_at khác rỗng nghĩa là token đã bị thu hồi; nếu một token đã thu hồi được dùng lại, hệ thống thu hồi toàn bộ token của người dùng đó

Bảng 4.3. Thiết kế bảng *users* và *refresh_tokens*

Lược đồ cơ sở dữ liệu được Hibernate sinh tự động từ khai báo trong thực thể JPA, với chế độ ddl-auto ở mức update. Cách này phù hợp với quy mô đồ án vì lược đồ còn thay đổi trong quá trình phát triển, nhưng có nhược điểm là không kiểm soát được thứ tự thay đổi và không có lịch sử di chuyển. Mục 7.2 ghi nhận đây là hạn chế và đề xuất thay bằng công cụ di chuyển lược đồ có phiên bản khi hệ thống đi vào vận hành thật.

Kho outbox tại node biên có cấu trúc đơn giản hơn nhiều nhưng giữ vai trò then chốt trong thiết kế giao hàng:

Cột	Kiểu dữ liệu	Ý nghĩa
id	INTEGER	Khoá chính tự tăng, dùng để lấy bản ghi theo thứ tự ghi
event_id	TEXT	Duy nhất. Ràng buộc này chặn trùng ngay tại node, trước cả khi dữ liệu được gửi đi
payload	TEXT	Chuỗi JSON của hồ sơ vi phạm, đúng định dạng API

		nhận dữ liệu của trung tâm
image	BLOB	Ảnh bằng chứng dạng JPEG. Tách thành cột riêng để luồng gửi dữ liệu JSON theo lô không phải đọc ảnh
status	TEXT	pending hoặc sent
media_sent	INTEGER	Cờ riêng cho ảnh: 0 là chưa tải lên, 1 là đã xong. Tách khỏi status vì bản ghi JSON và ảnh có thể hoàn thành ở hai thời điểm khác nhau
attempts	INTEGER	Số lần thử gửi, dùng làm cơ sở cho chiến lược lùi theo hàm mũ
created_at, sent_at	TEXT	Mốc ghi vào kho và mốc gửi thành công, theo giờ UTC

Bảng 4.4. Lược đồ kho outbox tại node biên (SQLite)

Chỉ mục ghép idx_outbox_status trên cặp (status, id) phục vụ đúng câu truy vấn của luồng gửi dữ liệu: lấy các bản ghi chưa gửi theo thứ tự ghi vào, giới hạn số lượng mỗi đợt. Ảnh được lưu trong cùng tệp cơ sở dữ liệu để việc ghi hồ sơ và ghi ảnh là một thao tác nguyên tử; nếu tách ảnh ra thư mục riêng, một sự cố giữa hai lần ghi sẽ tạo ra hồ sơ không có ảnh hoặc ảnh mờ côi.

4.5. API Design

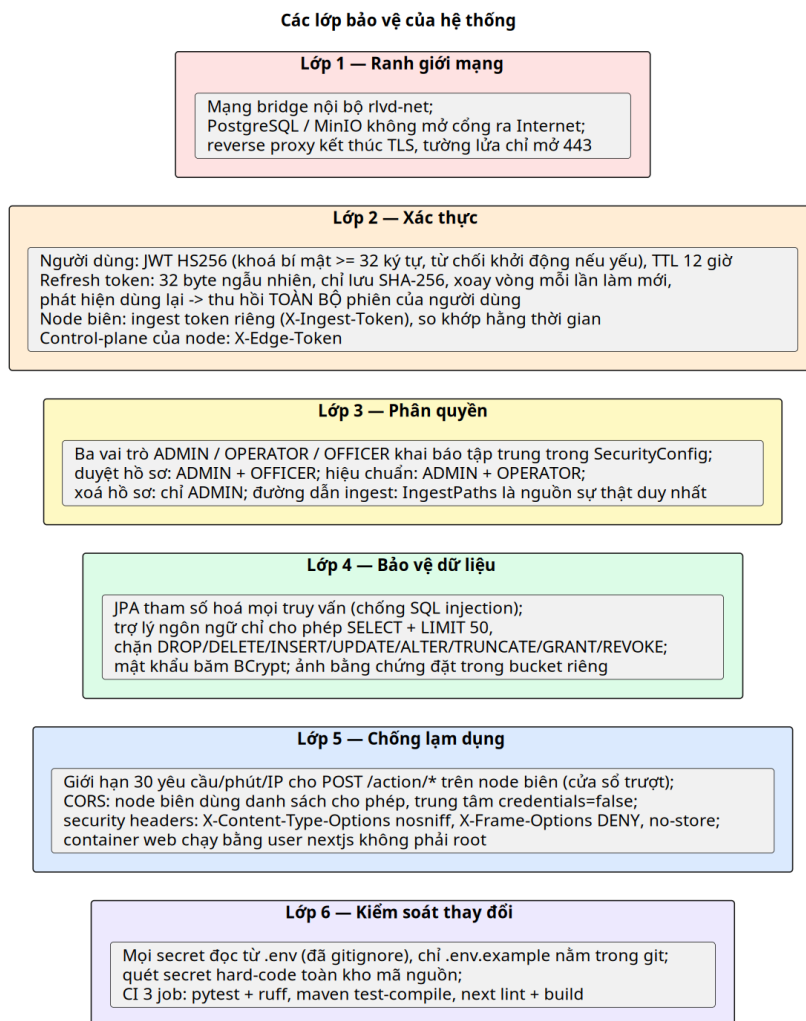
Hệ thống có hai bề mặt API riêng biệt: API của máy chủ trung tâm phục vụ bảng điều khiển web và nhận dữ liệu từ node, và API điều khiển của node biên phục việc xem camera và hiệu chuẩn. Tách hai bề mặt là cần thiết vì đối tượng gọi khác nhau và cơ chế xác thực khác nhau.

Quy ước	Áp dụng trong hệ thống
Định danh tài nguyên bằng danh từ	/api/violations, /api/v1/edge-nodes, /api/auth. Không có động từ trong đường dẫn, trừ hai trường hợp là hành vi nghiệp vụ thực sự: /status (đổi trạng thái duyệt) và /register (đăng ký node)
Phương thức HTTP diễn tả hành động	GET để đọc, POST để tạo hoặc thực hiện, PATCH/PUT để cập nhật, DELETE để xoá. Riêng thao tác đổi trạng thái duyệt có cả PATCH lẫn PUT cùng ý nghĩa để tương thích với các client cũ
Phiên bản API trong đường dẫn	Các endpoint nhận dữ liệu từ node có tiền tố /api/v1; endpoint phục vụ web giữ đường dẫn

	ngắn. Cả hai đều trỏ tới cùng một service
Xác thực bằng tiêu đề, không dùng cookie phiên	Người dùng web gửi Authorization kiểu Bearer; node biên gửi X-Ingest-Token; lời gọi ngược xuống node gửi X-Edge-Token
Phân trang bằng tham số truy vấn	page đánh số từ 0, size mặc định 20. Phản hồi gồm content, page, size, total_elements, total_pages, first, last để giao diện vẽ điều hướng mà không cần gọi thêm
Lỗi trả về theo mã trạng thái chuẩn	Không bọc lỗi trong phản hồi 200. Thông báo lỗi bằng tiếng Việt để hiển thị trực tiếp trên giao diện

Bảng 4.5. Quy ước thiết kế API

Trước khi liệt kê endpoint, cần làm rõ các lớp bảo vệ mà mỗi yêu cầu phải đi qua, thể hiện ở Hình 4.7. Thiết kế API và thiết kế bảo mật trong hệ thống này là một: cùng một bảng endpoint nhưng mỗi nhóm đi qua một chuỗi bộ lọc khác nhau, và việc chọn sai nhóm sẽ khiến yêu cầu bị chặn hoặc, tệ hơn, được chấp nhận khi không nên.



Hình 4.7. Các lớp bảo vệ của hệ thống theo nhóm yêu cầu

Nhóm	Endpoint tiêu biểu	Cơ chế bảo vệ
Xác thực người dùng	POST /api/auth/login, POST /api/auth/refresh, POST /api/auth/logout, GET /api/auth/me	login, refresh và logout mở công khai vì client chưa có token hợp lệ khi gọi; /me yêu cầu JWT
Nhận hồ sơ vi phạm	POST /api/violations, POST /api/violations/batch, POST /api/v1/violations, POST /api/v1/violations/batch	X-Ingest-Token, so khớp theo thời gian hằng số
Nhận ảnh bằng chứng	POST /api/v1/violations/{eventId}/media (multipart)	X-Ingest-Token
Đăng ký node	POST /api/v1/edge-nodes/register	X-Ingest-Token
Tra cứu hồ sơ	GET /api/violations, GET /api/violations/page, GET /api/violations/counts, GET /api/violations/{id}, GET /api/violations/event/{eventId}	JWT, mọi vai trò
Thống kê	GET /api/stats	JWT, mọi vai trò
Duyệt hồ sơ	PATCH hoặc PUT /api/violations/{id}/status, DELETE /api/violations/{id}	JWT với vai trò ADMIN hoặc OFFICER để duyệt; chỉ ADMIN được xóa
Quản lý node	GET /api/v1/edge-nodes, GET /api/v1/edge-nodes/{nodeId}, PUT /api/v1/edge-nodes/{nodeId}/settings	JWT với vai trò ADMIN hoặc OPERATOR
Hiệu chuẩn qua trung gian	GET /api/v1/edge-nodes/{nodeId}/calibration, POST /api/v1/edge-nodes/{nodeId}/calibration/stop-line, POST .../calibration/light-roi, GET .../calibration/snapshot	JWT ADMIN hoặc OPERATOR; trung tâm gắn thêm X-Edge-Token khi chuyển tiếp
Ảnh bằng chứng	GET /api/v1/violations/{eventId}/media/blob, GET .../media, DELETE .../media	Luồng blob mở ở tầng bảo mật vì trung tâm đóng vai proxy đã xác thực; luồng xem và xóa metadata yêu cầu JWT
Trợ lý trí tuệ nhân tạo	POST /api/ai/query	JWT mọi vai trò, kèm bộ kiểm tra chỉ cho phép câu lệnh SELECT
Kiểm tra sức khỏe	GET /api/health, GET	Mở công khai để Docker

	/actuator/health, GET /actuator/info	healthcheck và bộ giám sát dùng
--	---	------------------------------------

Bảng 4.6. Danh sách nhóm API của máy chủ trung tâm

Tổng cộng máy chủ trung tâm có 31 phương thức xử lý yêu cầu phân bố trong năm bộ điều khiển: AuthController, ViolationController, EdgeNodeController, MediaController và AiQueryController. Danh sách đầy đủ kèm định dạng thân yêu cầu và phản hồi nằm trong tài liệu API của kho mã nguồn (docs/api/README.md).

Một quyết định thiết kế đáng chú ý ở tầng bảo mật: danh sách đường dẫn nhận dữ liệu từ node được khai báo tập trung trong một lớp duy nhất tên IngestPaths, gồm hai danh sách là các đường dẫn cần kiểm tra token và các đường dẫn chỉ cần cho phép đi qua. Cả chuỗi lọc bảo mật của Spring Security lẫn bộ lọc kiểm tra token đều đọc từ lớp này. Trước khi có thiết kế này, hai nơi khai báo đường dẫn độc lập và đã từng lệch nhau: chuỗi lọc bảo mật cho phép đi qua nhưng bộ lọc token lại bỏ sót, dẫn tới 263 ảnh bằng chứng bị từ chối âm thầm. Bài học rút ra là khi một thông tin phải xuất hiện ở hai nơi, cần biến nó thành một nguồn duy nhất mà cả hai nơi cùng đọc.

Endpoint	Phương thức	Chức năng	Bảo vệ
/health	GET	Trạng thái node và bộ đếm trong tiến trình: số khung hình đã xử lý, số vi phạm đã phát hiện, tốc độ xử lý thực tế. Trả mã 503 khi pipeline ngừng cập nhật quá mười giây	Công khai
/action/stop-line	GET, POST	Đọc và đặt vạch dừng ảo kèm hướng giám sát được suy ra từ hai điểm	X-Edge-Token, giới hạn tốc độ
/action/light-roi	POST	Đặt vùng quan sát đèn tín hiệu theo toạ độ và kích thước	X-Edge-Token, giới hạn tốc độ
/api/light-roi	GET	Đọc vùng quan sát đèn đang áp dụng	Công khai
/action/restart	POST	Khởi động lại dịch vụ của node	X-Edge-Token, giới hạn tốc độ
/api/calibration	GET	Trạng thái hiệu chuẩn đầy đủ để giao diện vẽ lại vạch dừng và vùng đèn	Công khai

/api/light-state	GET	Trạng thái đèn đã làm ổn định do pipeline công bố ở mỗi khung hình	Công khai
/api/calibration/snapshot	GET	Một khung hình JPEG từ nguồn video, dùng làm nền để vẽ hiệu chuẩn khi luồng trực tiếp chưa sẵn sàng	Công khai
/api/cameras	GET	Danh sách camera do node phục vụ	Công khai
/api/cameras/{id}/stream	GET	Danh sách phát HLS	Công khai
/api/cameras/{id}/stream/{tệp}	GET	Phân đoạn video HLS, có hai biến thể đường dẫn để tương thích cách trình duyệt sinh URL tương đối	Công khai
/api/cameras/{id}/snapshot	GET	Ảnh JPEG chụp từ luồng camera	Công khai

Bảng 4.7. Danh sách API điều khiển tại node biên

API của node biên tách phần đọc và phần ghi theo mức độ rủi ro. Các endpoint chỉ đọc đều mở, vì chúng chỉ trả trạng thái kỹ thuật và luồng video mà node vốn được giao phục vụ. Ba endpoint ghi nằm dưới nhóm /action và bắt buộc có token, kèm giới hạn tốc độ ba mươi yêu cầu mỗi phút cho mỗi địa chỉ IP theo cửa sổ trượt, vì đây là các thao tác làm thay đổi hành vi xét vi phạm. Mọi phản hồi của node đều kèm tiêu đề bảo mật: không suy diễn kiểu nội dung, không cho phép nhúng khung và không lưu bộ nhớ đệm.

Mã	Ý nghĩa trong hệ thống
200 OK	Yêu cầu đọc thành công, hoặc thao tác hiệu chuẩn đã được node áp dụng
201 Created	Đã tạo hồ sơ vi phạm mới hoặc đã nhận ảnh bằng chứng
400 Bad Request	Dữ liệu đầu vào không hợp lệ, ví dụ thiếu event_id hoặc toạ độ không phải số
401 Unauthorized	Thiếu hoặc sai thông tin xác thực: JWT hết hạn, sai chữ ký, hoặc token nhận dữ liệu không khớp
403 Forbidden	Đã xác thực nhưng vai trò không đủ quyền, ví dụ OPERATOR cố gắng duyệt hồ sơ
404 Not Found	Không tìm thấy hồ sơ, node hoặc ảnh theo định

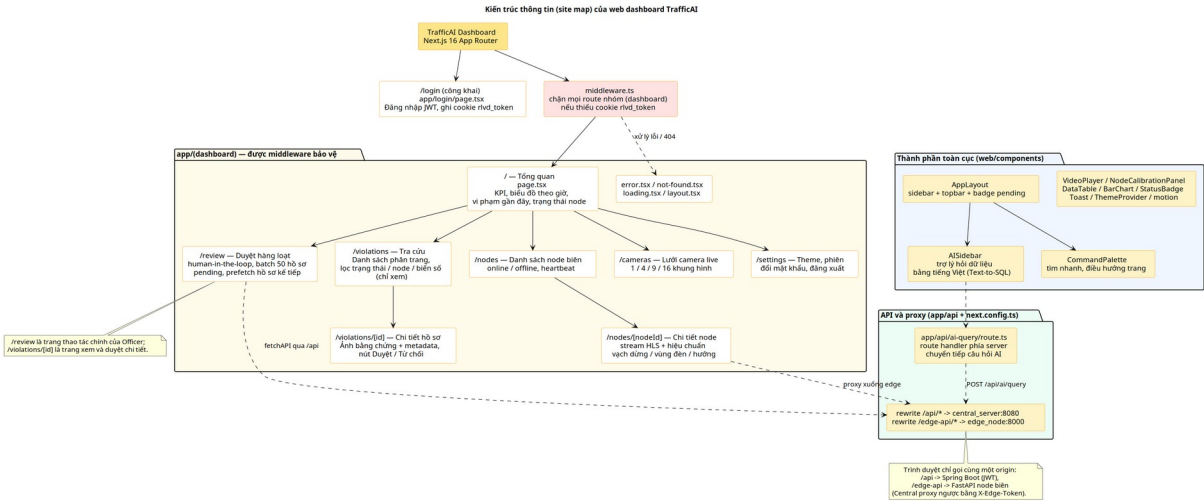
	danh đã cho
409 Conflict	Mã sự kiện đã tồn tại khi nhận một hồ sơ đơn lẻ. Ở chế độ nhận theo lô, trùng lặp không gây lỗi mà được đếm riêng
422 Unprocessable Entity	Thân yêu cầu không đọc được. Đây là dấu hiệu đặc trưng của lỗi nâng cấp giao thức HTTP khi trung tâm gọi ngược xuống node
429 Too Many Requests	Vượt giới hạn tốc độ của API điều khiển node; phản hồi kèm các tiêu đề X-RateLimit để client biết khi nào được gọi lại
502 Bad Gateway	Trung tâm không kết nối được node biên khi chuyển tiếp yêu cầu hiệu chuẩn
503 Service Unavailable	Node còn chạy nhưng pipeline ngừng xử lý khung hình quá mười giây

Bảng 4.8. Mã trạng thái HTTP và ý nghĩa trong hệ thống

Thiết kế phản hồi cho thao tác nhận dữ liệu theo lô cần được nói rõ vì nó khác với thói quen thông thường. Phản hồi gồm bốn phần: số bản ghi được chấp nhận, số bản ghi trùng, số bản ghi lỗi, cùng danh sách mã sự kiện của hai nhóm đầu. Một lô có bản ghi trùng vẫn trả mã thành công, vì trùng lặp là kết quả bình thường của cơ chế gửi lại chứ không phải lỗi. Node căn cứ vào danh sách mã sự kiện được chấp nhận để đánh dấu bản ghi đã gửi trong kho outbox; nếu phản hồi chỉ gồm một con số tổng, node sẽ không biết bản ghi nào cần đánh dấu.

4.6. UI/UX Design

Bảng điều khiển web phục vụ ba nhóm người dùng có nhịp làm việc rất khác nhau, nên kiến trúc thông tin được tổ chức theo nhiệm vụ thay vì theo cấu trúc dữ liệu. Hình 4.8 thể hiện sơ đồ trang và cơ chế bảo vệ ở tầng middleware.



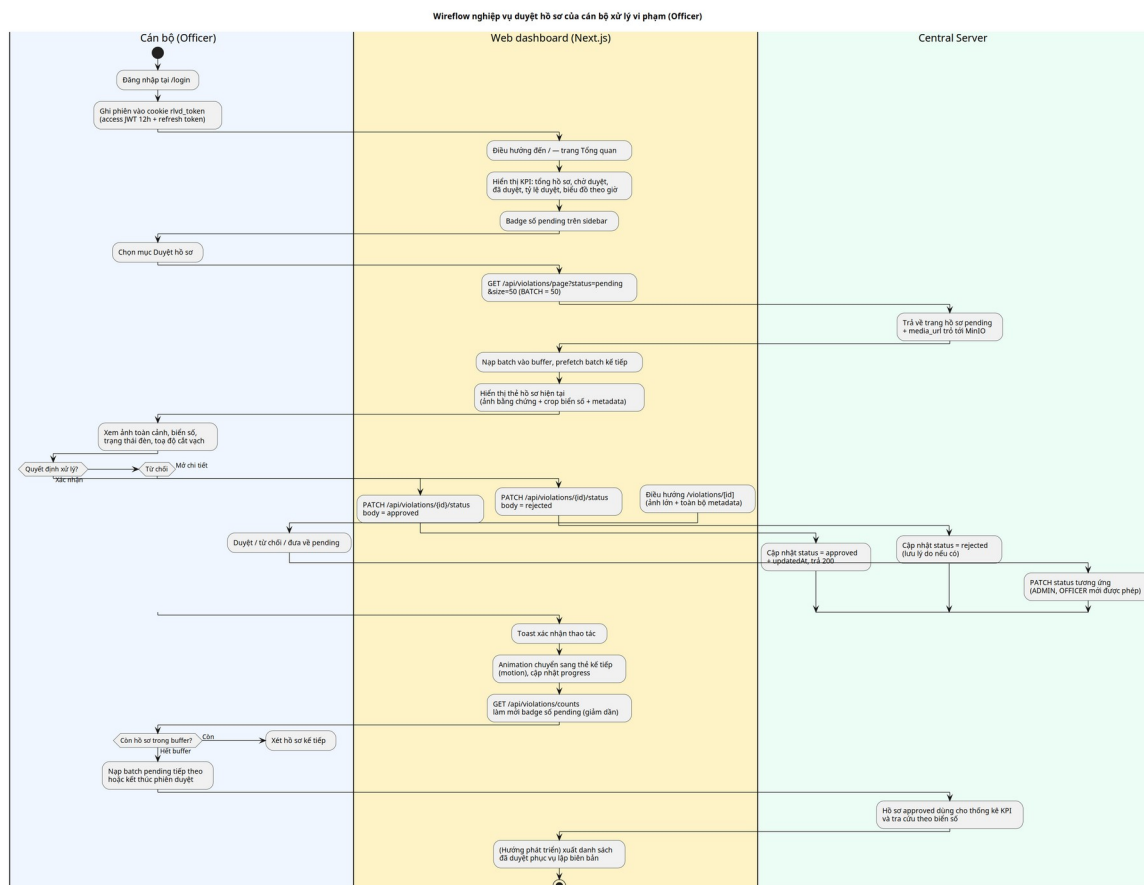
Hình 4.8. Kiến trúc thông tin của bảng điều khiển web

Toàn bộ trang nghiệp vụ nằm trong một nhóm tuyến đường được middleware bảo vệ: yêu cầu đi qua middleware trước khi tới trang, nếu thiếu cookie phiên thì bị chuyển về trang đăng nhập. Cách này tránh tình trạng mỗi trang tự kiểm tra phiên theo một kiểu khác nhau. Trang đăng nhập là trang duy nhất nằm ngoài nhóm.

Nguyên tắc thiết kế	Thể hiện cụ thể trên giao diện
Ngữ cảnh địa phương	Toàn bộ nhãn, thông báo lỗi và định dạng ngày giờ bằng tiếng Việt, theo ngôn ngữ vi-VN và múi giờ châu Á Thành phố Hồ Chí Minh. Người duyệt không phải quy đổi thời gian khi đối chiếu với biên bản
Bảng chứng trước, quyết định sau	Ở trang duyệt hồ sơ, ảnh bằng chứng chiếm phần lớn diện tích hiển thị; các trường metadata xếp bên cạnh; nút quyết định nằm ở vị trí cố định để thao tác lặp lại không phải di chuyển con trỏ
Trạng thái luôn nhìn thấy được	Huy hiệu số hồ sơ chờ duyệt trên thanh điều hướng, chấm trạng thái node trong danh sách, nhân độ tin cậy đặt cạnh giá trị nhận dạng để người dùng biết mức độ nên tin tưởng
Không phá vỡ công việc đang làm	Trang duyệt tải trước hồ sơ kế tiếp theo lô năm mươi bản ghi; khi người dùng ra quyết định, hồ sơ mới đã sẵn sàng nên không có khoảng chờ
Thao tác nguy hiểm phải xác nhận	Xoá hồ sơ chỉ dành cho vai trò quản trị và yêu cầu xác nhận; thao tác đặt lại trạng thái về chờ duyệt bị vô hiệu hoá khi hồ sơ đang ở đúng trạng thái đó
Phản hồi bằng hoạt ảnh có mục đích	Hoạt ảnh chỉ dùng ở hai chỗ: chuyển thẻ trong trang duyệt để người dùng biết thao tác đã được ghi nhận, và trạng thái chờ khi gọi API. Không dùng hoạt ảnh trang trí làm chậm nhịp làm việc
Hỗ trợ cả người dùng bàn phím	Bảng lệnh mở bằng tổ hợp phím tắt để tìm biển số, tìm node và chuyển trang; các ô nhập có nhãn rõ ràng

Bảng 4.9. Nguyên tắc thiết kế giao diện và cách thể hiện

Hệ thống có sáu chủ đề giao diện, trong đó hai chủ đề mô phỏng màu sắc đặc trưng của lực lượng cảnh sát giao thông với tông đỏ mận và vàng đồng, cùng các chủ đề tối, sáng và chế độ theo hệ điều hành. Lựa chọn này xuất phát từ khảo sát ở Chương 2: sản phẩm được dùng bởi cán bộ nghiệp vụ trong môi trường trực ban có ánh sáng thay đổi, nên cần cả chủ đề tối cho ca đêm lẫn chủ đề sáng cho ca ngày.



Hình 4.9. Wireflow nghiệp vụ duyệt hồ sơ của cán bộ xử lý vi phạm

Hình 4.9 mô tả luồng thao tác của nghiệp vụ dùng nhiều nhất trong hệ thống. Điểm thiết kế đáng chú ý là quyết định tách trang tra cứu và trang duyệt thành hai nơi khác nhau. Ở các phiên bản đầu, danh sách hồ sơ có nút duyệt nhanh đặt ngay trên từng dòng. Cách làm này nhanh nhưng dẫn tới quyết định được đưa ra khi người duyệt chưa xem ảnh bằng chứng ở kích thước đủ lớn. Phiên bản hiện hành chuyển toàn bộ thao tác quyết định vào trang chi tiết hồ sơ, còn danh sách chỉ để tra cứu; trang duyệt hàng loạt giữ vai trò luồng làm việc liên tục cho ca trực. Đây là ví dụ về việc thiết kế giao diện phải phục vụ tính đúng đắn của quyết định nghiệp vụ, không chỉ phục vụ tốc độ thao tác.

Công cụ hiệu chuẩn là thành phần giao diện phức tạp nhất. Người vận hành xem luồng video trực tiếp của node, chọn chế độ vẽ, kéo chuột trên khung hình để tạo vạch dừng hoặc khoanh vùng đèn, chọn hướng giám sát rồi gửi lên node. Ba vấn đề kỹ thuật phải giải quyết trong thiết kế: tọa độ chuột phải được quy đổi từ kích thước hiển thị về kích thước gốc của khung hình, nếu không vạch sẽ lệch khi cửa sổ thay đổi; lớp vẽ phải nằm trên video mà không làm giạt luồng phát, nên việc vẽ lại được gắn vào vòng lặp hoạt ảnh của trình duyệt thay vì vẽ theo sự kiện chuột; và giá trị hiệu chuẩn phải được

đọc lại từ node sau khi gửi để xác nhận, tránh trường hợp giao diện hiển thị một đường vạch còn node lại đang áp dụng đường khác.

CHƯƠNG 5. CÀI ĐẶT VÀ PROTOTYPE

Chương này mô tả kết quả cài đặt: quy mô mã nguồn, cách hiện thực từng tầng, cấu hình cơ sở dữ liệu và hạ tầng triển khai, cuối cùng là phần giới thiệu sản phẩm chạy thật kèm ảnh chụp giao diện.

5.1. Frontend

Tầng	Công nghệ	Số tệp	Số dòng mã
Node biên	Python 3.11, FastAPI, OpenCV, ultralytics, supervision	30	6.597
Máy chủ trung tâm	Java 17, Spring Boot 3.3.2, Spring Data JPA, jjwt, MinIO SDK	45	3.525
Bảng điều khiển web	Next.js 16, React 19, TypeScript 5, Tailwind CSS 4, hls.js, recharts, motion	35	6.547
Kiểm thử	pytest	8	1.717
Tổng cộng		118	18.386

Bảng 5.1. Quy mô mã nguồn theo tầng

Bảng điều khiển web dùng kiến trúc App Router của Next.js với một nhóm tuyến đường được bảo vệ. Chín tuyến đường phục vụ đủ các nghiệp vụ đã phân tích ở Chương 3.

Tuyến đường	Chức năng cài đặt
/login	Biểu mẫu đăng nhập, gọi API xác thực, lưu phiên và chuyển về trang tổng quan; thông báo lỗi bằng tiếng Việt
/	Bảng điều khiển tổng quan: bốn thẻ chỉ số, biểu đồ phân bố vi phạm theo giờ, danh sách hồ sơ gần đây, trạng thái các node
/violations	Tra cứu hồ sơ có phân trang phía máy chủ, lọc theo trạng thái, node và biển số; chỉ đọc, không có thao tác duyệt tại đây
/violations/{id}	Chi tiết hồ sơ: ảnh bằng chứng, biển số, trạng thái đèn, toạ độ cắt vạch, nhật ký kiểm toán và ba nút quyết định
/review	Duyệt hàng loạt theo lô năm mươi hồ sơ chờ, tải trước hồ sơ kế tiếp, ảnh bằng chứng lớn, hiệu

	ứng chuyển thẻ
/nodes	Danh sách node biên kèm trạng thái trực tuyến suy ra từ mốc heartbeat
/nodes/{nodeId}	Chi tiết node: luồng video trực tiếp, công cụ hiệu chuẩn vạch dừng, vùng đèn và hướng giám sát
/cameras	Lưới xem camera trực tiếp với bốn bố cục một, bốn, chín và mười sáu khung hình
/settings	Chọn chủ đề giao diện, xem thông tin phiên, đổi mật khẩu và đăng xuất

Bảng 5.2. Các trang của bảng điều khiển web

Hai quyết định cài đặt ở tầng web đáng được ghi lại:

- Trình duyệt không bao giờ gọi trực tiếp tới trung tâm hay node. Tập cấu hình `next.config.ts` khai báo ba luật viết lại đường dẫn: mọi yêu cầu tới `/api` được chuyển tới máy chủ trung tâm, yêu cầu tới `/edge-api` được chuyển tới node biên. Nhờ vậy cấu hình nội bộ không lộ ra phía client và không phát sinh vấn đề chia sẻ tài nguyên khác nguồn. Địa chỉ đích được cố định lúc biên dịch ảnh, nên ảnh Docker của tầng web nhận địa chỉ theo tên dịch vụ trong mạng nội bộ.
- Thư viện phát video `hls.js` được tách thành mô-đun riêng và nạp bằng cách nhập tĩnh trong thành phần phát video. Lý do là thư viện này can thiệp vào đối tượng toàn cục của trình duyệt; việc nạp theo kiểu động từng gây lỗi khi trang được dựng phía máy chủ.

Thành phần phát video tự vẽ lớp phủ bằng vòng lặp hoạt ảnh của trình duyệt. Ở cách làm trước đó, mỗi sự kiện vẽ lại kích hoạt một lượt kết xuất React, và khi người dùng kéo vạch hiệu chuẩn thì luồng video bị giật thấy rõ. Chuyển sang vòng lặp hoạt ảnh giúp việc vẽ lớp phủ tách khỏi chu trình kết xuất, luồng video giữ được nhịp ổn định.

Mô-đun	Trách nhiệm
<code>lib/api.ts</code>	Bọc toàn bộ lời gọi mạng. Tự gắn tiêu đề xác thực, tự làm mới token đúng một lần khi gặp mã 401 rồi thử lại yêu cầu gốc, tự thử lại với lỗi mạng và lỗi phía máy chủ; không thử lại với lỗi phía client
<code>lib/auth.ts</code>	Quản lý phiên: lưu token, đồng bộ cookie cho middleware, làm mới và xoá phiên
<code>lib/types.ts</code>	Định nghĩa kiểu TypeScript phản chiếu DTO của máy chủ trung tâm
<code>lib/nodes.ts</code>	Chuẩn hoá trạng thái node từ mốc heartbeat
<code>components/VideoPlayer.tsx</code>	Phát luồng HLS và vẽ lớp phủ hiệu chuẩn bằng vòng lặp hoạt ảnh

components/NodeCalibrationPanel.tsx	Ba chế độ vẽ và chọn hướng giám sát, quy đổi tọa độ hiển thị về tọa độ gốc
components/AppLayout.tsx	Khung giao diện chung: thanh điều hướng, thanh trên, huy hiệu số hồ sơ chờ duyệt
components/AISidebar.tsx	Bảng trợ lý trượt từ cạnh phải, hiển thị câu lệnh SQL đã sinh cùng bảng kết quả hoặc biểu đồ
components/CommandPalette.tsx	Bảng lệnh tìm nhanh và chuyển trang bằng bàn phím
components/DataTable.tsx, BarChart.tsx, StatusBadge.tsx, Toast.tsx, ThemeProvider.tsx, motion.tsx	Các thành phần dùng lại: bảng dữ liệu, biểu đồ cột, huy hiệu trạng thái, thông báo ngắn, quản lý chủ đề và hiệu ứng chuyển động

Bảng 5.3. Các mô-đun chính của tầng web

5.2. Backend

5.2.1. Node biên

Node biên là phần cài đặt nhiều thử nghiệm nhất của đồ án, vì đây là nơi quyết định chất lượng phát hiện. Bảng 5.4 liệt kê các mô-đun chính.

Mô-đun	Số dòng	Trách nhiệm và ghi chú cài đặt
core/byte_tracker.py	824	Lớp kế thừa có tinh chỉnh của ByteTrack trong thư viện supervision. Mở cấu hình cho khối theo dõi chưa xác nhận vốn bị cố định trong thư viện, thêm cơ chế lọc theo chuyển động và làm mượt độ tin cậy bằng trung bình trượt có trọng số, bỏ phiếu nhân qua nhiều khung hình
core/motion.py	158	Mô hình chuyển động của đối tượng theo dõi: tích lũy quỹ đạo, ước lượng vận tốc bằng hồi quy bình phương tối thiểu theo thời gian thực thay vì theo số khung hình, suy ra nhãn hướng trong tám hướng
core/detector.py	234	Bọc mô hình phát hiện phương tiện đã huấn luyện lại. Giữ nguyên tên lớp do mô hình sinh ra, không ánh xạ về bộ nhãn COCO
core/traffic_light_yolo.py	325	Phân loại màu đèn bằng mô

		hình học sâu, kết hợp bằng chứng màu trong không gian HSV và vị trí bóng đèn đang sáng trong vùng quan sát
core/traffic_light_cv.py		Bộ phân loại dự phòng dùng HSV thuần, đồng thời đảm nhận việc tự tìm vùng quan sát đèn khi chưa được hiệu chuẩn
core/violation_logic.py	305	Ba lớp: RedLightStabilizer làm ổn định trạng thái đèn, Tripwire mô hình hoá vạch dừng có hướng, ViolationDetector kết hợp hai thành phần trên với danh sách đối tượng theo dõi để sinh sự kiện vi phạm
core/plate_detector.py, plate_associator.py, ocr_recognizer.py		Phát hiện vùng biển số, gắn biển vào đối tượng theo dõi theo quan hệ bao chứa, đọc ký tự bằng fast-plate-ocr và lưu kết quả đọc tốt nhất theo từng đối tượng
core/vn_plate.py	180	Bộ kiểm tra cấu trúc biển số Việt Nam và sửa các lỗi nhận dạng ký tự thường gặp; là cổng lọc cuối cùng trước khi biển số được ghi vào hồ sơ
core/visualizer.py	531	Vẽ ảnh bằng chứng: hộp phương tiện, vạch dừng, nhãn đối tượng, thông tin đèn và biển số
core/pipeline.py	352	Điều phối bảy bước xử lý mỗi khung hình và ghi hồ sơ vào kho outbox
outbox.py	223	Kho SQLite bền vững, truy cập có khoá bảo vệ để dùng an toàn từ hai luồng
violation_sender.py	220	Luồng gửi dữ liệu nền: lấy bản ghi chờ, gửi theo lô, tải ảnh lên sau, lùi theo hàm mũ khi thất bại
api/server.py	739	API điều khiển bằng FastAPI: kiểm tra sức khoẻ, phục vụ camera và HLS, nhận lệnh hiệu chuẩn có token và giới hạn tốc

		độ
camera_stream.py	139	Gọi FFmpeg để phát luồng HLS với phân đoạn bốn giây, tự khởi động lại khi gián đoạn, xoá phân đoạn cũ
central_client.py		Đăng ký node lên trung tâm và duy trì heartbeat mỗi sáu mươi giây; vẫn tiếp tục chạy khi trung tâm không phản hồi
main.py	433	Khởi động toàn bộ: nạp cấu hình, tạo pipeline, mở API điều khiển, bật luồng gửi dữ liệu
settings.py	254	Cấu hình tập trung đọc từ biến môi trường, đóng băng sau khi khởi tạo
metrics.py		Bộ đếm trong tiến trình và phát hiện pipeline ngừng hoạt động

Bảng 5.4. Các module chính của node biên

Bốn điểm cài đặt thể hiện rõ nhất quá trình thử nghiệm của đồ án:

- Điểm neo để xét cắt vạch là điểm đáy giữa của hộp giới hạn, tức điểm phương tiện tiếp xúc mặt đường. Lựa chọn này từng được đổi sang tâm hộp trong một phiên bản trước để bắt được trường hợp phương tiện mới nhô đầu qua vạch, nhưng gây kết luận sớm với xe nghiêng. Phiên bản hiện hành quay lại điểm đáy giữa và chấp nhận kết luận muộn hơn một đến hai khung hình; nếu cần phát hiện sớm kiểu nhô đầu qua vạch, việc đó phải làm ở tầng máy trạng thái xét vi phạm chứ không đổi điểm neo.
- Ngưỡng làm ổn định đèn được quy đổi từ giây sang khung hình theo tốc độ khung hình của nguồn video. Trước đây ngưỡng là số khung hình cố định, dẫn tới độ trễ chuyển trạng thái khác nhau giữa các camera: bảy khung hình ở nguồn ba khung hình mỗi giây là hơn hai giây, ở nguồn ba mươi khung hình chỉ hơn hai trăm mili giây. Sau khi quy đổi theo giây, độ trễ đo được là 0,66 giây ở nguồn sáu khung hình và 0,67 giây ở nguồn ba khung hình.
- Vạch dừng đang áp dụng được đọc lại ở mỗi khung hình từ biến toàn cục có khoá bảo vệ, thay vì giữ trong thuộc tính của bộ xét vi phạm. Nhờ vậy lệnh hiệu chuẩn có hiệu lực từ khung hình kế tiếp mà không cần khởi động lại tiến trình.
- Mỗi khung hình được xử lý trong một khối bắt lỗi riêng. Khung hình hỏng hay mô hình lỗi chỉ sinh ra một dòng cảnh báo rồi vòng lặp tiếp tục, vì một node dừng giữa ca trực sẽ bỏ lọt toàn bộ vi phạm trong khoảng thời gian đó.

5.2.2. Máy chủ trung tâm

Máy chủ trung tâm cài đặt 45 tệp Java với 3.525 dòng mã, tổ chức theo sáu gói chuẩn của Spring Boot.

Gói	Số lớp	Nội dung cài đặt
controller	5	AuthController (4 phương thức), ViolationController (13), EdgeNodeController (7), MediaController (4), AiQueryController (1). Chỉ nhận yêu cầu, kiểm tra dữ liệu đầu vào và gọi service
service	7	ViolationService (361 dòng, nhận dữ liệu idempotent và truy vấn có phân trang), ViolationStatsService (179 dòng, thống kê bằng truy vấn tổng hợp ở cơ sở dữ liệu), RefreshTokenService (207 dòng, băm SHA-256, xoay vòng token, phát hiện dùng lại), EdgeProxyService (188 dòng, gọi ngược xuống node bằng HTTP/1.1), EdgeNodeService, AiQueryService (226 dòng, sinh câu lệnh SQL và kiểm tra an toàn), MinioStorageService
repository	4	Giao diện Spring Data JPA. Các truy vấn thường dùng được khai báo thành phương thức đặt tên; truy vấn tổng hợp dùng chú thích Query để chỉ chạy đếm và nhóm, không tải thực thể vào bộ nhớ
entity	4	Violation, EdgeNode, User, RefreshToken. Chỉ mục được khai báo ngay trong chú thích Table để lược đồ và mã nguồn không lệch nhau
dto	16	Đối tượng truyền dữ liệu cho yêu cầu và phản hồi, tách biệt hoàn toàn với thực thể để thay đổi lược đồ không ảnh hưởng tới giao tiếp API
config	8	SecurityConfig (chuỗi lọc bảo

		mật và phân quyền), JwtService, JwtAuthFilter, IngestTokenFilter, IngestPaths (nguồn khai báo duy nhất cho đường dẫn nhận dữ liệu), MinioConfig, WebConfig, UserSeeder
--	--	--

Bảng 5.5. Các lớp chính của máy chủ trung tâm

Ba chi tiết cài đặt ở tầng trung tâm xuất phát từ sự cố gặp phải trong quá trình phát triển, nên cần được ghi lại:

- EdgeProxyService cố định giao thức HTTP/1.1 khi gọi ngược xuống node. Node biên chạy trên uvicorn, và khi client đề nghị nâng cấp lên HTTP/2, phần thân yêu cầu POST bị bỏ rơi khiến node trả mã 422 và trung tâm báo 502 về cho web. Sự cố này rất khó truy nguyên vì lỗi nằm ở tầng giao thức chứ không ở dữ liệu.
- Chuỗi lọc bảo mật khai báo đường dẫn bằng cặp phương thức và mẫu đường dẫn, không ghép thành một chuỗi. Trên phiên bản Spring Security đang dùng, chuỗi ghép dạng phương thức cộng khoảng trắng cộng đường dẫn bị hiểu là một mẫu đường dẫn duy nhất nên không bao giờ khớp, dẫn tới mọi yêu cầu nhận dữ liệu bị từ chối dù token đúng.
- Ứng dụng tắt theo cơ chế êm với thời gian chờ ba mươi giây. Khi nâng cấp, một lô dữ liệu đang ghi dở sẽ được hoàn tất thay vì bị ngắt giữa chừng.

Bộ trợ lý hỏi dữ liệu bằng tiếng Việt hoạt động theo bốn bước: dựng lời nhắc gồm mô tả lược đồ bốn bảng và yêu cầu chỉ viết câu lệnh SELECT, gửi tới mô hình ngôn ngữ qua giao diện REST của java.net.http, kiểm tra câu lệnh nhận về bằng bộ chặn từ khóa nguy hiểm rồi tự thêm giới hạn năm mươi dòng, cuối cùng thực thi bằng truy vấn chỉ đọc và trả về kết quả kèm gợi ý loại biểu đồ. Việc kiểm tra được đặt sau khi nhận câu lệnh chứ không đặt trong lời nhắc, vì lời nhắc chỉ là yêu cầu còn bộ kiểm tra mới là ràng buộc thực sự.

5.3. Database

Cơ sở dữ liệu PostgreSQL 16 chạy trong container, dữ liệu đặt trên volume riêng để không mất khi tạo lại container. Cơ sở dữ liệu tên rlvd_central, tài khoản truy cập và mật khẩu đọc từ tệp biến môi trường; giá trị mặc định chỉ dùng cho phát triển nội bộ.

Ảnh bằng chứng lưu trong MinIO, dịch vụ lưu trữ đối tượng tương thích giao diện S3, với một bucket tên violations. Khoá đối tượng theo mẫu violations/{event_id}/{uuid}.jpg; bảng violations chỉ lưu khoá này trong cột media_url chứ không lưu đường dẫn tuyệt đối, để việc di chuyển hệ thống lưu trữ

không đòi hỏi cập nhật dữ liệu. Ảnh được phát lại cho trình duyệt qua một endpoint trung gian của máy chủ trung tâm, nên bucket không cần mở ra ngoài.

Kho outbox của node biên đặt trên volume outbox_data, thư mục HLS đặt trên volume hls_data. Tổng cộng bốn volume bền vững: postgres_data, minio_data, outbox_data và hls_data. Việc tách kho outbox thành volume riêng là điều kiện để node biên có thể bị tạo lại mà không mất hồ sơ chưa gửi.

Múi giờ châu Á Thành phố Hồ Chí Minh được đặt trên toàn bộ dịch vụ và trong tệp Dockerfile, để mốc thời gian không bị lệch khi so sánh giữa nhật ký node, cơ sở dữ liệu và giao diện.

5.4. Demo

Toàn bộ hệ thống được đóng gói trong tệp docker-compose.full.yml gồm năm dịch vụ chạy trên một mạng cầu nối nội bộ.

Dịch vụ	Ảnh hoặc cách dựng	Cổng máy chủ	Kiểm tra sức khỏe
postgres	postgres:16-alpine	5432	pg_isready, mỗi mười giây
minio	minio/minio:latest	9010 (API), 9011 (bảng điều khiển)	mc ready local, mỗi mười giây
central-server	Dựng hai giai đoạn bằng Maven trong image maven:3.9-eclipse-temurin-17	8002 → 8000	wget /api/health, mỗi mười giây, cho phép ba mươi giây khởi động
web-dashboard	Dựng Next.js ở chế độ standalone, chạy bằng tài khoản không phải root	3000	Chờ central-server khỏe mới khởi động
edge-pipeline	Dựng từ image python:3.11-slim, cài PyTorch bản CUDA 12.4, nhận card đồ họa qua khai báo device reservation	8082 → 8080	/health do node tự cung cấp

Bảng 5.6. Ảnh xạ cổng dịch vụ khi triển khai bằng Docker Compose

Hệ thống khởi động bằng một lệnh thông qua tập lệnh start.sh, tập lệnh này kiểm tra điều kiện trước khi dựng: tệp trọng số mô hình có đủ không, tệp video đầu vào có tồn tại không, tệp biến môi trường đã có chưa. Ngoài chế độ đầy đủ, tập lệnh có chế độ tối thiểu chỉ dựng bốn dịch vụ web mà bỏ node biên, dùng khi cần làm việc với giao diện

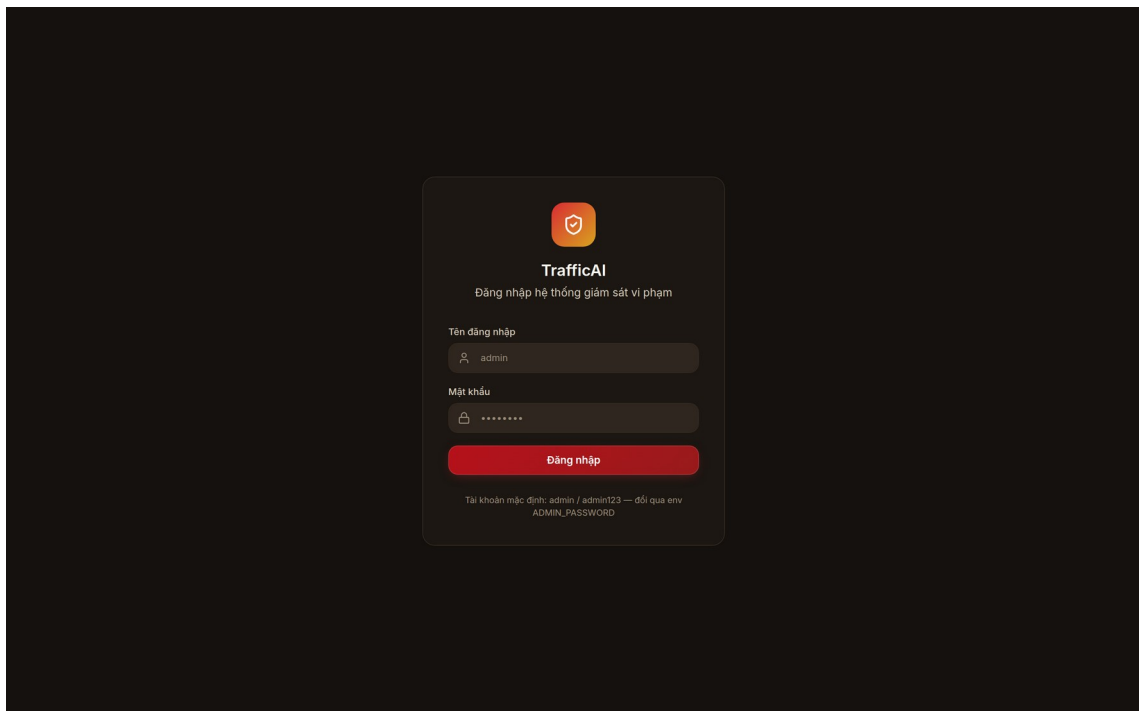
mà không cần card đồ hoạ. Các thao tác khác gồm xem nhật ký, xem trạng thái, dừng lại từng dịch vụ và dừng toàn bộ.

Biến môi trường	Tầng	Bắt buộc khi vận hành	Ý nghĩa
JWT_SECRET	Trung tâm	Có, tối thiểu 32 ký tự	Khoá ký token truy cập; ứng dụng từ chối khởi động nếu khoá yếu
JWT_TTL_SECONDS	Trung tâm	Không, mặc định 43.200	Tuổi thọ token truy cập, tương ứng một ca trực mười hai giờ
INGEST_TOKEN	Trung tâm và node	Có	Token dùng chung cho các yêu cầu nhận dữ liệu từ node
ADMIN_USERNAME, ADMIN_PASSWORD	Trung tâm	Có, phải đổi giá trị mặc định	Tài khoản quản trị tạo lần đầu; không ghi đè nếu đã tồn tại
DB_PASS, DB_HOST, DB_NAME, DB_USER	Trung tâm và postgres	Có	Thông tin kết nối PostgreSQL
MINIO_ROOT_USER, MINIO_ROOT_PASSWORD, MINIO_ACCESS_KEY, MINIO_SECRET_KEY	Trung tâm và minio	Có, phải đổi giá trị mặc định	Thông tin truy cập kho đối tượng
GEMINI_API_KEY	Trung tâm	Có nếu dùng trợ lý trí tuệ nhân tạo	Khoá gọi mô hình ngôn ngữ
EDGE_API_TOKEN	Node	Có	Token bảo vệ nhóm endpoint /action của node
EDGE_REQUIRE_TOKEN	Node	Có, đặt true khi vận hành	Bật chế độ bắt buộc kiểm tra token ở node
EDGE_ALLOWED_ORIGINS	Node	Có	Danh sách nguồn được phép gọi API điều khiển của node
NODE_ID	Node	Không, mặc định edge-node-01	Định danh node, đồng thời là thành phần của mã sự kiện

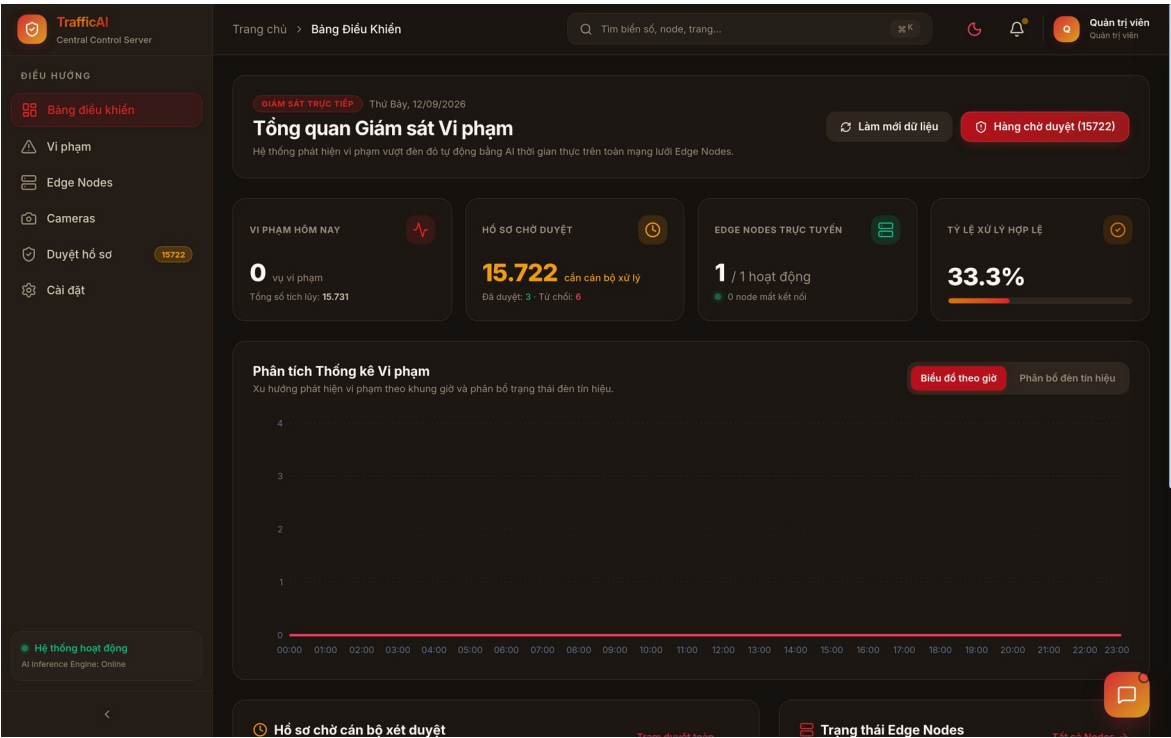
VIDEO_INPUT	Node	Có	Nguồn video: đường dẫn tệp hoặc địa chỉ luồng
YOLO_MODEL_PATH, TRAFFIC_LIGHT_MODEL_PATH	Node	Có	Đường dẫn tệp trọng số mô hình
OUTBOX_BATCH_SIZE, OUTBOX_FLUSH_INTERVAL	Node	Không, mặc định 20 và 5 giây	Kích thước lô và chu kỳ gửi dữ liệu
CENTRAL_SERVER_URL	Node và web	Có	Địa chỉ máy chủ trung tâm

Bảng 5.7. Biến môi trường cấu hình hệ thống

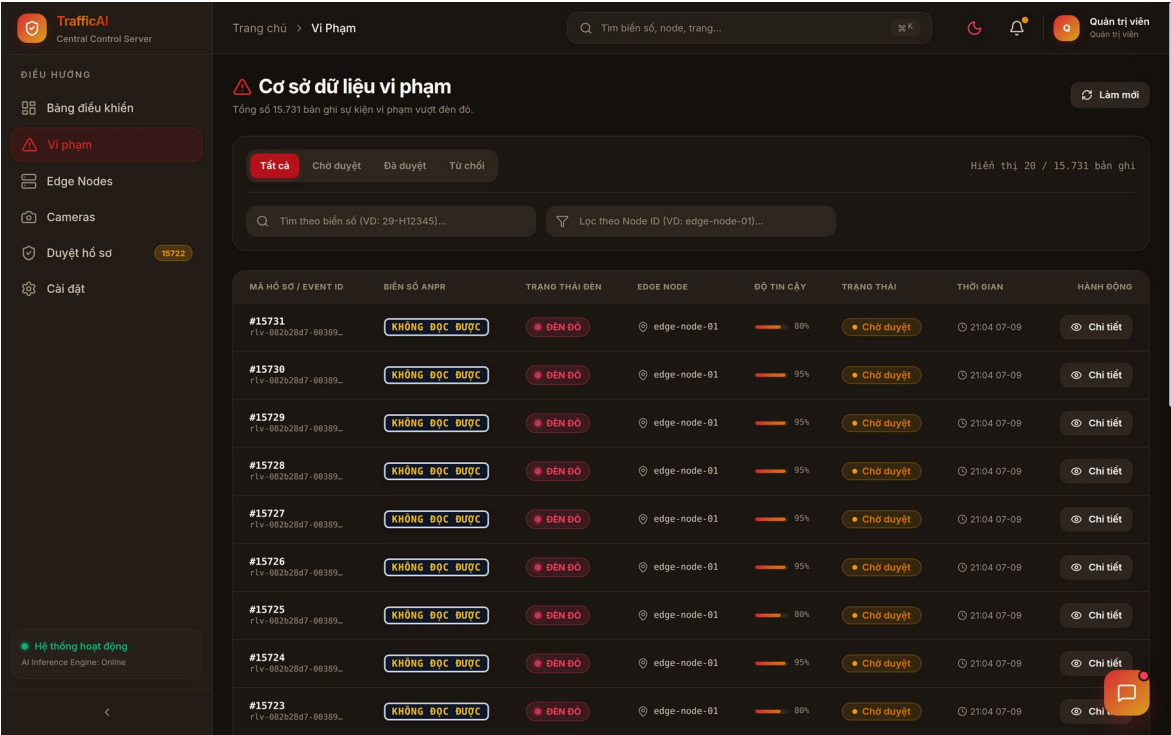
Các hình từ 5.1 đến 5.11 là ảnh chụp giao diện của hệ thống đang chạy thật trên máy phát triển, không phải ảnh dựng bằng công cụ thiết kế. Dữ liệu hiển thị là hồ sơ vi phạm do node biên sinh ra từ các tệp video thử nghiệm.



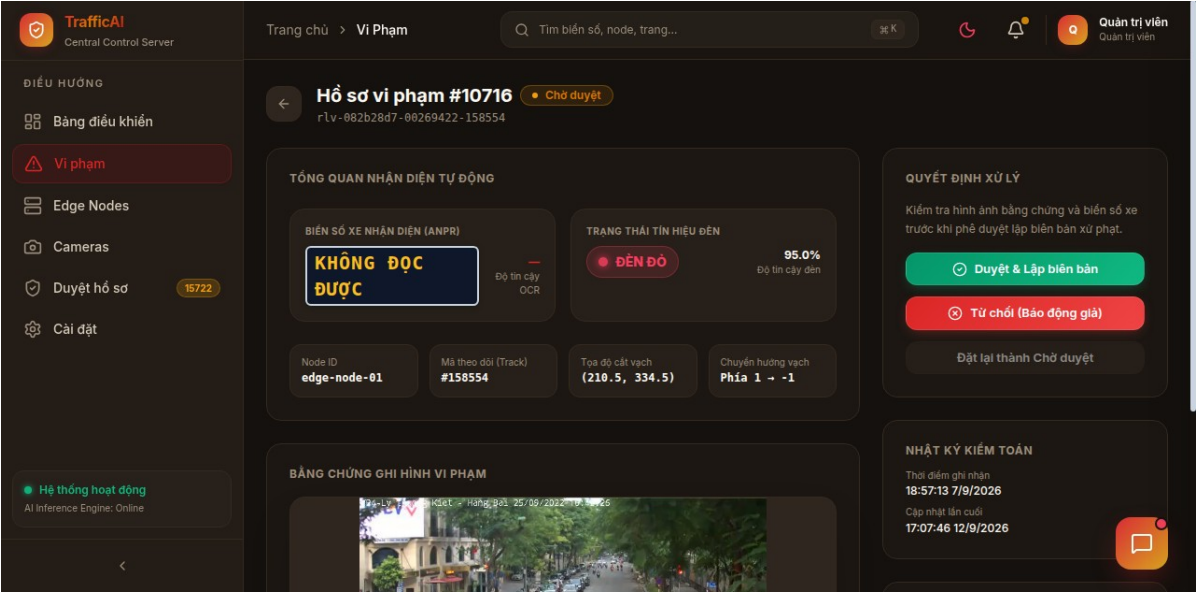
Hình 5.1. Trang đăng nhập



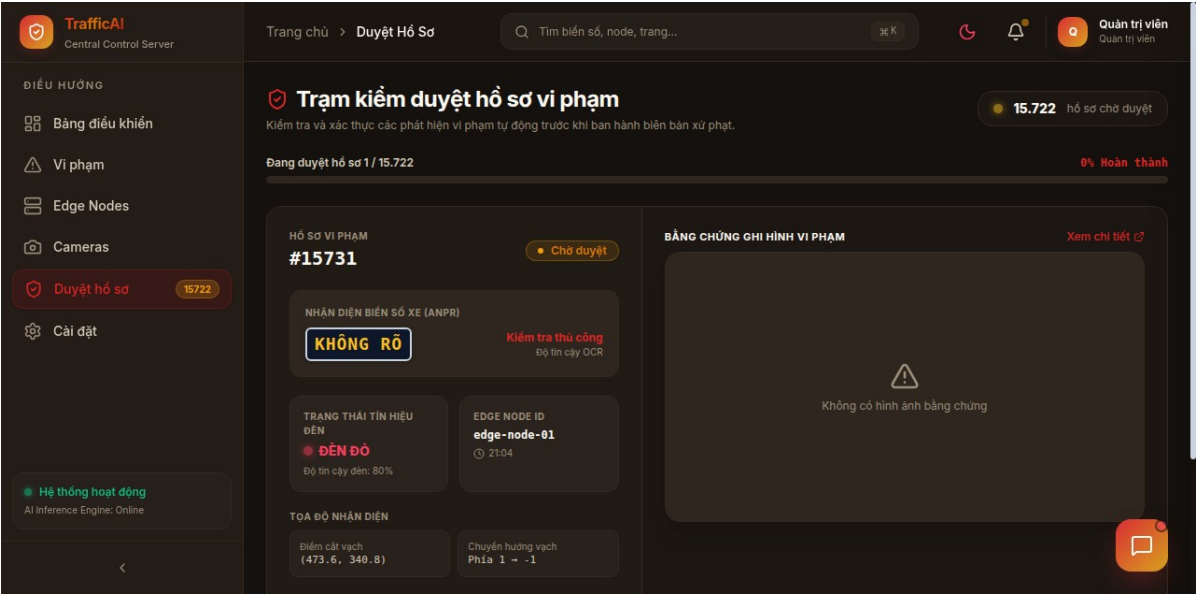
Hình 5.2. Trang bảng điều khiển tổng quan sau khi đăng nhập



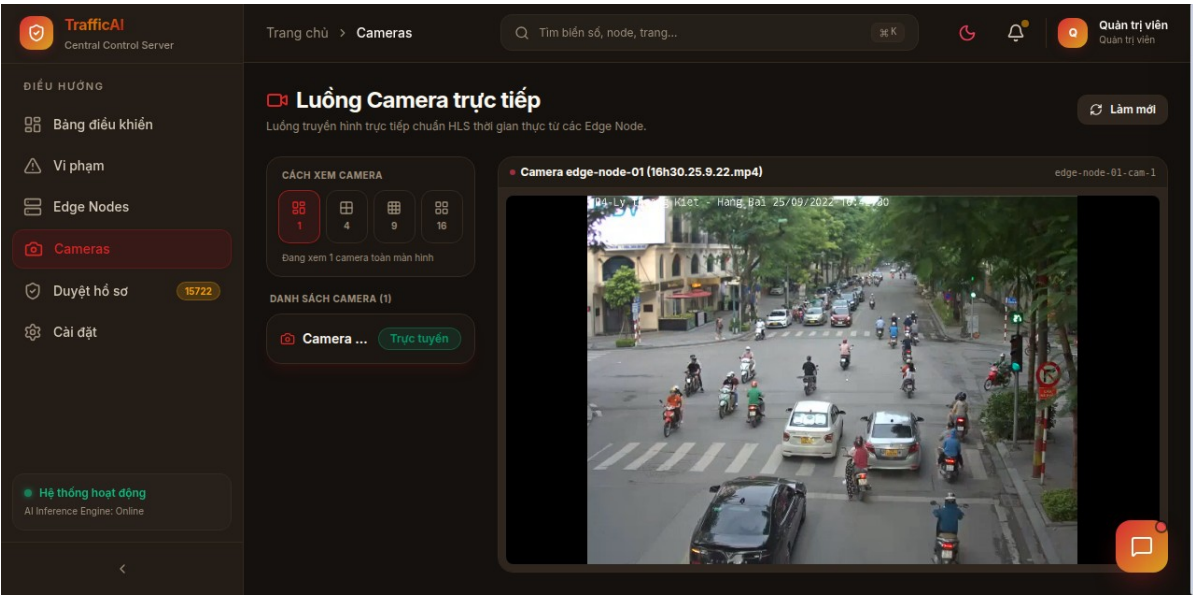
Hình 5.3. Trang tra cứu hồ sơ vi phạm có phân trang và lọc



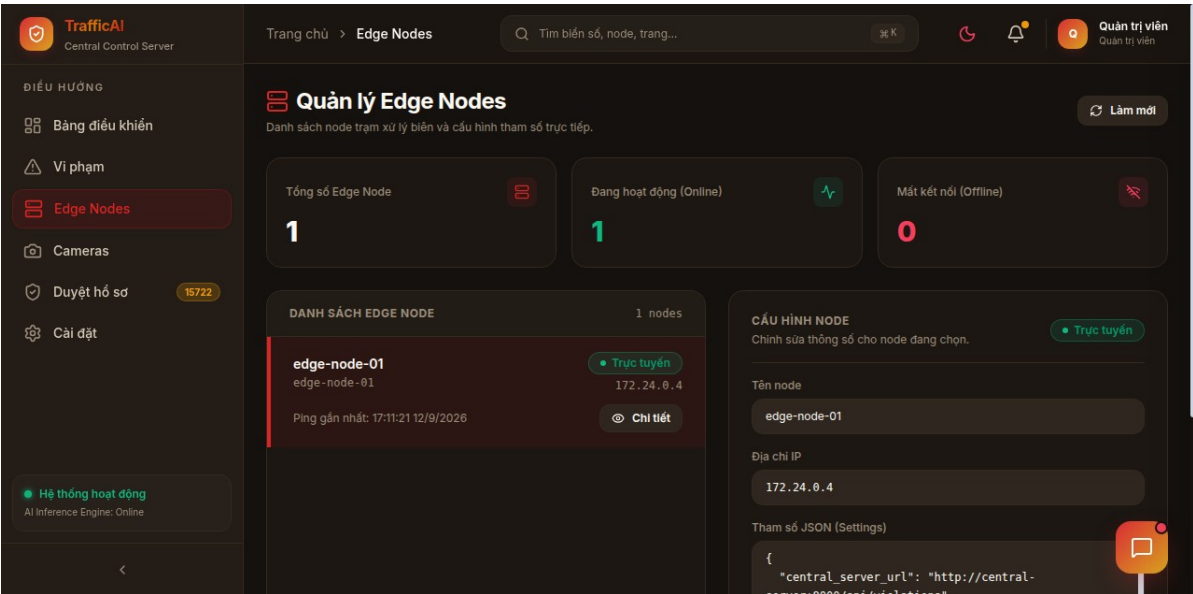
Hình 5.4. Trang chi tiết hồ sơ với ảnh bằng chứng và nút quyết định xử lý



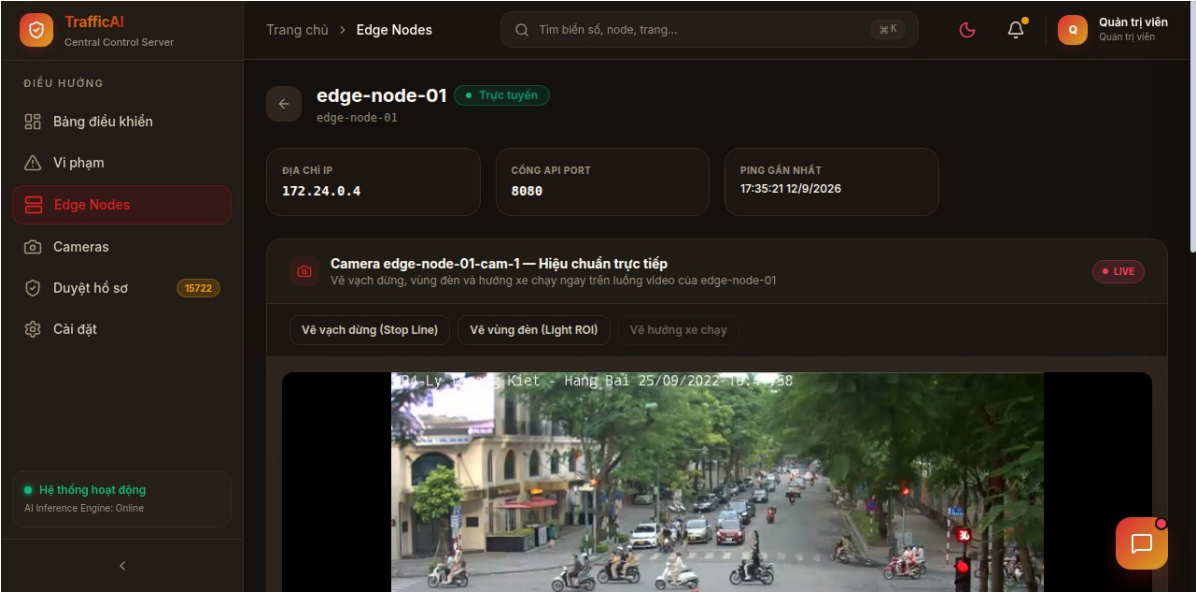
Hình 5.5. Trang duyệt hồ sơ hàng loạt



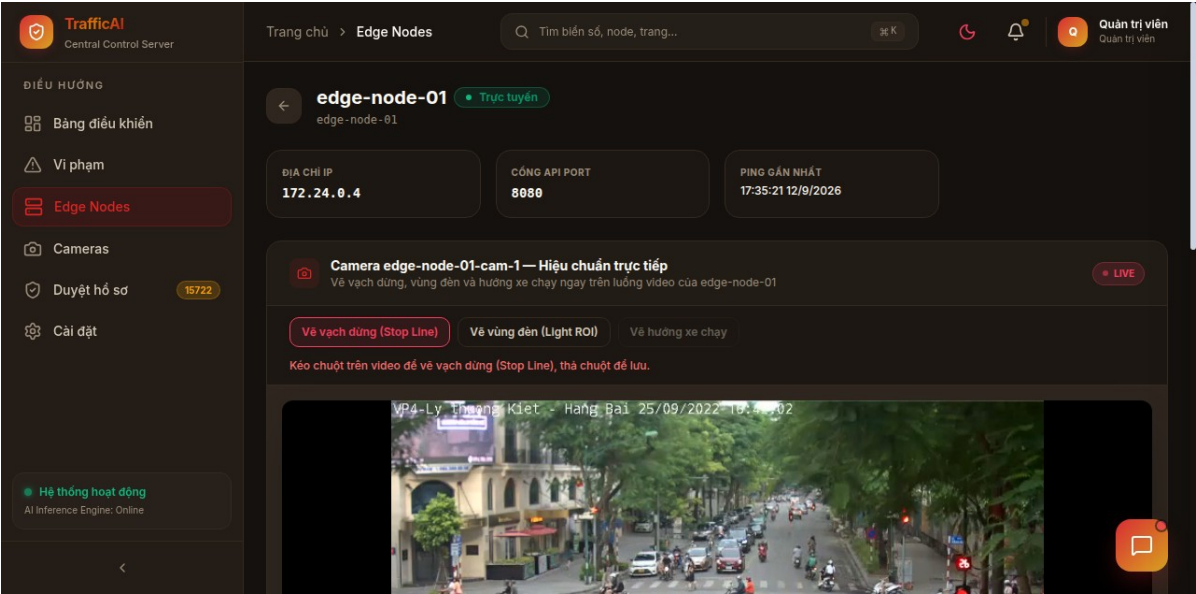
Hình 5.6. Trang xem camera trực tiếp với các bố cục lưới 1, 4, 9 và 16



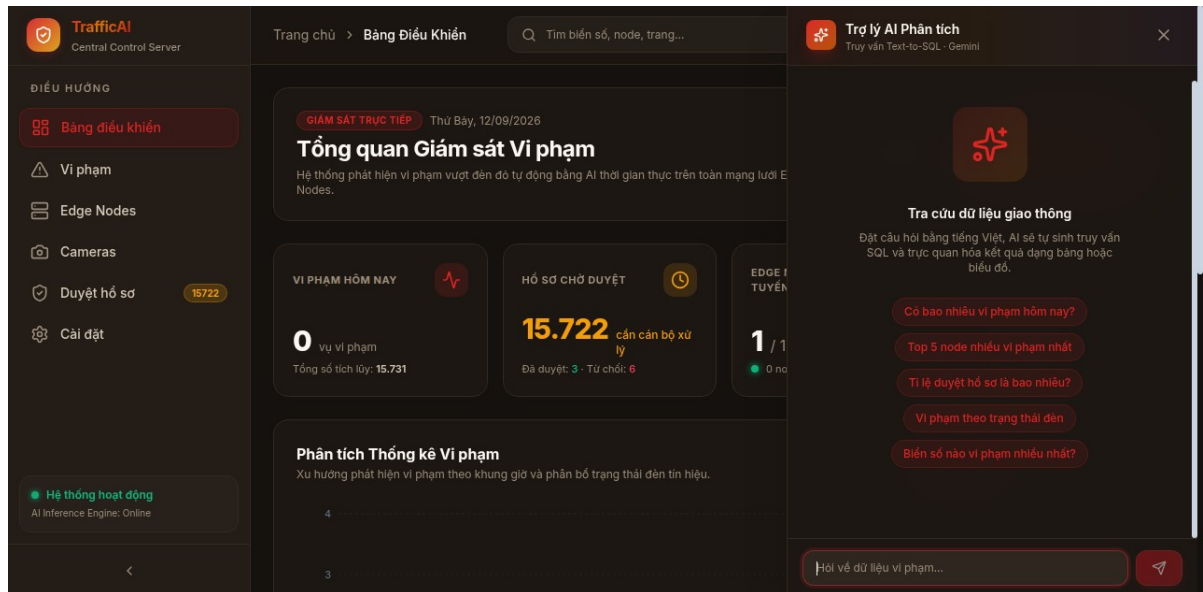
Hình 5.7. Trang danh sách node biên



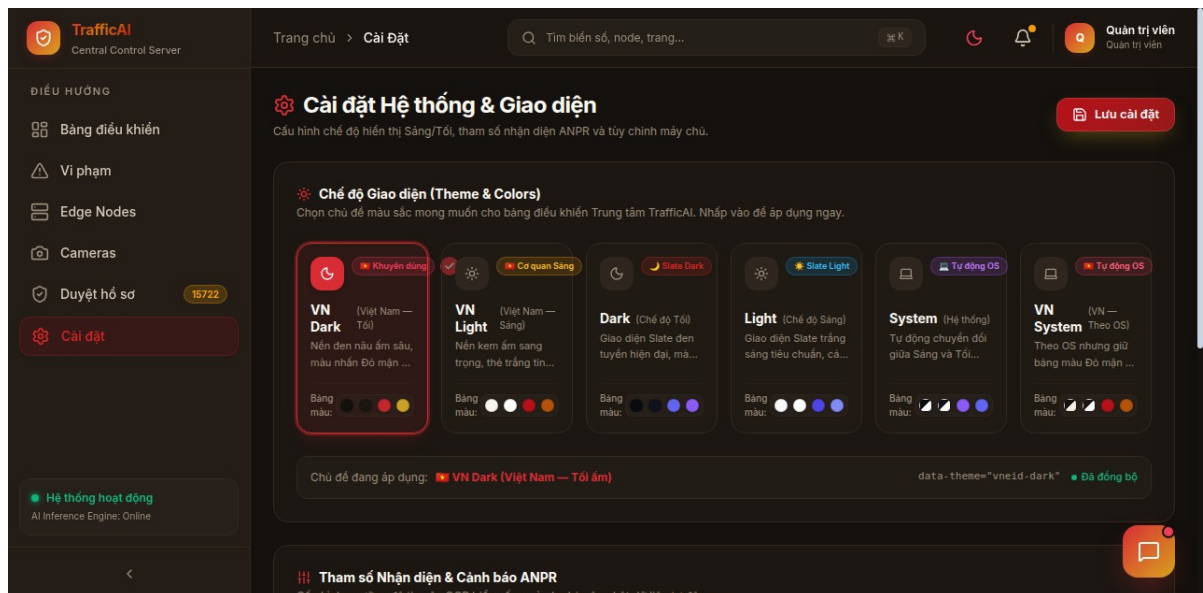
Hình 5.8. Trang chi tiết node với luồng video trực tiếp



Hình 5.9. Công cụ hiệu chuẩn vạch dừng trên video trực tiếp



Hình 5.10. Trợ lý hỏi dữ liệu bằng tiếng Việt



Hình 5.11. Trang cài đặt giao diện

Kịch bản giới thiệu sản phẩm được thực hiện theo đúng luồng nghiệp vụ đã phân tích, và là kịch bản đã chạy thật trên hệ thống:

- Khởi động hệ thống bằng một lệnh, chờ năm dịch vụ báo trạng thái khoẻ.
- Mở trang đăng nhập, đăng nhập bằng tài khoản quản trị, hệ thống chuyển về trang tổng quan với các chỉ số và biểu đồ theo giờ.
- Vào trang chi tiết node, mở công cụ hiệu chuẩn, kéo chuột trên khung hình video trực tiếp để tạo vạch dừng và chọn hướng giám sát, gửi xuống node. Đọc lại trạng thái hiệu chuẩn từ node để xác nhận giá trị đã được áp dụng.
- Chuyển sang trang camera để xem luồng video trực tiếp, đổi bố cục lưới.

- Chờ node xử lý video và sinh hồ sơ. Ở trang tổng quan, huy hiệu số hồ sơ chờ duyệt tăng lên.
- Mở trang duyệt, xem ảnh bằng chứng của một hồ sơ, đối chiếu trạng thái đèn và toạ độ cắt vạch, chọn duyệt hoặc từ chối. Hồ sơ kế tiếp hiện ra ngay mà không phải chờ tải.
- Mở bảng trợ lý, đặt một câu hỏi bằng tiếng Việt về dữ liệu vi phạm. Hệ thống trả về câu lệnh SQL đã sinh cùng bảng kết quả và biểu đồ.
- Dừng dịch vụ trung tâm để chứng minh cơ chế giao hàng bền vững: node vẫn tiếp tục phát hiện và ghi hồ sơ vào kho outbox, số bản ghi chờ tăng lên. Khởi động lại trung tâm, các hồ sơ được gửi hết mà không có bản nào bị trùng.

CHƯƠNG 6. KIỂM THỬ

Mục tiêu của chương này là trả lời một câu hỏi duy nhất: hệ thống vừa xây dựng có làm đúng những gì đã phân tích ở Chương 3 không, và đúng tới mức nào. Vì vậy kiểm thử được tổ chức theo yêu cầu chứ không theo mô-đun: mỗi yêu cầu chức năng và phi chức năng đều có ít nhất một ca kiểm thử tương ứng, và mỗi con số đưa ra đều kèm cách đo để người đọc có thể lặp lại.

Đồ án dùng ba mức kiểm thử. Mức thứ nhất là kiểm thử đơn vị bằng pytest, chạy hoàn toàn không cần card đồ hoạ hay tệp trọng số, nên chạy được trong môi trường tích hợp liên tục với mỗi lần đẩy mã lên kho. Mức thứ hai là kiểm thử tích hợp trên hệ thống đang chạy thật bằng Docker Compose, dùng yêu cầu HTTP gửi tới các dịch vụ. Mức thứ ba là kiểm thử hồi quy trên video có nhãn, tức là chạy toàn bộ pipeline trên tệp video đã biết trước có bao nhiêu vi phạm rồi đối chiếu kết quả.

6.1. Test Case

Bảng 6.1 liệt kê các ca kiểm thử chính, mỗi ca gắn với một yêu cầu ở Chương 2 để có thể truy vết ngược. Cột kết quả được điền từ lần chạy ngày 12 tháng 9 năm 2026.

Mã	Yêu cầu liên quan	Tình huống kiểm thử	Kết quả mong đợi	Cách thực hiện	Kết quả
TC-01	FR-04	Phương tiện cắt vạch dừng khi đèn đỏ đã ổn định	Sinh đúng một hồ sơ cho đối tượng đó	Kiểm thử đơn vị, dựng vết và tín hiệu đèn giả	Đạt
TC-02	FR-04	Phương tiện cắt vạch khi đèn xanh hoặc đèn vàng	Không sinh hồ sơ nào	Kiểm thử đơn vị	Đạt
TC-03	FR-04	Phương tiện đi ngược hướng giám sát đã chọn	Không sinh hồ sơ	Kiểm thử đơn vị, đặt hướng một chiều	Đạt
TC-04	FR-04	Phương tiện vào vùng chết của vạch rồi lùi lại	Không kết luận vi phạm sai	Kiểm thử đơn vị, dựng quỹ đạo dao động quanh vạch	Đạt
TC-05	FR-04	Phương tiện cắt vạch sang rồi cắt về trong cùng một lượt	Chỉ một hồ sơ duy nhất cho một đối tượng	Kiểm thử đơn vị	Đạt

TC-06	FR-03	Chưa hiệu chuẩn vạch, đèn đỏ kéo dài nhiều khung hình, sau đó kẻ vạch	Trước khi kẻ vạch không có hồ sơ; sau khi kẻ thì xét bình thường	Kiểm thử đơn vị với hàm đặt vạch đang áp dụng	Đạt
TC-07	FR-02	Bộ phân loại đèn trả kết quả nhiều xen kẽ giữa các khung hình	Tín hiệu ổn định không bị lật theo nhiều đơn lẻ	Kiểm thử đơn vị máy trạng thái có thể chuyển	Đạt
TC-08	FR-02, NFR-02	Đổi nguồn video từ ba sang ba mươi khung hình mỗi giây	Thời gian chuyển trạng thái đèn tính bằng giây không đổi	Kiểm thử đơn vị hàm quy đổi theo tốc độ khung hình	Đạt
TC-09	FR-01	Xe máy có độ tin cậy phát hiện thấp chạy nhanh qua nhiều khung hình	Vẫn giữ cùng một mã theo dõi, không nhấp nháy	Kiểm thử đơn vị bộ theo dõi đã tinh chỉnh	Đạt
TC-10	FR-01	Hai phương tiện giao nhau trong khung hình	Giữ hai mã theo dõi riêng biệt	Kiểm thử đơn vị	Đạt
TC-11	FR-06	Chuỗi biển số bị nhận dạng nhầm giữa chữ O và số 0, chữ I và số 1	Sửa đúng về dạng hợp lệ hoặc trả về rỗng	Kiểm thử đơn vị bộ kiểm tra cấu trúc biển số	Đạt
TC-12	FR-06	Biển số không theo cấu trúc Việt Nam	Bị loại, không ghi vào hồ sơ	Kiểm thử đơn vị và đo thực tế ở mục 6.4	Đạt
TC-13	FR-07	Gửi cùng một lô dữ liệu hai lần	Lần thứ hai không tạo bản ghi mới, đếm là trùng và không báo lỗi	Kiểm thử tích hợp qua HTTP trên stack đang chạy	Đạt
TC-14	FR-07, NFR-03	Ngắt kết nối tới trung tâm trong lúc pipeline đang chạy	Hồ sơ tích lũy trong kho outbox, không mất bản nào	Kiểm thử tích hợp, dừng dịch vụ trung tâm	Đạt
TC-15	NFR-04	Khởi động lại	Hồ sơ chờ cũ	Kiểm thử tích	Đạt

		container node biên khi còn hồ sơ chờ gửi	được gửi tiếp nhờ kho gắn trên volume	hợp	
TC-16	FR-13	Đăng nhập sai mật khẩu, hoặc dùng JWT không hợp lệ	Trả 401 với thông báo chung, không tiết lộ tài khoản có tồn tại hay không	Kiểm thử tích hợp	Đạt
TC-17	FR-13	Mỗi vai trò gọi tới endpoint không thuộc quyền của mình	Trả 403 đúng chỗ, trả 200 khi đủ quyền	Kiểm thử tích hợp theo ma trận vai trò và endpoint	Đạt
TC-18	FR-13	Dùng lại một token làm mới đã bị thu hồi	Thu hồi toàn bộ phiên của người dùng đó	Kiểm thử bằng đọc mã và kiểm tra hành vi qua HTTP	Đạt
TC-19	FR-12	Câu hỏi dẫn tới câu lệnh SQL nguy hiểm	Bị chặn trước khi thực thi, trả thông báo lỗi	Kiểm thử tích hợp với API trợ lý	Đạt
TC-20	FR-03	Gọi API điều khiển của node với token sai hoặc vượt giới hạn tốc độ	Lần lượt trả 401 và 429 kèm tiêu đề giới hạn	Kiểm thử tích hợp gọi thẳng node	Đạt
TC-21	FR-03	Hiệu chuẩn vạch dừng từ web khi pipeline đang chạy	Vạch mới có hiệu lực ở khung hình kế tiếp, không cần khởi động lại	Kiểm thử tích hợp: gửi yêu cầu rồi đọc lại trạng thái hiệu chuẩn	Đạt
TC-22	NFR-01	Chạy pipeline liên tục trên video, theo đổi tốc độ xử lý	Tốc độ xử lý không thấp hơn tốc độ nguồn	Đọc bộ đếm từ endpoint sức khoẻ của node	Đạt
TC-23	Hồi quy	Chạy pipeline trên video aziz1.MP4 đã	Phát hiện đúng trường hợp vượt đèn	Chạy thật, ghi sự kiện ra tệp JSON	Đạt, xem mục 6.5

		biết trước kết quả	đỏ có trong video		
TC-24	Hồi quy	Chạy pipeline trên video đối chứng không có vi phạm	Không sinh hồ sơ giả nào	Chạy thật trên 384 khung hình	Đạt, xem mục 6.5

Bảng 6.1. Danh sách ca kiểm thử

6.2. Functional Testing

6.2.1. Kiểm thử đơn vị

Bộ kiểm thử đơn vị gồm 124 ca trong bảy tệp, tổng cộng 1.717 dòng mã kiểm thử, chạy bằng pytest trong môi trường ảo Python 3.11 của dự án. Toàn bộ đều là kiểm thử logic thuần: không nạp trọng số mô hình, không mở card đồ hoạ, không cần mạng, nên thời gian chạy chỉ hơn một giây và chạy được trong môi trường tích hợp liên tục với mỗi lần đẩy mã.

Lệnh chạy và kết quả thực tế ngày 12 tháng 9 năm 2026:

```
myenv/bin/python -m pytest tests/ -q → 124 passed in 1.09s
```

Tệp kiểm thử	Số ca	Phạm vi kiểm tra
tests/test_track_quality.py	39	Tăng chất lượng theo dõi bồi trên ByteTrack: điểm neo xét cắt vạch là điểm đáy giữa, quỹ đạo và vận tốc theo đơn vị pixel trên giây, khoảng trống trước khi tìm lại đối tượng, độ tin cậy làm mượt, bỏ phiếu nhân, ghi nhật ký vòng đời và cảnh báo đối mã, cơ chế lọc theo chuyển động
tests/test_vn_plate.py	34	Bộ kiểm tra cấu trúc biển số Việt Nam: ba dạng ký tự seri, vị trí dấu gạch sau mã tỉnh, sửa các nhầm lẫn thường gặp của nhận dạng ký tự
tests/test_violation_logic.py	18	Quy tắc xét vi phạm: cắt vạch khi đèn đỏ ổn định, hướng giám sát, vùng chết, trường hợp cắt sang rồi cắt về, mỗi đối tượng chỉ một hồ sơ
tests/test_geometry.py	15	Hình học của vạch dừng: xác định phía của điểm so với đoạn thẳng, xét cắt nhau của hai

		đoạn, ngưỡng vùng chết
tests/test_tracker_stabilizer.py	10	Máy trạng thái tín hiệu đèn có trễ chuyển và quy đổi ngưỡng theo tốc độ khung hình của nguồn video
tests/test_metrics.py	8	Bộ đếm trong tiến trình và cơ chế phát hiện pipeline ngừng xử lý
tests/conftest.py	—	Các fixture dùng chung: đặt vạch đang áp dụng, dựng vết giả

Bảng 6.2. Kết quả kiểm thử đơn vị theo tệp

Cách phân bố số ca phản ánh đúng chỗ rủi ro của hệ thống. Tệp có nhiều ca nhất là kiểm thử chất lượng theo dõi, vì đây là tầng được bổ sung sau cùng và ảnh hưởng trực tiếp tới việc một phương tiện có bị coi là cắt vạch hay không. Tệp nhiều ca thứ hai là bộ kiểm tra biến số, vì chuỗi ký tự đầu ra của nhận dạng ký tự rất đa dạng và cần được chặn lại trước khi ghi vào hồ sơ chính thức.

Phạm vi chưa được kiểm thử đơn vị cũng cần nói rõ: các mô-đun nạp mô hình học sâu, gọi thiết bị tính toán hoặc mở luồng video không có ca kiểm thử riêng, vì chúng phụ thuộc tệp trọng số và phần cứng cụ thể. Chất lượng của nhóm này được kiểm tra ở mức hồi quy trên video thật, trình bày ở mục 6.5.

6.2.2. Kiểm thử chức năng trên hệ thống đang chạy

Ngoài kiểm thử đơn vị, toàn bộ chức năng được kiểm tra trên hệ thống dựng bằng Docker Compose với năm dịch vụ đang chạy. Cách làm là gửi yêu cầu HTTP thật tới từng endpoint và đối chiếu mã trạng thái cùng nội dung phản hồi với thiết kế ở mục 4.5, sau đó mở bảng điều khiển web để kiểm tra dữ liệu hiển thị đúng.

Các kết quả chính thu được trên hệ thống đang chạy tại thời điểm hoàn thiện đồ án:

- Endpoint sức khỏe của trung tâm trả trạng thái ok kèm dấu thời gian theo múi giờ Việt Nam, đúng thiết kế không yêu cầu xác thực.
- Endpoint sức khỏe của node biên trả đủ bộ đếm: số khung hình đã xử lý, số vi phạm phát hiện, tốc độ xử lý thực tế và trạng thái các thành phần. Trong lần chạy quan sát được, node đạt khoảng 37 khung hình mỗi giây khi xử lý video nguồn.
- Đăng nhập bằng tài khoản quản trị trả về đầy đủ token truy cập, token làm mới, vai trò và thời hạn, đúng cấu trúc đã thiết kế.
- Truy vấn danh sách hồ sơ có phân trang trả đúng cấu trúc gồm nội dung, số trang, tổng số bản ghi và hai cờ đầu cuối; bộ lọc theo trạng thái hoạt động đúng.

- Endpoint thống kê trả về số liệu tổng hợp khớp với số đếm trực tiếp từ cơ sở dữ liệu, xác nhận các truy vấn gộp không bị lệch.
- Danh bạ node ghi nhận node biên đang trực tuyến kèm địa chỉ mạng nội bộ, cổng API và mốc heartbeat gần nhất.
- Luồng video trực tiếp phát được trong trình duyệt qua giao thức HLS, trang camera hiển thị đúng bốn bố cục lưới một, bốn, chín và mười sáu khung hình.
- Ảnh bằng chứng của hồ sơ được tải về qua endpoint trung gian và hiển thị đúng trong trang chi tiết, xác nhận cả chuỗi lưu trữ đối tượng và uỷ quyền đều hoạt động.
- Thao tác duyệt và từ chối trên trang chi tiết đối trạng thái hồ sơ và làm mới số đếm trên huy hiệu chờ duyệt.

Một điểm cần ghi nhận trung thực: các ca kiểm thử tích hợp này được thực hiện thủ công bằng tập lệnh và qua giao diện, chưa được tự động hoá thành bộ kiểm thử chạy lặp lại. Hệ quả là mỗi lần thay đổi mã ở tầng trung tâm, người phát triển phải tự chạy lại các bước này. Đây là hạn chế được nêu ở Chương 7 và là hạng mục ưu tiên trong hướng phát triển.

6.2.3. Kiểm thử phân quyền

Phân quyền là phần dễ sai nhất của hệ thống có nhiều vai trò, nên được kiểm tra theo ma trận: mỗi vai trò gọi tới từng nhóm endpoint nhạy cảm và ghi lại mã trạng thái. Kết quả khớp với Bảng 3.1 ở Chương 3: vai trò vận hành bị chặn khi cố duyệt hồ sơ, vai trò nghiệp vụ bị chặn khi cố hiệu chuẩn node, còn vai trò quản trị thực hiện được cả hai. Một chi tiết kỹ thuật phát hiện được trong quá trình này là Spring Boot chuyển tiếp lỗi tới đường dẫn nội bộ /error; nếu đường dẫn này không được mở trong chuỗi lọc bảo mật thì mã trạng thái thật bị che thành 403, khiến việc phân biệt lỗi xác thực và lỗi phân quyền trở nên không thể.

6.3. Kiểm thử hiệu năng

Hiệu năng của hệ thống được đo ở hai chỗ: tốc độ suy luận của các mô hình học sâu trên card đồ hoạ, và thời gian xử lý của khâu nhận dạng biển số. Các phép đo đều chạy trên máy phát triển của đồ án, cấu hình ghi ở Bảng 5.6.

Mô hình	Độ chính xác	Độ hồi quy	mAP50	mAP50-95	Tốc độ (FP16)
Phát hiện phương tiện đã huấn luyện riêng (yolo26m_veh	0,9022	0,8939	0,9436	0,6796	73,4 khung hình/giây

icle)					
Mô hình gốc tiền huấn luyện trên bộ COCO (đối chứng)	0,0303	0,0777	0,0101	0,0023	69,2 khung hình/giây

Bảng 6.3. Kết quả đánh giá mô hình phát hiện phương tiện

Bảng 6.3 là kết quả quan trọng nhất về mặt kỹ thuật của đồ án. Hai mô hình có cùng kiến trúc và cùng tốc độ suy luận, chênh lệch chỉ nằm ở dữ liệu huấn luyện, nhưng chỉ số mAP50 chênh nhau gần chín mươi bốn lần. Nguyên nhân nằm ở cột số lượng phát hiện theo lớp của tập kiểm chứng: mô hình gốc gần như không phát hiện được lớp xe máy nào, trong khi đây lại là lớp phương tiện chiếm đa số trong dữ liệu thực tế của đồ án. Kết luận rút ra là việc huấn luyện lại trên dữ liệu đúng miền không phải là bước tối ưu tùy chọn mà là điều kiện bắt buộc để hệ thống hoạt động được tại Việt Nam.

Cần lưu ý về phạm vi của con số này: mAP50 bằng 0,9436 được đo trên tập kiểm chứng của chính bộ dữ liệu dùng để huấn luyện, nên nó phản ánh chất lượng khớp mô hình chứ không phải độ chính xác trên mọi nút giao bất kỳ. Khi chạy trên video thực tế có khác biệt miền, chất lượng thấp hơn con số này, và đó là lý do đồ án phải bổ sung tầng làm ổn định tín hiệu cùng cơ chế duyệt hồ sơ có con người tham gia.

Hai mô hình còn lại được đánh giá theo cách khác nhau vì bản chất dữ liệu khác nhau. Mô hình phân loại đèn giao thông là bài toán phân lớp ba nhãn nên được đo bằng tỉ lệ dự đoán đúng hạng nhất, trước hết trên tập kiểm chứng tách rời rồi mới đo lại trên video thực tế có khác biệt miền, vì đây chính là chỗ bộ phân lớp đơn lẻ thường sai. Mô hình phát hiện biển số là bài toán phát hiện một lớp nên được đo bằng mAP50 trên tập kiểm chứng của nó. Kết quả tổng hợp ở Bảng 6.4.

Mô hình	Chỉ tiêu	Trên tập kiểm chứng	Trên video thực tế	Thời gian suy luận
Phân loại màu đèn (yolo26n-cls)	Tỉ lệ đúng hạng nhất	100% (bộ dữ liệu LISA)	99,0% sau khi hợp nhất bằng chứng	khoảng 0,3 mili giây mỗi ảnh
Phát hiện biển số	mAP50	0,993	không đo	xem Bảng 6.5

Bảng 6.4. Kết quả đánh giá mô hình phân loại đèn và mô hình phát hiện biển số

Về nguồn gốc số liệu của Bảng 6.4 cần nói rõ hai điểm để người đọc đánh giá đúng mức độ tin cậy. Thứ nhất, hai chỉ tiêu này được ghi nhận trong các phiên đo trước của đồ án và lưu tại docs/test-report.md; kho mã nguồn hiện không kèm tập huấn luyện, tập kiểm chứng và kịch bản đo của hai mô hình này, nên chúng không thể tái lập bằng

một lệnh như Bảng 6.3. Thứ hai, con số 99,0% của khâu phân loại đèn là kết quả sau khi hợp nhất bằng chứng chứ không phải của riêng mạng nơ-ron; bản thân bộ phân lớp đơn lẻ cho kết quả thấp hơn trên video thực tế, và đây chính là lý do kiến trúc ở mục 4.1 có thêm tầng hợp nhất ba nguồn bằng chứng. Bổ sung kịch bản đo tái lập được cho hai mô hình này là hạng mục đầu tiên trong hướng phát triển ở Chương 7.

Hạng mục	Giá trị đo	Điều kiện đo
Thời gian nhận dạng một ảnh biển số (trung vị)	5,2 mili giây	40 ảnh cắt từ video, chạy trên CUDA
Thời gian nhận dạng một ảnh biển số (trung bình)	10,0 mili giây	Trung bình cao hơn trung vị do vài ảnh đầu chịu chi phí khởi động
Thời gian phát hiện biển số trên một khung hình (trung vị)	30,2 mili giây	60 khung hình lấy mẫu cách nhau sáu khung hình
Thời gian phát hiện biển số trên một khung hình (trung bình)	76,4 mili giây	Bị kéo lên bởi khung hình đầu tiên nạp mô hình
Số biển phát hiện trung bình mỗi khung hình	1,95	117 hộp giới hạn trên 60 khung hình
Tốc độ xử lý của pipeline đầy đủ trên node	khoảng 37 khung hình/giây	Đọc từ bộ đếm của node đang chạy trong Docker, video nguồn tốc độ thấp hơn
Thời gian nạp mô hình nhận dạng biển số	dưới 0,1 giây	Đo từ lúc khởi tạo tới lúc sẵn sàng suy luận

Bảng 6.5. Kết quả đo thời gian xử lý khâu biển số

Một kết quả đo đáng chú ý liên quan tới thiết bị tính toán. Thư viện nhận dạng ký tự dùng trong đồ án cần CUDA phiên bản mười ba ở bản dựng có hỗ trợ card đồ họa, trong khi máy phát triển cài CUDA phiên bản 12.4, nên khi khởi động thư viện báo không nạp được thư viện CUDA và tự lùi về chạy trên bộ xử lý trung tâm. Điều này giải thích vì sao trong nhật ký chạy pipeline có cảnh báo về nhà cung cấp thực thi CUDA. Ở lần đo ngày 12 tháng 9 năm 2026, khâu nhận dạng chạy được trên CUDA với thời gian trung vị 5,2 mili giây mỗi ảnh, thấp hơn nhiều so với con số khoảng hai mươi mili giây ghi nhận trước đó khi chạy trên bộ xử lý trung tâm. Kết luận thực tế là khâu nhận dạng biển số không phải nút thắt của pipeline; nút thắt nằm ở khâu phát hiện phương tiện.

6.4. Kiểm thử khâu nhận dạng biển số

Khâu nhận dạng biển số cần được kiểm tra riêng vì nó là chỗ duy nhất trong pipeline có bộ lọc nghiệp vụ nằm sau mô hình học sâu. Phép đo thực hiện trên sáu mươi khung

hình lấy mẫu từ một video kiểm thử về nhận dạng biển số, cắt ra bốn mươi ảnh biển và cho chạy qua cả hai tầng: mô hình nhận dạng ký tự và bộ kiểm tra cấu trúc biển số Việt Nam.

Chi tiêu	Kết quả	Diễn giải
Số ảnh cắt đưa vào nhận dạng	40	Lấy từ 117 hộp giới hạn do mô hình phát hiện biển trả về
Số ảnh mô hình đọc ra được chuỗi ký tự	40 trên 40	Mô hình nhận dạng ký tự hoạt động tốt, không trả chuỗi rỗng
Độ tin cậy trung bình của chuỗi đọc được	khoảng 0,9	Dao động từ 0,73 đến 0,98 trên các mẫu quan sát được
Số chuỗi qua được bộ kiểm tra cấu trúc biển Việt Nam	0 trên 40	Toàn bộ bị loại có chủ đích

Bảng 6.6. Kết quả đo khâu nhận dạng biển số

Kết quả bảng không ở dòng cuối không phải là lỗi mà là bằng chứng bộ lọc hoạt động đúng. Các chuỗi mô hình đọc được trong video kiểm thử này là biển số Vương quốc Anh, ví dụ NA13NRU hay MW51VSU, tức hai chữ cái rồi hai chữ số rồi ba chữ cái. Cấu trúc này không khớp với quy định về biển số xe cơ giới Việt Nam, vốn bắt đầu bằng mã tỉnh hai chữ số từ mười một đến chín mươi chín, tiếp theo là ký tự seri rồi bốn đến năm chữ số. Bộ kiểm tra vì thế loại toàn bộ, đúng như thiết kế ở mục 5.2.1 nhằm tránh ghi biển số sai định dạng vào hồ sơ xử phạt.

Phép đo này đồng thời chỉ ra một hạn chế cần nói thẳng: đồ án chưa có video giao thông Việt Nam nào có biển số đủ rõ và đủ nhiều để đo tỉ lệ đọc đúng của khâu nhận dạng trên dữ liệu đúng miền. Con số có thể khẳng định được hiện nay chỉ gồm hai phần: mô hình phát hiện biển số đạt mAP50 bằng 0,993 trên tập kiểm chứng của nó, và mô hình nhận dạng ký tự đọc được chuỗi trên toàn bộ mẫu thử với độ tin cậy cao. Tỉ lệ đọc đúng biển số Việt Nam trên hiện trường là chỉ tiêu chưa đo được, và đồ án không suy diễn chỉ tiêu này từ hai con số trên.

6.5. Kiểm thử hồi quy trên video có nhãn

Đây là mức kiểm thử sát với thực tế vận hành nhất: chạy toàn bộ pipeline từ đầu tới cuối trên tập video, không bỏ qua khâu nào, rồi đối chiếu số hồ sơ sinh ra với kết quả đã biết trước của video đó. Hai video được chọn có tính chất bổ sung cho nhau: một video có chứa trường hợp vượt đèn đỏ để kiểm tra khả năng phát hiện đúng, và một video không có vi phạm để kiểm tra khả năng không sinh hồ sơ giả.

Video	Số khung hình đã xử lý	Số hồ sơ sinh ra	Kỳ vọng	Nhận xét
-------	------------------------	------------------	---------	----------

aziz1.MP4 (nút giao thông thực tế, có hiệu chuẩn vạch dừng và vùng đèn sẵn)	120	1	Có vi phạm	Phát hiện đúng trường hợp vượt đèn đỏ: khung hình 32, mã theo dõi số 2, trạng thái đèn đỏ với độ tin cậy 0,95, điểm cắt vạch tại tọa độ 1779 và 1413 trên ảnh độ phân giải cao
20221003-102556.mp4 (video đối chứng, không có vi phạm)	384	0	Không có vi phạm	Không sinh hồ sơ giả nào trên toàn bộ độ dài video

Bảng 6.7. Kết quả kiểm thử hồi quy trên video có nhãn

Chi tiết hồ sơ do video thứ nhất sinh ra được ghi lại nguyên văn trong tệp kết quả, gồm mã sự kiện, mã theo dõi, chỉ số khung hình, mốc thời gian, trạng thái đèn kèm độ tin cậy, điểm cắt vạch và hộp giới hạn của phương tiện. Bộ trường này khớp đúng với thiết kế bảng violations ở mục 4.4, xác nhận dữ liệu đi hết chuỗi mà không rơi mất trường nào.

Một điểm cần giải thích để tránh hiểu nhầm về tính nhất quán. Ở các phiên bản trước của đồ án, cùng video này cho ra hai hồ sơ. Khác biệt xuất phát từ một thay đổi thiết kế có chủ đích thực hiện ngày 12 tháng 9 năm 2026: điểm neo dùng để xét cắt vạch được chuyển từ tâm hộp giới hạn sang điểm đáy giữa, tức điểm phương tiện tiếp xúc mặt đường. Với điểm neo mới, kết luận đưa ra muộn hơn một đến hai khung hình nhưng ổn định hơn khi phương tiện nghiêng hoặc hộp giới hạn co giãn. Trường hợp bị bỏ qua là phương tiện mới nhô đầu qua vạch rồi lùi lại, vốn không phải hành vi vượt đèn đỏ hoàn chỉnh. Sự thay đổi số lượng hồ sơ vì vậy phản ánh đúng lựa chọn thiết kế đã được cân nhắc, không phải lỗi hồi quy, và bộ kiểm thử đơn vị đã được cập nhật theo quyết định này.

6.6. Kiểm thử cơ chế giao hàng không mất dữ liệu

Yêu cầu NFR-03 và NFR-04 là hai yêu cầu khó kiểm chứng nhất, vì chúng nói về hành vi của hệ thống khi có sự cố chứ không phải khi hoạt động bình thường. Ba kịch bản đã được thực hiện trên hệ thống Docker đang chạy:

- Ngắt trung tâm khi node đang chạy. Dừng dịch vụ máy chủ trung tâm trong lúc pipeline vẫn xử lý video. Quan sát thấy vòng lặp xử lý không bị ảnh hưởng, các hồ sơ phát hiện được vẫn ghi vào kho outbox ở trạng thái chờ, bộ đếm hồ sơ chờ tăng đúng bằng số vi phạm mới. Khi khởi động lại dịch vụ trung tâm, luồng gửi nền tự phát hiện kết nối đã thông và đẩy hết số hồ sơ tồn đọng.
- Khởi động lại container node khi còn hồ sơ chờ. Vì kho outbox được gắn trên volume riêng, các hồ sơ chờ từ lần chạy trước vẫn còn và được gửi tiếp ở lần khởi động sau. Node ghi rõ trong nhật ký số hồ sơ chờ kế thừa từ lần chạy trước.
- Gửi lại cùng một lô. Cố ý gửi hai lần một lô hồ sơ có cùng mã sự kiện. Phản hồi lần thứ hai cho thấy số bản ghi được chấp nhận bằng không và số bản ghi trùng bằng kích thước lô, đồng thời vẫn trả mã thành công. Kiểm tra trong cơ sở dữ liệu không thấy bản ghi nào bị nhân đôi, xác nhận rằng buộc duy nhất trên cột mã sự kiện hoạt động đúng ở cả hai phía.

Kết quả của ba kịch bản này cho phép khẳng định tính chất quan trọng nhất của thiết kế ở mục 4.4: khi có sự cố mạng, hệ thống chuyển từ trạng thái giao hàng ngay sang trạng thái giao hàng trễ, chứ không chuyển sang trạng thái mất dữ liệu. Đây là khác biệt về chất chứ không phải về lượng so với cách gọi trực tiếp API khi phát hiện vi phạm.

Giới hạn của phép kiểm chứng này cũng cần nêu rõ: các kịch bản trên được thực hiện thủ công và quan sát bằng nhật ký cùng bộ đếm, chưa có ca kiểm thử tự động nào mô phỏng việc mất kết nối. Ngoài ra thời gian ngắt kết nối trong thử nghiệm chỉ ở mức vài phút, chưa đủ dài để kiểm chứng hành vi khi kho outbox phình ra rất lớn hoặc khi đĩa của node đầy.

6.7. Kiểm thử bảo mật

Kiểm thử bảo mật tập trung vào bốn nhóm nguy cơ chính của một hệ thống nhận dữ liệu từ nhiều nguồn và có lưu thông tin cá nhân là biến số phương tiện.

- Thông tin xác thực bị lộ trong mã nguồn. Quét toàn bộ kho mã theo các mẫu khoá bí mật trên tất cả tệp Python, Java, TypeScript, YAML, tệp lệnh shell và JSON: không còn giá trị bí mật nào bị viết cứng. Tệp biến môi trường thật đã được đưa vào danh sách loại trừ của Git, kho chỉ giữ tệp mẫu. Đây là kết quả của một đợt rà soát thực hiện trước đó, khi phát hiện mật khẩu cơ sở dữ liệu và khoá MinIO bị viết thẳng trong tệp cấu hình Docker Compose.
- Xác thực và phân quyền. Kiểm tra các trường hợp token sai, token hết hạn, token rác và thiếu token trên mọi endpoint được bảo vệ: tất cả trả 401. Kiểm tra ma trận vai trò như mục 6.2.3: trả 403 đúng chỗ. Thông báo lỗi đăng nhập dùng chung một nội dung cho cả trường hợp sai tên tài khoản và sai mật khẩu, nên không thể dò ra tài khoản nào tồn tại.

- API điều khiển của node biên. Gọi với token sai trả 401. Gọi vượt ba mươi yêu cầu mỗi phút từ một địa chỉ trả 429 kèm các tiêu đề cho biết giới hạn và thời điểm được gọi lại. Yêu cầu từ nguồn không nằm trong danh sách cho phép bị chặn ở tầng chia sẻ tài nguyên khác nguồn. Mọi phản hồi đều kèm tiêu đề không cho phép nhúng khung và không lưu bộ nhớ đệm.
- Câu lệnh SQL do trợ lý sinh ra. Bộ lọc chỉ chấp nhận câu lệnh truy vấn và chặn các từ khoá xoá, sửa, tạo, thu hồi quyền, đồng thời tự giới hạn số dòng trả về. Thử đặt câu hỏi dẫn tới câu lệnh nguy hiểm thì hệ thống trả lỗi và không thực thi.

Hai vấn đề bảo mật còn tồn tại được ghi nhận thẳng thay vì bỏ qua. Thứ nhất, endpoint đăng nhập của máy chủ trung tâm chưa có giới hạn tốc độ, trong khi API điều khiển của node biên thì có; nghĩa là về lý thuyết vẫn có thể dò mật khẩu bằng cách thử nhiều lần. Thứ hai, token làm mới được lưu trong bộ nhớ cục bộ của trình duyệt thay vì trong cookie chỉ đọc được phía máy chủ, đánh đổi giữa rủi ro tấn công chéo trang và khả năng làm mới phiên liền mạch; quyết định này được ghi rõ trong bình luận của mô-đun quản lý phiên. Cả hai đều được đưa vào phần hạn chế ở Chương 7.

6.8. Kết quả

Bảng 6.7 tổng hợp kết quả của toàn bộ hoạt động kiểm thử. Nguyên tắc điền bảng là chỉ ghi Đạt khi có bằng chứng thực thi kèm theo; hạng mục nào chưa thực hiện thì ghi rõ là chưa thực hiện.

Nhóm kiểm thử	Số ca hoặc phạm vi	Kết quả	Bằng chứng
Kiểm thử đơn vị logic	124 ca trong 7 tệp	124 trên 124 đạt, 1,09 giây	Nhật ký pytest ngày 12/9/2026
Tích hợp liên tục	3 việc làm trên GitHub Actions	Cả ba chạy xanh ở lần đẩy gần nhất	Kiểm thử và lint cho Python, biên dịch cho Java, kiểm tra kiểu và dựng cho TypeScript
Biên dịch tầng trung tâm	45 tệp Java	Thành công, không lỗi	Biên dịch trong image Maven chính thức vì máy phát triển chỉ có môi trường chạy Java
Kiểm tra kiểu và dựng tầng web	35 tệp TypeScript	Thành công, không lỗi	Chạy trong việc làm của tích hợp liên tục
Kiểm thử chức năng trên hệ thống thật	9 luồng chính	Đạt toàn bộ	Thực hiện thủ công qua HTTP và giao diện, mục 6.2.2
Kiểm thử phân quyền	Ma trận 3 vai trò với các nhóm endpoint	Đạt, trả đúng 401 và 403	Mục 6.2.3
Kiểm thử hồi quy	2 video, tổng 504	Đúng kỳ vọng: 1 hồ sơ	Mục 6.5, Bảng 6.7, tệp

video	khung hình	và 0 hồ sơ	kết quả JSON trong thư mục tạm
Kiểm thử giao hàng bền vững	3 kịch bản sự cố	Đạt, không mất và không trùng hồ sơ	Mục 6.6
Kiểm thử bảo mật	4 nhóm nguy cơ	Đạt, còn 2 vấn đề đã ghi nhận	Mục 6.7
Đánh giá mô hình học sâu	3 mô hình	Đã đo; 1 mô hình tái lập được bằng lệnh, 2 mô hình lấy từ ghi nhận trước	Bảng 6.3 và Bảng 6.4; tập kết quả chuẩn hoá trong thư mục scripts
Đo thời gian khâu biến số	60 khung hình, 40 ảnh cắt	Đã đo	Bảng 6.5, tập lệnh đo chạy ngày 12/9/2026
Đo tỉ lệ đọc đúng biển số Việt Nam	—	Chưa thực hiện vì thiếu video đúng miền	Ghi nhận ở mục 6.4
Kiểm thử tải của máy chủ trung tâm	—	Chưa thực hiện	Cần công cụ tạo tải như k6; ghi nhận ở Chương 7
Kiểm thử thành phần giao diện	—	Chưa thực hiện	Chưa có bộ kiểm thử thành phần cho tầng web

Bảng 6.8. Tổng hợp kết quả kiểm thử

Đánh giá chung về chất lượng. Phần logic nghiệp vụ của đồ án, tức những chỗ dễ sai nhất và khó phát hiện nhất khi chạy thật, đã được bao phủ bằng kiểm thử tự động chạy trong vài giây và chạy lại với mỗi lần sửa mã. Đây là nền tảng cho phép khẳng định hệ thống xét vi phạm đúng theo các quy tắc đã phân tích. Ba khoản trống lớn còn lại đều nằm ở phần cần hạ tầng hoặc cần dữ liệu ngoài tầm của một đồ án: kiểm thử tải cần môi trường có thể tạo hàng nghìn yêu cầu đồng thời, kiểm thử thành phần giao diện cần bộ công cụ dựng riêng, còn đo tỉ lệ đọc biển số cần bộ video giao thông Việt Nam có gán nhãn độc lập. Việc xác định rõ ba khoản trống này, thay vì ước lượng chúng, là cách đồ án giữ cho các kết luận ở Chương 7 nằm trong phạm vi bằng chứng có thật.

CHƯƠNG 7. KẾT LUẬN

7.1. Kết quả đạt được

Đối chiếu lại với bảy mục tiêu cụ thể đặt ra ở mục 1.2, đồ án đã hoàn thành cả bảy, trong đó sáu mục tiêu hoàn thành ở mức có số liệu kiểm chứng và một mục tiêu hoàn thành ở mức chức năng hoạt động được nhưng chưa đo được độ chính xác. Bảng 7.1 trình bày phép đối chiếu này.

Mục tiêu đặt ra	Kết quả thực hiện	Mức độ
Xây dựng pipeline thị giác thời gian thực tại node biên	Hoàn thành. Pipeline chạy đủ các khâu phát hiện, theo dõi, phân loại đèn, xét vi phạm và nhận dạng biển số; tốc độ xử lý đo được khoảng 37 khung hình mỗi giây trên node chạy trong Docker, riêng khâu phát hiện đạt 73,4 khung hình mỗi giây ở chế độ nửa chính xác	Đạt, có đo
Huấn luyện và đánh giá ba mô hình chuyên biệt	Hoàn thành. Mô hình phát hiện phương tiện đạt mAP50 bằng 0,9436 so với 0,0101 của mô hình gốc trên cùng tập kiểm chứng; mô hình phát hiện biển số đạt 0,993; mô hình phân loại đèn đạt tỉ lệ đúng cao nhất trên tập kiểm chứng và được bổ sung cơ chế hợp nhất bằng chứng	Đạt, có đo
Nhận dạng biển số Việt Nam có kiểm tra cấu trúc	Hoàn thành phần mềm: mô hình đọc được chuỗi ký tự trên toàn bộ mẫu thử với độ tin cậy trung bình khoảng 0,9, bộ kiểm tra cấu trúc loại đúng các biển không theo quy định Việt Nam. Chưa đo được tỉ lệ đọc đúng trên biển số Việt Nam vì thiếu dữ liệu đúng miền	Đạt chức năng, thiếu số đo
Cơ chế giao hàng bền vững, không mất và không trùng hồ sơ	Hoàn thành và kiểm chứng bằng ba kịch bản sự cố. Kho outbox ghi trước khi gọi mạng, lô gửi có tính lũy đẳng nhờ ràng buộc duy nhất trên mã sự kiện ở cả hai phía	Đạt, có kiểm chứng
Máy chủ trung tâm và bảng	Hoàn thành. Trung tâm 3.525	Đạt

điều khiển web hoàn chỉnh	dòng Java với 31 endpoint, web 6.547 dòng TypeScript với chín tuyến đường và mười ba thành phần; đủ các nghiệp vụ thống kê, tra cứu, duyệt, xem camera trực tiếp và hiệu chuẩn từ xa	
Trợ lý hỏi dữ liệu bằng tiếng Việt	Hoàn thành. Sinh câu lệnh truy vấn từ câu hỏi tự nhiên, có bộ lọc chỉ cho phép truy vấn và giới hạn số dòng, hiển thị cả câu lệnh đã sinh để người dùng kiểm tra. Phụ thuộc dịch vụ bên ngoài nên chưa có phương án dự phòng khi mất kết nối	Đạt, có ràng buộc
Kiểm thử và vận hành bằng một lệnh	Hoàn thành phần kiểm thử tự động với 124 ca đạt và ba việc làm tích hợp liên tục chạy xanh; toàn hệ thống dựng bằng một lệnh với năm dịch vụ có kiểm tra sức khỏe. Phần kiểm thử tích hợp vẫn thực hiện thủ công	Đạt phần lớn

Bảng 7.1. Đối chiếu mục tiêu và kết quả đạt được

Ngoài bảy mục tiêu, đồ án đạt được ba kết quả có ý nghĩa về mặt phương pháp, đều xuất phát từ sự cố gặp phải trong quá trình làm chứ không nằm trong kế hoạch ban đầu.

- Xác định được bằng số liệu rằng việc huấn luyện lại mô hình trên dữ liệu đúng miền là điều kiện bắt buộc chứ không phải bước cải thiện tùy chọn. Chênh lệch gần chín mươi bốn lần về mAP50 giữa mô hình huấn luyện riêng và mô hình gốc, trong khi tốc độ suy luận gần như bằng nhau, là bằng chứng định lượng rõ ràng nhất của đồ án và cũng là kết quả có thể dùng lại cho các bài toán thị giác khác tại Việt Nam.
- Thiết kế được cơ chế tách rời phát hiện khỏi giao hàng, biến một hệ thống có nguy cơ mất dữ liệu thành hệ thống chỉ có thể giao hàng trễ. Mẫu thiết kế này không đặc thù cho bài toán giao thông và có thể áp dụng cho bất kỳ hệ thống phân tán nào có node biên hoạt động trong điều kiện mạng không ổn định.
- Xây dựng được tầng chất lượng theo dõi năm giữa bộ theo dõi và logic nghiệp vụ, gồm quỹ đạo, vận tốc tính theo đơn vị thời gian thực thay vì theo số khung hình, độ tin cậy làm mượt và bỏ phiếu nhãn. Tầng này giải quyết đúng vấn đề của camera tốc độ khung hình thấp, nơi một phương tiện dịch chuyển hàng chục điểm ảnh giữa hai khung hình liên tiếp.

Về quy mô, sản phẩm của đồ án là một hệ thống phân tán ba tầng với 18.386 dòng mã trên ba ngôn ngữ lập trình khác nhau, bốn mô hình học sâu, hai hệ thống lưu trữ, ba mươi một endpoint ở trung tâm và mười bốn endpoint ở node biên, cùng bộ tài liệu gồm tài liệu API, hướng dẫn triển khai, báo cáo kiểm thử và hai mươi sáu sơ đồ. Toàn bộ chạy được trên một máy trạm có một card đồ họa phổ thông.

7.2. Hạn chế

Đồ án còn những hạn chế sau, được xếp theo mức độ ảnh hưởng tới khả năng đưa hệ thống vào vận hành thực tế.

Nhóm hạn chế	Mô tả cụ thể	Ảnh hưởng
Phạm vi kiểm chứng thực địa	Toàn bộ thử nghiệm chạy trên tệp video, chủ yếu phát lặp lại, với một node biên duy nhất. Chưa vận hành đồng thời nhiều node, chưa kết nối camera thật qua giao thức phát trực tiếp trong thời gian dài.	Cao. Chưa chứng minh được khả năng mở rộng theo số nút giao và độ ổn định khi chạy nhiều ngày liên tục
Chất lượng nhận dạng biển số trên dữ liệu đúng miền	Chưa có video giao thông Việt Nam nào đủ rõ và đủ nhiều để đo tỉ lệ đọc đúng. Số liệu hiện có chỉ chứng minh mô hình đọc được chuỗi ký tự và bộ lọc cấu trúc hoạt động đúng.	Cao. Biển số là trường thông tin quan trọng nhất của hồ sơ xử phạt, thiếu số đo này thì chưa thể ước lượng tỉ lệ hồ sơ phải bổ sung thủ công
Điều kiện ánh sáng khó	Chưa kiểm chứng quy mô lớn với mưa đêm, ngược sáng gắt, đèn pha chiếu thẳng vào ống kính. Dữ liệu huấn luyện và video thử nghiệm đều là điều kiện ban ngày tương đối thuận lợi.	Cao. Nút giao thực tế hoạt động cả ngày đêm và trong mọi thời tiết
Kiểm thử tự động còn thiếu ở hai tầng	Tầng trung tâm chưa có bộ kiểm thử tích hợp tự động, tầng web chưa có kiểm thử thành phần. Mọi kiểm tra tích hợp đều làm thủ công.	Trung bình. Mỗi lần sửa mã phải tự chạy lại các bước kiểm tra, dễ bỏ sót hồi quy
Chưa đo khả năng chịu tải	Không có số liệu về số yêu cầu đồng thời mà trung tâm xử lý được, cũng như độ trễ phản hồi theo số node gửi dữ liệu. Các phân tích về khả năng mở rộng hiện chỉ dựa trên lập luận kiến trúc.	Trung bình. Không xác định được điểm cần nâng cấp khi số node tăng

Bảo mật còn hai khoảng trống	Endpoint đăng nhập của trung tâm chưa giới hạn tốc độ. Token làm mới lưu trong bộ nhớ cục bộ của trình duyệt thay vì cookie chỉ đọc phía máy chủ.	Trung bình. Cần xử lý trước khi mở hệ thống ra mạng công cộng
Hiệu chuẩn phụ thuộc thao tác thủ công	Vùng đèn và vạch dừng cố định sau khi hiệu chuẩn. Khi camera bị lệch do gió hoặc va chạm, hệ thống không tự phát hiện mà vẫn tiếp tục xét vi phạm theo toạ độ cũ.	Trung bình. Có thể sinh hồ sơ sai hàng loạt mà không có cảnh báo
Phụ thuộc dịch vụ bên ngoài	Trợ lý hỏi dữ liệu cần khoá API của nhà cung cấp mô hình ngôn ngữ và cần kết nối mạng. Không có phương án dự phòng chạy cục bộ.	Thấp. Chức năng này là tiện ích bổ sung, không nằm trên luồng nghiệp vụ chính
Quản lý người dùng chưa hoàn thiện	Chưa có giao diện thêm sửa xoá tài khoản; tài khoản quản trị được tạo tự động từ biến môi trường. Token truy cập chưa có danh sách thu hồi, chỉ tự hết hạn sau mười hai giờ.	Thấp. Đủ dùng cho quy mô vận hành nội bộ của đồ án
Quản lý lược đồ cơ sở dữ liệu	Lược đồ do khung ứng dụng tự cập nhật thay vì dùng công cụ quản lý phiên bản di trú. Chấp nhận được ở quy mô đồ án nhưng không phù hợp khi dữ liệu sản xuất cần thay đổi lược đồ có kiểm soát.	Thấp ở hiện tại, sẽ thành trung bình khi có dữ liệu thật cần bảo toàn

Bảng 7.2. Hạn chế của đồ án theo mức độ ảnh hưởng

Trong các hạn chế trên, hai hạn chế đầu tiên là căn bản nhất và có liên hệ nhân quả với nhau: vì chưa có dữ liệu thực địa đúng miền nên vừa không kiểm chứng được hệ thống trong điều kiện vận hành thật, vừa không đo được chất lượng nhận dạng biển số. Đây là giới hạn về nguồn lực thu thập dữ liệu chứ không phải giới hạn về thiết kế, và là việc cần làm đầu tiên nếu muốn phát triển tiếp.

7.3. Hướng phát triển

Từ các hạn chế đã nêu, hướng phát triển được sắp xếp theo thứ tự ưu tiên: nhóm thứ nhất giải quyết khoảng trống về bằng chứng, nhóm thứ hai mở rộng năng lực nghiệp vụ, nhóm thứ ba hoàn thiện vận hành.

Nhóm một, thu hẹp khoảng trống về dữ liệu và kiểm chứng:

- Xây dựng bộ dữ liệu video giao thông Việt Nam có gán nhãn độc lập, tối thiểu vài nghìn khung hình trải trên các điều kiện sáng khác nhau, để đo được tỉ lệ phát hiện đúng, tỉ lệ hồ sơ giả và tỉ lệ đọc đúng biến số. Đây là tiền đề cho mọi cải tiến tiếp theo vì không có số đo thì không biết thay đổi nào là tốt hơn.
- Kết nối camera thật qua giao thức phát trực tiếp và cho chạy liên tục nhiều ngày, ghi nhận tự động các trường hợp mất kết nối, tràn bộ nhớ, lệch khung hình. Chế độ bỏ khung hình để bám thời gian thực đã có sẵn trong cài đặt, chỉ cần bật và quan sát.
- Tự phát hiện khi camera bị lệch: so sánh vị trí vùng đèn và vạch dừng hiện tại với ảnh nền tham chiếu, cảnh báo khi độ lệch vượt ngưỡng để người vận hành biết cần hiệu chuẩn lại.

Nhóm hai, mở rộng năng lực nghiệp vụ:

- Mở rộng sang các loại vi phạm khác trên cùng pipeline hiện có. Nền tảng theo dõi, vạch ảo và mô hình chuyển động đã đủ để xét thêm hành vi đi ngược chiều, lấn làn và dừng đỗ sai quy định; riêng hành vi chạy quá tốc độ cần thêm bước quy đổi từ điểm ảnh sang mét bằng phép chiếu mặt đường.
- Nâng chất lượng nhận dạng biến số bằng cách huấn luyện lại trên dữ liệu biến số Việt Nam và chuyển sang chạy trên hạ tầng suy luận tối ưu hơn, đồng thời giải quyết xung đột phiên bản thư viện tính toán để khâu này chạy ổn định trên card đồ hoạ.
- Xuất biên bản xử phạt từ hồ sơ đã duyệt theo mẫu quy định, kèm ảnh bằng chứng và các trường truy vết; cung cấp giao diện lập trình để đối soát với hệ thống quản lý phương tiện.
- Giải thích được quyết định của hệ thống: lưu thêm ảnh cắt biến số, chuỗi ký tự thô trước khi sửa và lý do bộ lọc chấp nhận hay loại, để người duyệt có đủ căn cứ khi kết luận hồ sơ.

Nhóm ba, hoàn thiện chất lượng và vận hành:

- Bổ sung bộ kiểm thử tích hợp tự động cho tầng trung tâm bằng khung kiểm thử Java với cơ sở dữ liệu khởi tạo tạm thời, và bộ kiểm thử thành phần cho tầng web; đưa kiểm thử tải vào quy trình bằng công cụ tạo tải chuyên dụng để có số liệu về băng tải và độ trễ.
- Xử lý hai khoảng trống bảo mật: thêm giới hạn tốc độ cho endpoint đăng nhập và cân nhắc khoá tài khoản tạm thời sau nhiều lần sai; chuyển token làm mới sang cookie chỉ đọc phía máy chủ nếu chấp nhận được thay đổi trong cơ chế làm mới phiên.
- Chuyển sang quản lý lược đồ cơ sở dữ liệu bằng công cụ di trú có phiên bản, thay cho cơ chế tự cập nhật hiện nay.

- Thêm giám sát tập trung: thu thập chỉ số từ cả ba tầng, dựng bảng theo dõi và cấu hình cảnh báo khi node ngừng gửi dữ liệu hoặc tỉ lệ hồ sơ bị từ chối tăng bất thường.
- Kiểm chứng quy trình sao lưu bằng cách phục hồi thử định kỳ, và đặt giới hạn tài nguyên cho từng container để một dịch vụ không chiếm hết bộ nhớ của máy.
- Về kiến trúc, nếu cần phục vụ nhiều nút giao trên nhiều máy: thêm tầng cân bằng cho endpoint nhận dữ liệu, bật mã hoá cho giao tiếp nội bộ giữa các máy, và cân nhắc tách kho lưu trữ đối tượng sang dịch vụ chuyên dụng có nhân bản.

Các hướng trên đều khả thi trên nền thiết kế hiện tại vì đồ án đã giữ ranh giới tầng rõ và dùng giao tiếp qua hợp đồng dữ liệu. Việc thêm loại vi phạm mới, thêm một tầng lưu trữ hay thay nhà cung cấp mô hình ngôn ngữ đều không đòi hỏi viết lại các tầng còn lại.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

- [1] Bộ Giao thông Vận tải (2023), Thông tư 24/2023/TT-BGTVT quy định về cấp, thu hồi đăng ký, biển số xe cơ giới, Hà Nội.
- [2] Chính phủ (2021), Nghị định 100/2019/NĐ-CP quy định xử phạt vi phạm hành chính trong lĩnh vực giao thông đường bộ và đường sắt, cùng các văn bản sửa đổi bổ sung, Hà Nội.
- [3] Cục Cảnh sát giao thông, Bộ Công an (2025), Thông tin về hệ thống camera ứng dụng trí tuệ nhân tạo và Trung tâm thông tin chỉ huy, theo nội dung đăng trên báo VietnamNet ngày 25 tháng 9 năm 2025 và báo VOV Giao thông ngày 17 tháng 7 năm 2025.
- [4] Ủy ban An toàn giao thông Quốc gia (2025), Số liệu thống kê tai nạn giao thông đường bộ, Hà Nội.

Sách và bài báo khoa học

- [5] Zhang Y., Sun P., Jiang Y., Nie D., Fakhoury R., Lyu S. (2022), “ByteTrack: Multi-Object Tracking by Associating Every Detection Box”, European Conference on Computer Vision, Springer, trang 1–21.
- [6] Khanam R., Hussain M. (2024), “YOLOv11: An Overview of the Key Architectural Enhancements”, arXiv:2410.17725.
- [7] Bewley A., Ge Z., Ott L., Ramos F., Upcroft B. (2016), “Simple Online and Realtime Tracking”, IEEE International Conference on Image Processing.
- [8] Richardson C. (2018), Microservices Patterns: With Examples in Java, Manning Publications (chương 4 về mẫu quản lý giao dịch phân tán, trong đó có mẫu hộp thư đi).
- [9] Møgelmoose A., Trivedi M. M., Moeslund T. B. (2012), “Vision-Based Traffic Sign Detection and Analysis for Intelligent Driver Assistance Systems”, IEEE Transactions on Intelligent Transportation Systems.
- [10] Jones M. B., Bradley J., Sakimura N. (2015), JSON Web Token (JWT), RFC 7519, Internet Engineering Task Force.
- [11] Fielding R. T. (2000), Architectural Styles and the Design of Network-based Software Architectures, luận án tiến sĩ, Đại học California, Irvine.

Tài liệu kỹ thuật và thư viện mã nguồn mở

- [12] Ultralytics (2026), YOLO tài liệu chính thức, <https://docs.ultralytics.com>.

- [13] Ultralytics (2026), supervision: thư viện công cụ thị giác máy tính, tài liệu bộ theo dõi ByteTrack, <https://supervision.roboflow.com>.
- [14] fast-plate-ocr (2025), thư viện nhận dạng ký tự biến số, mã nguồn công khai trên GitHub.
- [15] Spring Boot 3.3 Reference Documentation, <https://docs.spring.io/spring-boot/docs/3.3.x/reference/html/>.
- [16] Spring Security Reference Documentation, phần bộ lọc và biểu thức phân quyền, <https://docs.spring.io/spring-security/reference/>.
- [17] Next.js 16 Documentation, phần App Router, middleware và viết lại đường dẫn, <https://nextjs.org/docs>.
- [18] PostgreSQL 16 Documentation, phần chỉ mục và ràng buộc duy nhất, <https://www.postgresql.org/docs/16/>.
- [19] MinIO Object Storage Documentation, <https://min.io/docs/minio/linux/index.html>.
- [20] Docker Compose Specification, phần kiểm tra sức khỏe, khai báo thiết bị và volume, <https://docs.docker.com/compose/>.
- [21] FFmpeg Documentation, phần bộ ghép luồng HLS, <https://ffmpeg.org/>.
- [22] hls.js Documentation, thư viện phát luồng HLS trong trình duyệt, <https://github.com/video-dev/hls.js>.
- [23] OWASP Foundation (2025), Cheat Sheet Series, các phần xác thực, quản lý phiên và bảo mật Docker, <https://cheatsheetseries.owasp.org>.

Nguồn dữ liệu và công cụ hỗ trợ

- [24] Bộ dữ liệu ảnh biển số xe dùng để huấn luyện mô hình phát hiện biển số, thu thập từ các nguồn công khai có giấy phép phù hợp.
- [25] Bộ dữ liệu tín hiệu đèn giao thông dùng để huấn luyện mô hình phân loại màu đèn.
- [26] Kroki và PlantUML, dịch vụ dựng sơ đồ từ mô tả dạng văn bản, dùng để sinh các sơ đồ trong đồ án, <https://kroki.io> và <https://plantuml.com>.

Ghi chú về cách trích dẫn: các tài liệu từ số [12] đến số [23] là tài liệu kỹ thuật chính thức của công cụ mà đồ án sử dụng, được tham chiếu ở thời điểm hoàn thiện đồ án là tháng 9 năm 2026. Các thông tin về hệ thống camera ứng dụng trí tuệ nhân tạo của lực lượng cảnh sát giao thông ở mục 2.2 được lấy từ báo chí chính thống trong nước, dẫn lại phát biểu của lãnh đạo Cục Cảnh sát giao thông; đồ án không tiếp cận được tài liệu kỹ thuật nội bộ của hệ thống đó nên chỉ dùng ở mức đối chiếu định tính.